Campus Information System (CIS)

Complete Documentation Bundle

Requirements · ERD · BPMN · Per-role Use Cases, Activity & Sequence Diagrams

FabioH28 — 2026

Overview

Student Information and Course Registration System (CIS)

A full-stack web application for managing academic operations, student services, and institutional administration in a university environment.

Tech Stack

Layer	Technology
Frontend	React 18, TypeScript, Vite
Styling	Tailwind CSS, shadcn/ui (Radix UI)
Routing	React Router v6
Data Fetching	TanStack Query
Forms	React Hook Form + Zod
Charts	Recharts
Animation	Framer Motion
Backend	FastAPI (Python), SQLAlchemy ORM
Database	MySQL 8 / MariaDB 10.4+
Auth	JWT (HS256) + bcrypt (passlib)
Tooling	Vitest, ESLint, Docker Compose

User Roles & Permissions

The system has five roles with row-level access control enforced on the backend.

Feature	Student	Instructor	Academic Staff	Finance Staff	System Admin
View own profile	✅	✅	✅	✅	✅
Edit own profile	✅	✅	✅	✅	✅
Change own password	✅	✅	✅	✅	✅

Feature	Student	Instructor	Academic Staff	Finance Staff	System Admin
View grades	Own only	Own courses	All	❌	All
Edit / publish grades	❌	✅	✅	❌	✅
Mark attendance	❌	✅	✅	❌	❌
Course registration (self)	✅	❌	❌	❌	❌
Approve registrations	❌	❌	✅	❌	✅
Manage course catalog	❌	❌	✅	❌	✅
Manage course offerings	❌	❌	✅	❌	❌
Manage timetable	❌	❌	✅	❌	❌
Manage buildings & rooms	❌	❌	✅	❌	❌
Students list	❌	✅	✅	✅ (financial view)	✅
View payment status	Own only	❌	❌	All	All
Issue invoices	❌	❌	❌	✅	✅
Record payments	❌	❌	❌	✅	✅
Place / resolve holds	❌	❌	View	✅	✅
Create announcements	❌	❌	✅	❌	❌
Create events	❌	❌	✅	❌	❌
News & events feed	✅	❌	✅ (publish)	❌	❌
Send messages	✅	✅	✅	❌	❌
Clubs (join/manage)	✅ (join)	❌	✅ (approve)	❌	❌
Manage semesters	❌	❌	❌	❌	✅
Manage users	❌	❌	❌	❌	✅
System analytics	❌	❌	❌	❌	✅
System settings	❌	❌	❌	❌	✅
Full system access	❌	❌	❌	❌	✅

Repository Structure

```
├── backend/                 # FastAPI backend
│   ├── src/
│   │   ├── config/          # Database connection, settings
│   │   ├── models/          # SQLAlchemy ORM models (30+ tables)
│   │   ├── routes/          # API endpoints grouped by domain
│   │   ├── schemas/         # Pydantic request/response schemas
│   │   ├── utils/           # Auth (JWT, RBAC), audit, email
│   │   └── main.py          # FastAPI app entry
│   ├── tests/               # Pytest unit & integration tests
│   ├── .env.example         # Environment variable template
│   └── requirements.txt     # Python dependencies
├── database/
│   ├── schema.sql           # MySQL table definitions
│   └── seed.sql             # Demo users, courses, grades
├── frontend/
│   └── src/
│       ├── components/      # Shared UI (sidebar, top bar, shadcn primitives)
│       ├── contexts/        # AuthContext (token + role state)
│       ├── hooks/           # Reusable hooks (use-toast, use-mobile)
│       ├── layouts/         # Per-role layout shells
│       ├── lib/             # Typed API client, role helpers
│       └── pages/
│           ├── student/         # 17 pages
│           ├── instructor/      # 9 pages
│           ├── academic-staff/  # 10 pages (incl. coworker module)
│           ├── finance-staff/   # 6 pages
│           └── admin/           # 8 pages
├── docs/                    # Architecture, ERD, demo credentials
├── docker-compose.yml       # MySQL + backend + frontend
├── DEVELOPER_NOTES.md       # Run quickstart, gotchas, module map
└── README.md
```

Getting Started

Prerequisites

- Node.js 18+
- Python 3.11+
- MySQL 8.0+ (or XAMPP MariaDB)

1. Database Setup

Create the database and load the schema + seed:

```
mysql -u root -p < database/schema.sql
mysql -u root -p CampusIS < database/seed.sql
```

Or open both files in MySQL Workbench and execute them in order.

> **XAMPP note:** if you use XAMPP locally, MariaDB defaults to port `3307`. Adjust `DATABASE_URL` and the `mysql -P 3307` flag accordingly.

2. Backend Setup

```
 cd backend

# Create virtual environment
python -m venv .venv

# Activate it
.venv\Scripts\activate         # Windows
source .venv/bin/activate      # Mac/Linux

# Install dependencies (python-jose and bcrypt 4.0.1 are required pins)
pip install -r requirements.txt "python-jose[cryptography]" "bcrypt==4.0.1"

# Configure environment
copy .env.example .env         # Windows
cp .env.example .env           # Mac/Linux
# Then edit .env with your MySQL credentials

# Start the server
uvicorn src.main:app --reload --port 8000
```

- API runs at `http://127.0.0.1:8000`
- Interactive docs at `http://127.0.0.1:8000/docs` (Swagger UI / OpenAPI)
- Health check at `http://127.0.0.1:8000/health`

Environment Variables

Copy `backend/.env.example` to `backend/.env` and fill in:

Variable	Description
`DB_HOST`	MySQL host (default `localhost`)
`DB_PORT`	MySQL port (`3306` or `3307` for XAMPP)
`DB_NAME`	Database name (`CampusIS`)
`DB_USER`	MySQL user
`DB_PASSWORD`	MySQL password
`DATABASE_URL`	Full SQLAlchemy URL (overrides the four above if set)
`JWT_SECRET_KEY`	JWT signing secret — generate with `openssl rand -hex 32`
`JWT_ALGORITHM`	Signing algorithm (default `HS256`)
`JWT_EXPIRE_MINUTES`	Token lifetime (default `480`)
`SMTP_HOST`	SMTP server for verification/reset emails (optional)
`SMTP_PORT`	SMTP port

Variable	Description
`SMTP_USER`	SMTP login
`SMTP_PASSWORD`	SMTP credential

3. Frontend Setup

```
 # From the repo root (Vite project lives at the root, sources in frontend/src)
npm install
npm run dev -- --host 127.0.0.1 --port 8088
```

Frontend runs at `http://127.0.0.1:8088`.

The frontend reads `VITE_API_BASE_URL` from the root `.env` (defaults to `http://127.0.0.1:8000`).

4. Login Credentials (development)

All seeded accounts use password: **`password123`**

Role	Email
Student	`alice.smith@cis.edu`
Instructor	`john.carter@cis.edu`
Academic Staff	`rebecca.morgan@cis.edu`
Finance Staff	`finance.csit@cis.edu`
System Admin	`admin@cis.edu`

Additional seeded accounts (students, instructors, staff per faculty) are listed in docs/DEMO_LOGIN_CREDENTIALS.txt. All use the same password.

> **Production note:** the seeded password is for local development only. Rotate `JWT_SECRET_KEY` and reset all account passwords before deploying.

Docker Option

Bring up MySQL, backend, and frontend with one command:

```
docker compose up --build
```

Services:

- Frontend → `http://localhost:8088`
- Backend → `http://localhost:8000`
- MySQL → `localhost:3306`

The Docker MySQL container is initialized from `database/schema.sql` and `database/seed.sql`.

API Overview

The backend exposes 80+ REST endpoints across these areas:

Area	Prefix	Notes
Authentication	`/auth`	Login, register, email verify, password reset, change password
Students	`/students`	Profile self-service + admin list/edit
Courses	`/courses`	Catalog CRUD (academic_staff + admin)
Offerings	`/offerings`	Section instances per semester
Registrations	`/registrations`	Enrollment + status transitions
Course selections	`/api/student/course-selections`	Student request → staff approve
Grades	`/grades`	Config, upsert, publish
Attendance	`/attendance`	Sessions + per-student records
Materials	`/api/teacher/materials`, `/api/student/...`	Weekly course materials, files
Assignments	`/api/teacher/assignments`, `/api/student/...`	Briefs + submissions
Timetable	`/api/student/timetable`, `/api/teacher/timetable`	Weekly schedule
Staff Schedule	`/api/staff`	Offerings, timetable, rooms, selection approvals
Finance	`/finance`	Invoices, payments, holds
Notifications	`/notifications`	In-app inbox
Announcements	`/communications/announcements`	Staff broadcast
Events	`/communications/events`	Campus events + registration
Clubs	`/clubs`, `/communications/clubs`	Directory, join, approve
Messages	`/messages`	Direct messaging
Users (admin)	`/users`	Create, approve pending, reset, disable
Semesters	`/semesters`	Academic terms

Interactive Swagger UI at `http://127.0.0.1:8000/docs` lists everything with payload schemas.

Architecture Highlights

- **Role-Based Access Control** — every protected route uses `require_roles(...)` against a JWT claim. Role aliases (`teacher` → `instructor` , `staff` → `academic_staff` , `admin` → `system_admin`) are normalized via `canonical_role()` .
- **Faculty scoping** — finance and academic staff only see invoices/students within their assigned faculty (`FinanceFacultyScope` , `StaffFacultyScope`).
- **Auto-create migrations** — `ensure_new_feature_tables` startup hook in `backend/src/main.py` creates new tables on existing databases without manual migration.
- **Communications module** — announcements, campus events, clubs, and direct messaging share a single domain with role-aware endpoints.
- **Type-safe API client** — `frontend/src/lib/api.ts` exposes typed wrappers (`gradesApi` , `financeApi` , etc.) consumed by every page.

Conventions

- Backend ORM PK/FK columns use `UnsignedInteger` (matches `users.id` INT UNSIGNED) — required for FK type compatibility.
- Albanian grade scale (0–10) applies to course grades; GPA is on a 0–4 scale.
- Frontend pages auto-detect GPA scale via `gpaScale()` in `lib/utils.ts` .

Further Documentation

- DEVELOPER_NOTES.md — run quickstart, gotchas, communications module file map
- docs/DEMO_LOGIN_CREDENTIALS.txt — all seeded accounts
- docs/erd_dbdiagram.dbml — entity-relationship diagram (open at dbdiagram.io)
- CONTRIBUTING.md — branch, test, and release guidelines
- CHANGELOG.md — versioned changes

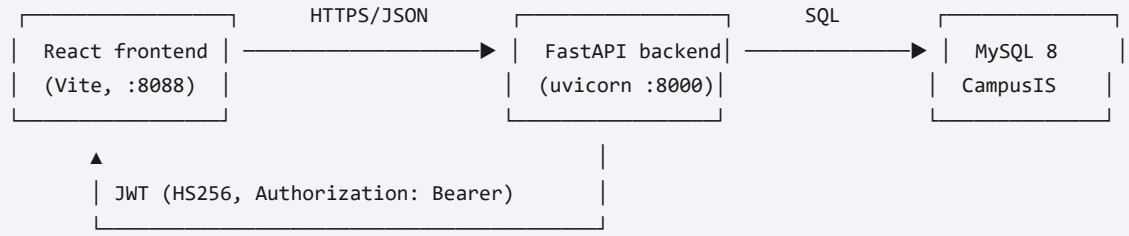
Architecture

CIS — Architecture

Reference document for the Campus Information System.

1. System overview

A three-tier web application:



- **Frontend** is a single-page React application served by Vite. State is managed by TanStack Query; auth state lives in `AuthContext`.
- **Backend** is a FastAPI service. SQLAlchemy ORM handles persistence; passlib + bcrypt hash passwords; python-jose signs/validates JWTs.
- **Database** is MySQL 8 (or MariaDB 10.4+ via XAMPP).

2. Roles & RBAC

Five canonical roles, with aliases normalized server-side:

Canonical	Aliases	Home route
`student`	—	`/student`
`instructor`	`teacher`, `professor`	`/instructor`
`academic_staff`	`staff`	`/academic-staff` (also `/staff`)
`finance_staff`	—	`/finance-staff`
`system_admin`	`admin`	`/admin`

Every protected endpoint declares its allowed roles via `Depends(require_roles(...))` in `backend/src/utils/security.py`. The frontend mirrors this with `RequireAuth` route guards and `canonicalRole()` in `frontend/src/lib/authRoles.ts`.

Additionally, `finance_staff` and `academic_staff` are scoped to their assigned faculties via `FinanceFacultyScope` / `StaffFacultyScope` join tables.

3. Data model (high-level)

Detailed ERD: see erd_dbdiagram.dbml.

Core entities and their relationships:

```
users ──┬── students ────┬── registrations ── grades
        │                ├── student_course_selections
        │                ├── attendance_records
        │                ├── invoices ── payments
        │                └── holds
        ├── instructors ── offerings ── timetable_entries ── class_sessions
        ├── staff_profiles
        └── ai_chat_sessions ── ai_chat_messages

courses ── offerings (per semester)
courses ── course_prerequisites
courses ── course_materials, weekly_topics, assignments ── assignment_submissions

faculties ── departments ── programs ── courses
faculties ── buildings ── classrooms
faculties ── student_groups

clubs ── club_memberships
campus_events ── event_registrations
announcements
notifications
messages
```

PK/FK columns use `INT UNSIGNED` (matches `users.id`); ORM models use a `UnsignedInteger` SQLAlchemy variant.

4. Module / page map

Domain	Backend routes	Frontend pages
Auth	`/auth/*`	`LoginPage`, `RegisterPage`, `VerifyEmailPage`, `ChangePasswordPage`
Student dashboard	`/students/me`, `/api/student/*`	17 pages under `pages/student/`
Instructor	`/api/teacher/*`, `/api/instructor/profile`, `/grades/*`, `/attendance/*`	9 pages under `pages/instructor/`
Academic staff	`/api/staff/*`, `/courses`, `/students`, `/communications/*`	10 pages under `pages/academic-staff/`
Finance	`/finance/*`	6 pages under `pages/finance-staff/`

Domain	Backend routes	Frontend pages
Admin	`/users`, `/semesters`, all of the above	8 pages under `pages/admin/`
Communications	`/communications/*` (announcements, events, clubs, dashboard)	`StaffCommunications`, student `News`, `Clubs`, `Inbox`

5. Key technical decisions

- **JWT in localStorage** — tokens stored client-side, sent in `Authorization` header. `JWT_EXPIRE_MINUTES=480` (8 hours).
- **bcrypt 4.0.1 pin** — newer bcrypt versions break passlib's bcrypt handler with the "password cannot be longer than 72 bytes" error.
- **TanStack Query for fetching** — pages use typed wrappers (`gradesApi`, `financeApi`, etc.) from `lib/api.ts`. Caching + invalidation per mutation.
- **shadcn/ui** — Radix primitives + Tailwind via `class-variance-authority`. No external UI library lock-in.
- **Auto-create migrations** — `ensure_new_feature_tables` on FastAPI startup creates any new ORM tables that aren't yet in the DB, so feature branches don't need manual migrations.
- **Communications consolidated into `academic_staff`** — there is intentionally no separate "communications staff" role; the academic staff handles announcements, events, and club approvals.

6. Security model

- **Authentication** — JWT signed with `JWT_SECRET_KEY` (HS256). Configurable lifetime.
- **Authorization** — `require_roles(*roles)` dependency injection. Aliases normalized via `canonical_role()`.
- **Password storage** — bcrypt via passlib `CryptContext`. No plaintext anywhere.
- **Email verification + password reset** — token-based flows in `email_verification_tokens` / `password_reset_tokens`.
- **Audit log** — `AuditLog` model captures sensitive mutations.
- **Faculty scoping** — finance and staff queries are filtered to their assigned faculties.

Known production hardening required

- Rotate the default `JWT_SECRET_KEY` placeholder before deploying.
- Tighten `CORSMiddleware` from `allow_origins=["*"]` to the actual frontend origin.
- Reseed all demo accounts with strong unique passwords.

7. Folder layout

```
backend/src/
  config/      database + settings
  models/      30+ SQLAlchemy ORM models
```

```
  routes/      domain-grouped FastAPI routers
  schemas/     Pydantic request/response shapes
  utils/       security (JWT, RBAC), audit, email
  main.py      app entry, startup hooks

frontend/src/
  components/  shared UI (sidebar, top bar, shadcn primitives)
  contexts/    AuthContext
  hooks/       use-toast, use-mobile
  layouts/     per-role layout shells
  lib/         typed API client, role helpers, utils
  pages/       role-grouped page components
```

8. Local development quickstart

See README.md and DEVELOPER_NOTES.md for setup, gotchas, and per-module file maps.

9. Test strategy

- Backend tests live in `backend/tests/` (pytest). Existing coverage: `test_security`, `test_rbac`, `test_app`.
- Frontend tests via Vitest (`npm run test`). Currently no UI tests in the suite.
- End-to-end smoke testing has been validated by hand against the live API for every role.

Requirements

CIS — Requirements Specification

This document specifies the functional and non-functional requirements of the Campus Information System (CIS). It is the canonical, browsable companion to docs/requirements/index.html.

1. System Overview

CIS is a full-stack web platform that manages academic operations, student services, and institutional administration. It supports five user roles (Student, Instructor, Academic Staff, Finance Staff, System Admin) with row-level access control enforced on the backend.

Layer	Technology
Frontend	React 18 + TypeScript + Vite + Tailwind + shadcn/ui
Backend	FastAPI (Python) + SQLAlchemy
Database	MySQL 8 / MariaDB 10.4+
Auth	JWT (HS256) + bcrypt

2. Stakeholders and Actors

Actor	Role
Student	Enrolls in courses, submits assignments, views grades, reads notifications, joins clubs, contacts staff and instructors.
Instructor	Manages assigned courses, weekly topics, materials, assignments, attendance, and grades.
Academic Staff	Owns the academic catalog: courses, offerings, timetable, buildings/rooms, registration approvals, semester windows.
Communications Staff	Publishes announcements, manages events, approves clubs, broadcasts. (Same backend role as academic staff; conceptually separated for governance.)
Finance Staff	Issues invoices, records payments, places/releases holds, reviews student balances.
System Admin	Provisions users, manages semesters, reviews analytics, configures system settings.

3. Functional Requirements

3.1 Access Control & Identity

ID	Requirement	Priority
FR-001	Administrators shall provision Student, Instructor, and staff accounts using institution-issued email addresses.	High
FR-002	The system shall enforce role-based access so that each role only sees functions within its authorization scope.	High
FR-003	First-time users shall be required to change their temporary password before accessing role workspaces.	High
FR-004	The system shall support password reset request and confirmation using short-lived tokens.	High
FR-020	Administrators shall be able to activate, suspend, or reset accounts without direct database intervention.	High

3.2 Academic Structure & Operations

ID	Requirement	Priority
FR-005	The system shall maintain departments, programs, academic terms, courses, and course offerings.	High
FR-006	Administrators / academic staff shall assign instructors to specific course offerings.	High
FR-007	Students shall browse the course catalog and view recommendation cues during registration.	Medium
FR-008	Students shall add or drop eligible course registrations subject to prerequisite, capacity, hold, and deadline rules.	High
FR-009	Students shall view a timetable derived from their confirmed registered offerings.	Medium
FR-010	Instructors shall create attendance sessions and mark attendance for students enrolled in their assigned offerings.	High
FR-011	Instructors shall define grade components, record marks, and publish final grade outcomes for assigned offerings.	High
FR-012	Students shall view their academic profile, attendance-related alerts, and published grades.	High
FR-021	Instructors shall have a workspace presenting assigned courses, rosters, attendance, gradebook, and inbox.	High

3.3 Communications & Campus Services

ID	Requirement	Priority
FR-013	The system shall maintain an inbox aggregating warnings, academic notices, finance reminders, and campus notifications.	High
FR-014	Administrators / academic staff shall publish announcements and events visible to relevant users.	High
FR-015	Students shall browse campus clubs and submit join requests or event participation requests.	Medium
FR-016	Administrators shall create and manage club records and review membership requests.	Medium

3.4 Finance

ID	Requirement	Priority
FR-017	Administrators shall issue student invoices, record payments, and apply or release finance holds.	High
FR-018	Students shall inspect finance statements and submit finance support requests.	Medium

3.5 Platform & Reporting

ID	Requirement	Priority
FR-019	The system shall maintain audit records for security-sensitive actions (login, password reset, provisioning, role changes).	High
FR-022	The system shall provide administrative reporting summaries for academic, finance, user, and engagement modules.	Medium
FR-023	The system shall store AI chat sessions and messages for student support interactions where institutional policy permits.	Planned
FR-024	The system shall expose stable REST API endpoints for frontend consumption across all domains.	High

4. Non-Functional Requirements

ID	Category	Requirement	Target
NFR-001	Security	All authenticated API calls shall require verified tokens and enforce server-side authorization.	Mandatory
NFR-002	Security	Passwords shall be stored only as secure hashes, never reversible.	Mandatory

ID	Category	Requirement	Target
NFR-003	Security	The platform shall provide login throttling or temporary lockout after repeated failed attempts.	Mandatory
NFR-004	Performance	Dashboards and list views shall load under typical user-perceived latency.	Sub-2s for common queries
NFR-005	Availability	The deployment shall support routine backup, restart, and recovery without data corruption.	High priority
NFR-006	Data Integrity	Relational constraints shall prevent duplicate or contradictory academic records.	Mandatory
NFR-007	Auditability	Security-sensitive actions shall be traceable through timestamped audit records.	Mandatory
NFR-008	Usability	The interface shall be role-appropriate, mobile-responsive, and understandable to non-technical users.	Mandatory
NFR-009	Accessibility	The platform should align with accessible semantic structure, readable contrast, and keyboard-operable workflows.	WCAG-aligned
NFR-010	Maintainability	Frontend, backend, and database layers shall be modular enough to evolve independently.	Mandatory
NFR-011	Scalability	The architecture shall permit growth in users, records, and modules using conventional scaling.	Institution-scale
NFR-012	Interoperability	The platform shall use stable REST contracts and environment-driven configuration.	Mandatory
NFR-013	Privacy	Users shall only see personally identifiable or academic data they are explicitly authorized for.	Mandatory
NFR-014	Reliability	Validation failures and operational errors shall return predictable responses without exposing internals.	Mandatory
NFR-015	Deployment	The application shall support containerized or equivalent repeatable deployment.	Preferred baseline

5. Business Rules

- Accounts are created by administrators; self-registration is outside scope.
- Student identifiers and employee identifiers are system-generated.
- Students may only register for offerings in accessible academic structures that satisfy prerequisite / policy rules.
- Students may not message other students; the messaging dropdown restricts students to instructors and academic staff.
- Instructors may only manage attendance and grades for offerings explicitly assigned to them.
- Attendance for past-dated sessions is locked once entered; corrections require fresh sessions.

- Assignment creation requires a real `class_session_id` whose week matches the assignment's `week_number`.
- Published final grades are authoritative unless superseded by approved administrative correction.
- Drop is enforced server-side against `semester.drop_deadline`.
- Required-subject selection enforces the strict program/year/prereq gate; elective offerings bypass program/year (same-faculty only).
- Finance holds may restrict downstream student actions such as registration if institutional policy requires.
- Announcements and events shall be attributable to an authorized publisher with publication metadata.
- Notifications shall be linked to a recipient or recipient group and support unread / read / archived state.
- Password reset and password change actions shall invalidate vulnerable session states.
- Critical user lifecycle operations shall leave auditable evidence.

6. Interfaces and Integrations

Interface	Purpose
REST API (FastAPI)	Frontend ↔ backend contract; 80+ endpoints across auth, students, courses, offerings, registrations, grades, attendance, finance, notifications, announcements, events, clubs, messages, users, semesters.
MySQL (SQLAlchemy)	Persistent academic, finance, and communications data.
SMTP (planned)	Email verification, password reset codes.
Ollama (optional)	Local LLM for the AI chatbot; deterministic fallback if unavailable.

7. Data and Reporting

- Albanian grade scale (0-10) for course grades; GPA on a 0-4 scale.
- Total credits and duration per program defined in `programs` table.
- Reporting surfaces: Admin Analytics, Finance Dashboard, Instructor Dashboard, Academic Staff Dashboard.
- Demo seed: ~201 users across 4 faculties and 19 programs.

8. Assumptions and Constraints

- Single-institution deployment per instance.
- All times stored in server local time; week numbers reference the active semester's start.
- The AI Chatbot is best-effort and clearly labelled; institutional answers come from the database.
- Frontend assumes modern evergreen browsers (Chromium, Firefox, Safari current versions).

9. Acceptance Position

The system is considered acceptance-ready when:

1. All HIGH-priority functional requirements are exercised end-to-end via the live UI.
2. RBAC restrictions in Section 4 of README.md are enforced on the backend (not only hidden in the UI).
3. The seed-data demo flow (5 demo accounts) reproduces the documented scenarios in docs/roles/.
4. Lint, build, and backend test suites are green.

10. Related Documents

- ARCHITECTURE.md — system architecture and module map
- ERD.md — entity relationship diagram + domain notes
- erd_dbdiagram.dbml — full ERD source (dbdiagram.io)
- roles/ — per-role overviews, scenarios, and diagrams
- DEMO_LOGIN_CREDENTIALS.txt — demo accounts
- requirements/Requirements Specifications.docx — original long-form specification (Word document)

Entity-Relationship Diagram

CIS — Entity Relationship Diagram

48 tables across 10 domains. Renders inline on GitHub. For a click-to-edit visual version, paste erd_dbdiagram.dbml at dbdiagram.io.

Core diagram

[Mermaid diagram — see PNG in diagrams/ folder]

Domain breakdown

1. Identity & Security (10 tables)

`users` is the root. Every role inherits via 1:1: `students`, `instructors`, `staff_profiles`. Faculty scopes (`staff_faculty_scopes`, `finance_faculty_scopes`) limit which records non-admin staff can see. Auth flows use `email_verification_tokens` + `password_reset_tokens`. All sensitive actions are tracked in `audit_logs`.

2. Org Structure (7 tables)

`faculties → departments → programs` is the org tree. Programs are majors students enroll into. `semesters` defines terms; `buildings → classrooms` is the physical campus inventory; `groups` are class sections within a program.

3. Catalog (2 tables)

`courses` is the catalog. `course_prerequisites` is the gating graph.

4. Offerings & Scheduling (3 tables)

`offerings` = `(course + instructor + semester + program)` — the central join. Unique constraint on those 4 ensures no duplicate offerings. `timetable_entries` adds the weekly room+time. `class_sessions` materializes each meeting date.

5. Enrollment (3 tables)

Students request via `student_course_selections` → academic staff approves → row added to `registrations` (the source of truth for "enrolled in"). `student_course_status` is a derived flat record of pass/fail per course per student.

6. Academic Content (6 tables)

Weekly structure: `weekly_topics` / `course_week_topics` are the topic per week. `course_materials` (files/links/videos/text) and `assignments` are tied to a week. `assignment_submissions` are student deliverables. `weekly_tasks` are non-graded weekly TODOs.

7. Attendance & Grades (4 tables)

`attendance_sessions` per offering, `attendance_records` per student per session. `course_grade_configurations` holds the per-offering grading components (midterm/project/quiz/final percentages). `grades` is per registration with all component scores + computed total + pass/fail + exam-block flag.

8. Finance (3 tables)

`invoices` billed per (student, semester). `payments` are partial-allowed (status auto-updates `unpaid → partial → paid`). `holds` are restrictions placed by finance staff with configurable `effect` (block_registration / block_grades / block_transcript / warning).

9. Communications (8 tables)

`announcements` and `notifications` for one-way broadcast. `messages` for direct DM. `clubs` + `club_memberships` for student orgs (open / request / waitlist join modes). `campus_events` + `event_registrations` for events.

10. AI Assistant (2 tables)

`ai_chat_sessions` + `ai_chat_messages` for the student chatbot (which uses Ollama locally — see `Chatbot.tsx`).

Naming & integrity notes

- All PK/FK columns are `INT UNSIGNED` to match `users.id`. ORM models use a custom `UnsignedInteger` SQLAlchemy variant.
- Cascade deletes are scoped: deleting a `users` row cascades to `students` / `instructors` / `staff_profiles`, but offerings/grades are preserved (instructors deletion blocked by FK to offerings).
- Status fields use string varchars (not enums) so new states can be added without migrations.
- Faculty scope filtering is enforced server-side via `_finance_faculty_ids()` / `_staff_faculty_ids()` helpers in `routes/finance.py` and `routes/staff.py`.

Source of truth

- Live DB schema: `database/schema.sql`
- ORM models: `backend/src/models/`
- This document is regenerated from those — keep them in sync.

Business Process Diagrams

CIS — Business Process (BPMN) Diagrams

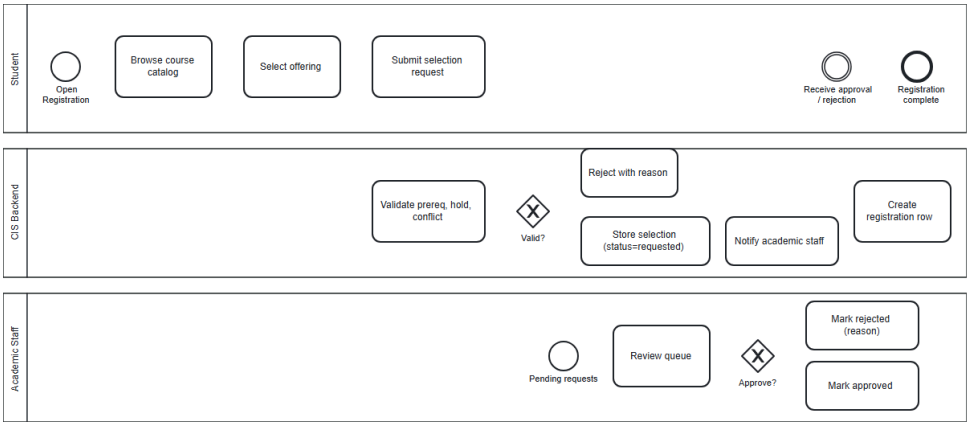
Five BPMN 2.0 collaboration diagrams covering the core institutional business processes of CIS. Each diagram includes pools, lanes, message flows, gateways, and end events per BPMN 2.0 notation.

#	Process	Source	SVG	PNG
1	Course registration (selection → approval → registration)	01-course-registration.bpmn	SVG	PNG
2	Grade publication (configure → enter → publish → student view)	02-grade-publication.bpmn	SVG	PNG
3	Invoice → payment → hold → release	03-invoice-payment-hold.bpmn	SVG	PNG
4	Announcement publication & broadcast	04-announcement-broadcast.bpmn	SVG	PNG
5	User onboarding (admin provisioning → first login → password change)	05-user-onboarding.bpmn	SVG	PNG

Viewing

- **In the browser / GitHub**: click any `.svg` link above — they render inline.
- **As fallback / for printing**: each `.png` is a static raster of the same diagram.
- **For editing**: open any `.bpmn` file in bpmn.io (drag & drop), the Camunda Modeler, or any BPMN 2.0–compliant tool. The XML is valid BPMN 2.0.

Diagram 1 — Course Registration

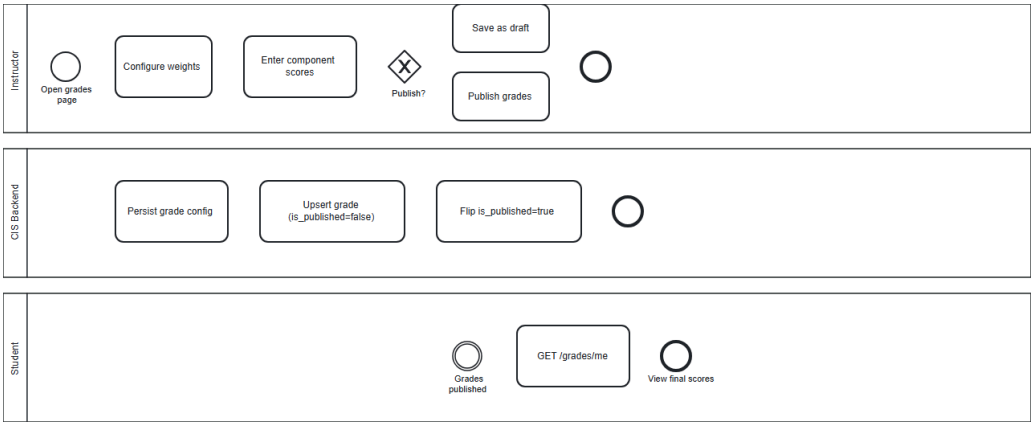


Pools: Student · CIS Backend · Academic Staff

Flow summary: Student browses catalog → submits selection → backend validates (prereq / hold / conflict) → stores as `requested` → notifies academic staff → staff approves/rejects → on approval, backend creates a `registrations` row and notifies the student.

Maps to: docs/roles/student/sequence-diagrams.md#S2, docs/roles/academic-staff/sequence-diagrams.md#S1, backend course_selections.py.

Diagram 2 — Grade Publication



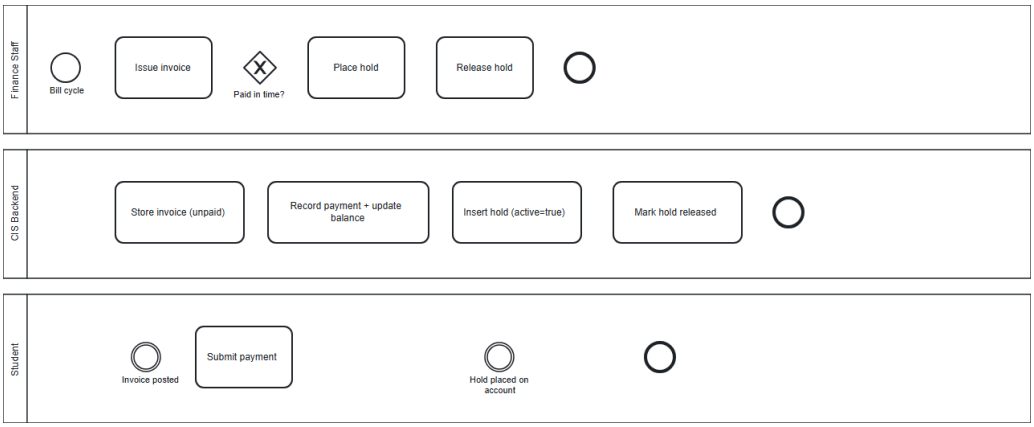
Pools: Instructor · CIS Backend · Student

Flow summary: Instructor configures weight components → enters per-registration component scores (saved as `is_published=false`) → chooses to publish now or later → on publish, backend flips

`is_published=true` → student views final scores via `GET /grades/me`.

Maps to: docs/roles/instructor/sequence-diagrams.md#S3, backend grades.py.

Diagram 3 — Invoice → Payment → Hold

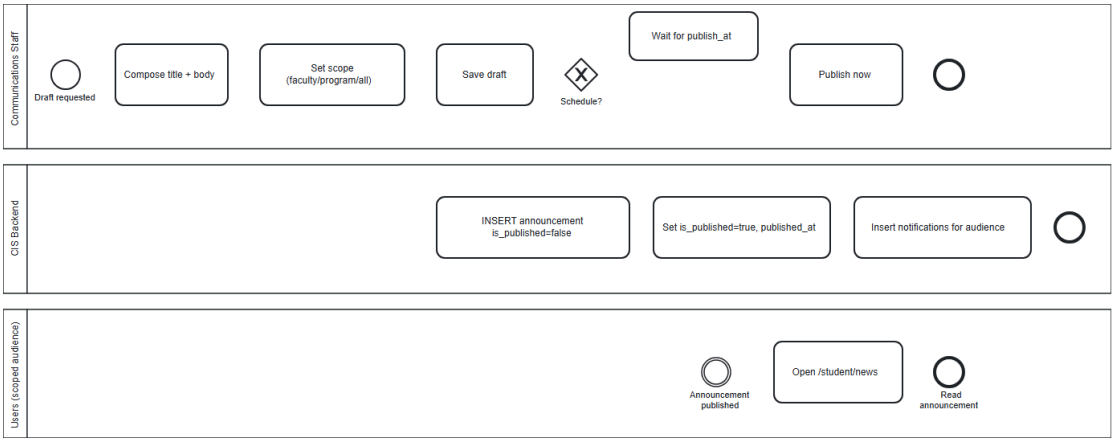


Pools: Finance Staff · CIS Backend · Student

Flow summary: Finance issues invoice → student receives → if paid in time, end. If overdue, finance places hold (which the backend then enforces against new course-selection submissions) → after payment, finance releases hold.

Maps to: docs/roles/finance-staff/sequence-diagrams.md#S2, backend finance.py.

Diagram 4 — Announcement Publication

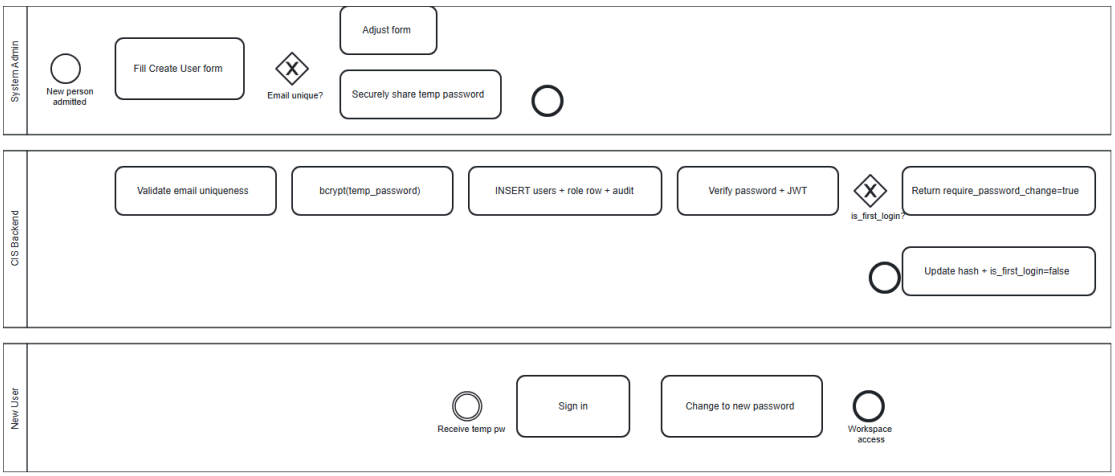


Pools: Communications Staff · CIS Backend · Users (scoped audience)

Flow summary: Comms staff composes title + body + scope → saves draft → chooses to publish now or schedule → on publish, backend flips `is_published=true` and writes notifications targeted at the configured scope (all / faculty / program) → users see the announcement at `/student/news` and in their inbox.

Maps to: [docs/roles/communications-staff/sequence-diagrams.md#S1](), backend [communications.py]().

Diagram 5 — User Onboarding



Pools: System Admin · CIS Backend · New User

Flow summary: Admin fills create-user form → backend validates email uniqueness → bcrypt-hashes temp password → inserts users + role-specific row → records audit log → admin securely shares

credentials → user signs in → backend returns `require_password_change=true` → user changes password → `is_first_login` flips to false → normal workspace access.

Maps to: docs/roles/system-admin/sequence-diagrams.md#S1, backend users.py + auth.py.

Notation cheat sheet (BPMN 2.0)

Symbol	Meaning
Circle (thin border)	Start event
Circle (double border)	Intermediate event (message catch)
Circle (thick border)	End event
Rounded rectangle	Task / activity
Diamond	Gateway (exclusive XOR by default)
Solid arrow	Sequence flow (within a pool)
Dashed arrow	Message flow (between pools)
Horizontal swimlane	Pool (one per actor)

Roles — Index

CIS — Role Documentation

Each subfolder documents one user role with overview, user scenarios, use case diagrams, activity diagrams, and sequence diagrams.

Role	Folder	Primary Responsibilities
Student	student/	Course registration, timetable, materials, assignments, grades, messaging, clubs
Instructor	instructor/	Course delivery, weekly materials, assignments, attendance, grading
Academic Staff	academic-staff/	Course catalog, offerings, timetable, buildings/rooms, registration approvals
Communications Staff	communications-staff/	Announcements, events, clubs, broadcast messages
Finance Staff	finance-staff/	Invoices, payments, holds, student balances
System Admin	system-admin/	Users, semesters, analytics, settings

> **Note on Academic Staff vs Communications Staff.** In the codebase these are the same backend role (`academic_staff`). They are separated in this documentation because the institutional responsibilities and screens differ enough to be treated as distinct roles for governance, training, and acceptance review.

Each role folder contains:

File	Purpose
`README.md`	Role overview, responsibilities, screens, key endpoints, permissions matrix
`user-scenarios.md`	Narrative scenarios walking through real tasks step by step
`use-cases.md`	Use case diagram (Mermaid) + use case descriptions
`activity-diagrams.md`	Activity diagrams (Mermaid) for the role's key workflows
`sequence-diagrams.md`	Sequence diagrams (Mermaid) showing system interactions
`diagrams/`	Pre-rendered PNG images of every diagram in this role (no Mermaid renderer needed)

All diagrams are written in Mermaid and render natively in GitHub-flavored markdown — no external tooling required to view them.

Demo accounts (password `password123`)

Role	Email
Student	`alice.smith@cis.edu`
Instructor	`john.carter@cis.edu`
Academic Staff	`rebecca.morgan@cis.edu`
Finance Staff	`finance.csit@cis.edu`
System Admin	`admin@cis.edu`

See DEMO_LOGIN_CREDENTIALS.txt for all seeded accounts.

Student

Student Role

The Student is the primary end-user of CIS. Each student is enrolled in one program at a faculty, and progresses through semesters by registering for course offerings, attending classes, submitting work, and receiving grades.

Responsibilities

- View personal academic profile, GPA, and progression status
- Browse the course catalog and view available subjects each semester
- Submit course-selection requests (academic-staff approves)
- Add and drop registrations within open enrollment / drop windows
- View weekly timetable derived from registered offerings
- Access weekly materials uploaded by instructors
- Submit assignments and review feedback / grades
- View personal attendance and absence percentage
- View published grades and risk warnings
- Receive notifications and messages (inbox)
- Send direct messages to instructors and academic staff
- Join campus clubs and register for campus events
- View finance statement (invoices, balance, holds)

Screens (Frontend Routes)

Route	Page
`/student`	Dashboard
`/student/profile`	Profile
`/student/courses`	My Courses
`/student/courses/:offeringId`	Course Detail (materials, assignments, attendance, grades per course)
`/student/registration`	Course Registration
`/student/available-subjects`	Available Subjects (per semester)
`/student/course-selections`	Selected Subjects (pending / approved selections)
`/student/timetable`	Weekly Timetable
`/student/materials`	Course Materials

Route	Page
`/student/assignments`	Assignments
`/student/attendance`	Attendance Summary
`/student/grades`	Grades + GPA
`/student/news`	Announcements & Events
`/student/clubs`	Clubs Directory
`/student/inbox`	Notifications + Direct Messages
`/student/risk`	Risk Warnings (failing grades / high absence)
`/student/chatbot`	AI Assistant
`/student/change-password`	Change Password

Key API Endpoints

Endpoint	Purpose
`POST /auth/login`	Sign in
`POST /auth/change-password`	Change own password
`GET /api/student/timetable`	Weekly schedule
`GET /api/student/courses`	Registered courses
`GET /api/student/courses/{offering}/weeks/{w}/materials`	Week-scoped topic, materials, tasks
`POST /api/student/course-selections`	Submit selection request
`POST /registrations/drop`	Drop a registration (within deadline)
`GET /grades/me`	Published grades
`GET /messages/inbox`	Inbox
`POST /messages`	Send direct message (to instructor or academic staff only)
`GET /messages/contacts`	List of valid recipients
`GET /notifications`	Notification list

Permissions Matrix

Action	Allowed?
View / edit own profile	✅
Change own password	✅
View own grades	✅ (published only)
View other students' grades	❌
Self-register for offerings	✅ (subject to rules)
Approve registrations	❌
Edit / publish grades	❌
Send direct message to instructor / academic staff	✅
Send direct message to other student	❌
Send direct message to finance / admin	❌
Create announcement / event	❌
View finance balance (own)	✅
Issue invoices	❌

Related Diagrams

- user-scenarios.md — task-oriented walkthroughs
- use-cases.md — use case diagram
- activity-diagrams.md — workflow diagrams
- sequence-diagrams.md — system interaction diagrams
- diagrams/ — rendered PNGs of every Mermaid diagram (auto-generated)

Student — User Scenarios

Student — User Scenarios

Each scenario is a narrative walk-through of a real student task. Used both as acceptance fixtures and as training material.

Scenario 1: First-time login and password change

Actor: Alice Smith (new student, account just provisioned by admin) **Precondition:** Admin has created Alice's account with a temporary password. `is_first_login = true`.

Steps: 1. Alice opens `http://campus-portal/` in her browser. 2. Enters `alice.smith@cis.edu` and her temporary password. 3. Login succeeds. Backend returns `require_password_change: true`. 4. Frontend redirects Alice to `/student/change-password`. 5. Alice enters her current temporary password, a new password (≥8 chars), and confirms. 6. System hashes the new password (bcrypt), clears `is_first_login`, audit-logs `CHANGE_PASSWORD`. 7. Frontend redirects to `/student` (dashboard).

Postcondition: Alice has a personal password; she sees the student dashboard.

Scenario 2: Registering for a required course

Actor: Alice Smith (Bachelor Year 1, Software Engineering, Spring 2026) **Precondition:** Spring 2026 registration window is open. Alice has no finance hold. Prerequisite is satisfied.

Steps: 1. Alice goes to `/student/registration`. 2. The page shows offerings for her active semester (Spring 2026). The system filters to her program/year for required subjects. 3. Alice clicks **Add** on `CS101 – Introduction to Programming`. 4. Frontend posts `POST /api/student/course-selections { course_offering_id: 1 }`. 5. Backend validates: same semester, program/year match (for required), no prerequisite gap, no timetable conflict with already-approved registrations, no finance hold. 6. Selection row is created with `status = 'requested'`. Academic staff is notified. 7. UI shows the selection on `/student/course-selections` with **Pending** badge.

Postcondition: A pending selection awaits academic-staff approval. After approval, a `registration` row is created and CS101 appears on Alice's timetable.

Alternate flow — rejection: If a prerequisite is missing or capacity is full, the backend returns 400 with a human-readable reason; the UI shows the reason inline.

Scenario 3: Dropping a course before the drop deadline

Actor: Alice Smith **Precondition:** Alice has an active registration for `BIO110`. Drop deadline is 2026-06-21 (open).

Steps: 1. Alice goes to `/student/courses` and clicks **Drop** on BIO110. 2. Confirmation dialog ("Are you sure?") appears. 3. Alice confirms. Frontend posts `POST /registrations/drop { registration_id }`. 4. Backend validates: registration belongs to Alice, current date is on or before `semester.drop_deadline`. 5. Registration `status` is updated to `dropped`. Timetable entries for BIO110 disappear from Alice's view. 6. Audit log entry recorded.

Postcondition: BIO110 no longer counts toward Alice's load. GPA / progression recalculate accordingly.

Alternate flow — after deadline: Backend returns 400 "Drop deadline passed (2026-06-21)". UI surfaces the error without removing the row.

Scenario 4: Submitting an assignment

Actor: Alice Smith **Precondition:** Instructor John Carter has published assignment "Lab 3 — Recursion" for CS101 with due-date in 5 days.

Steps: 1. Alice goes to `/student/assignments` → sees Lab 3 in **Open** tab. 2. Clicks the row → assignment detail panel opens. 3. Pastes her submission text (or uploads a file) and clicks **Submit**. 4. Frontend posts to `POST /api/student/assignment-submissions { assignment_id, content, file_url }`. 5. Backend stores submission with `is_published = false` (instructor must grade first). 6. UI moves Lab 3 to **Submitted** tab.

Postcondition: Instructor sees the submission in their roster. When the instructor grades and publishes, Alice's grade view updates.

Scenario 5: Sending a direct message to an instructor

Actor: Alice Smith **Precondition:** Alice has a question about a lab.

Steps: 1. Alice clicks **Inbox** → **New Message**. 2. Compose modal opens; the **To** dropdown lists only instructors and academic staff (students excluded; finance and admin excluded — per backend `messages.py:96-102`). 3. Alice selects "Dr. John Carter", types subject and body, clicks **Send**. 4. Frontend posts `POST /messages { recipient_id, subject, body, broadcast: false }`. 5. Backend writes a row with `recipient_id = John's user_id`. 6. Confirmation: "Message sent to Dr. John Carter."

Postcondition: Message appears in Alice's **Sent** tab and in Dr. Carter's inbox (with unread badge).

Scenario 6: Viewing risk warnings

Actor: Alice Smith **Precondition:** Alice has 30%+ absence in one offering and one published grade below 60.

Steps: 1. Alice opens `/student/risk`. 2. System composes the page client-side from `progressionApi.me + gradesApi.my + attendance + available-subjects`. 3. Risk page renders two sections: - **Attendance risk**: courses with absence ≥ 25% (exam-blocking threshold) or already exam-blocked. - **Grades at risk**: published grades below the pass mark. 4. Each card links to the relevant course detail page.

Postcondition: Alice is informed of remediation paths (retake, attendance recovery).

Scenario 7: Joining a campus club

Actor: Alice Smith **Precondition:** Robotics Club is open for membership.

Steps: 1. Alice goes to `/student/clubs` and clicks **Join** on Robotics Club. 2. Frontend posts `POST /clubs/{id}/join`. 3. Backend creates a `club_membership` row with `status = 'requested'`. 4. Communications staff is notified and approves/rejects from `/staff/communications`. 5. On approval, Alice's club card shows **Member**.

Postcondition: Alice is a member; she can register for club events.

Student — Use Cases

Student — Use Cases

Use Case Diagram

[Mermaid diagram — see PNG in diagrams/ folder]

Use Case Descriptions

UC1 — Sign in

Actor: Student **Goal:** Obtain an authenticated session. **Preconditions:** Account exists and is active. **Main flow:** Enter email + password → backend validates → JWT issued. **Exception:** Invalid credentials → 401. After 5 failures within 15 min → temporary lockout (NFR-003).

UC4 — Browse course catalog

Actor: Student **Goal:** See offerings available for the active semester. **Main flow:** Open `/student/available-subjects` → backend returns offerings filtered by student's faculty/program/year and semester window.

UC5 — Submit course selection

Actor: Student **Goal:** Request enrollment in an offering. **Preconditions:** Active semester registration window open. No finance hold. Prerequisite satisfied (for required courses). **Main flow:** Click **Add** on offering → `POST /api/student/course-selections` → row inserted with `status = 'requested'`. **Postcondition:** Academic staff sees a pending request.

UC6 — Drop course registration

Actor: Student **Preconditions:** Registration exists, current date ≤ `semester.drop_deadline`. **Main flow:** Click **Drop** → confirm → `POST /registrations/drop`. **Exception:** Past deadline → 400, registration unchanged.

UC7 — View timetable

Actor: Student **Goal:** See weekly schedule. **Main flow:** Open `/student/timetable` → `GET /api/student/timetable` returns deduplicated class_sessions for current week.

UC8 — View course materials

Actor: Student **Main flow:** Open `/student/courses/:id` → week tab → `GET /api/student/courses/{id}/weeks/{w}/materials` returns topic + materials + weekly tasks.

UC9 — Submit assignment

Actor: Student **Preconditions:** Assignment is open (between `start_at` and `end_at`). **Main flow:** Open assignment → enter content / attach file → `POST /api/student/assignment-submissions`.

UC10 — View own grades

Actor: Student **Main flow:** `GET /grades/me` returns only `is_published = true` grades.

UC11 — View attendance summary

Actor: Student **Main flow:** `/student/attendance` → summary cards + per-course color table; computed from `attendance_records`.

UC12 — View risk warnings

Actor: Student **Main flow:** `/student/risk` → composes attendance + grades client-side; flags absences ≥ 25% and grades below pass mark.

UC13 — Read inbox

Actor: Student **Main flow:** `GET /messages/inbox` (filtered to `recipient_id = me OR broadcast`) + `GET /notifications`.

UC14 — Send direct message

Actor: Student **Preconditions:** Recipient is instructor or academic staff (backend rejects others). **Main flow:** Compose → `POST /messages { recipient_id, subject, body, broadcast: false }`. **Exception:** Recipient is student/finance/admin → 403.

UC17 — Join club

Actor: Student **Main flow:** `POST /clubs/{id}/join` → membership row `status = 'requested'`; communications staff approves/rejects.

UC19 — Ask AI chatbot

Actor: Student **Main flow:** Open `/student/chatbot`; query is enriched with student context (registered offerings, grades summary); sent to local Ollama if available, else deterministic fallback.

UC20 — View invoices & balance

Actor: Student **Main flow:** Finance summary on dashboard → `GET /finance/me` returns invoices, payments, balance, holds.

Student — Activity Diagrams

Student — Activity Diagrams

All diagrams are rendered with Mermaid and display natively on GitHub.

A1 — Login + First-time password change

[Mermaid diagram — see PNG in diagrams/ folder]

A2 — Course registration (selection → approval)

[Mermaid diagram — see PNG in diagrams/ folder]

A3 — Drop course

[Mermaid diagram — see PNG in diagrams/ folder]

A4 — Submit assignment

[Mermaid diagram — see PNG in diagrams/ folder]

A5 — Send direct message

[Mermaid diagram — see PNG in diagrams/ folder]

A6 — View risk warnings

[Mermaid diagram — see PNG in diagrams/ folder]

Student — Sequence Diagrams

Student — Sequence Diagrams

System-interaction views of the same flows. Useful when implementing or debugging API contracts.

S1 — Login + JWT issuance

[Mermaid diagram — see PNG in diagrams/ folder]

S2 — Submit course selection

[Mermaid diagram — see PNG in diagrams/ folder]

S3 — Submit assignment + receive grade

[Mermaid diagram — see PNG in diagrams/ folder]

S4 — Send direct message to instructor

[Mermaid diagram — see PNG in diagrams/ folder]

S5 — Drop course (with deadline check)

[Mermaid diagram — see PNG in diagrams/ folder]

Instructor

Instructor Role

The Instructor (also called Teacher in code) delivers course content, runs class sessions, evaluates students, and is the primary interface between the academic structure and the student body.

Responsibilities

- View assigned course offerings and class rosters
- Set the weekly topic per course (drives student materials view)
- Upload weekly materials (files, links, text)
- Create assignments tied to a specific class session and week
- Grade assignment submissions and provide feedback
- Mark attendance per class session (with date-lock enforcement)
- Configure grade components (midterm / final / project / etc.) and weights
- Record component scores and publish final grades
- Read inbox + reply to student / staff messages
- View own profile and change password

Screens (Frontend Routes)

Route	Page
`/instructor`	Dashboard (assigned courses summary)
`/instructor/profile`	Profile
`/instructor/courses`	My Courses
`/instructor/courses/:offeringId`	Course Detail (materials/assignments/attendance/grades)
`/instructor/materials`	Weekly Materials manager
`/instructor/assignments`	Assignments manager
`/instructor/attendance`	Attendance marking
`/instructor/grades`	Grades management
`/instructor/student`	Students roster
`/instructor/inbox`	Inbox
`/instructor/change-password`	Change Password

Key API Endpoints

Endpoint	Purpose
`GET /api/teacher/courses`	Offerings assigned to me
`GET /api/teacher/sessions?courseId=`	Class sessions per offering (week + date)
`POST /api/teacher/course-offerings/{id}/weeks/{w}/topic`	Set weekly topic
`POST /api/teacher/course-offerings/{id}/materials`	Upload material
`POST /api/teacher/course-offerings/{id}/assignments`	Create assignment (requires `class_session_id`)
`PUT /api/teacher/assignment-submissions/{id}`	Score + feedback
`POST /attendance/offering/{id}/sessions`	Create attendance session
`POST /attendance/sessions/{sid}/records`	Per-student attendance records
`PUT /grades/offering/{o}/registration/{r}`	Upsert grade
`POST /grades/publish`	Publish grades (`{registration_ids}`)
`GET /messages/inbox`	Inbox
`POST /messages`	Send message

Permissions Matrix

Action	Allowed?
View own profile / change password	✅
View own courses' rosters	✅
View other instructors' courses	❌
Mark attendance for assigned offerings	✅ (current-date sessions)
Mark attendance for past-dated sessions	❌ (date-locked)
Create assignment for assigned offerings	✅ (requires real `class_session_id`)
Grade submissions	✅
Publish grades	✅
Approve registrations	❌
Create announcements / events	❌

Action	Allowed?
Issue invoices	❌
Send direct message	✅ (to anyone except finance/admin via UI restriction)

Business Rules

- **Date-locked attendance**: marking attendance for a past date returns 400 "Attendance for previous dates is locked". Workaround: create a fresh session via `POST /attendance/offering/{id}/sessions` with today's date.
- **Assignment-session link**: `POST .../assignments` requires `class_session_id` whose week matches the form's `week_number`. Get valid sessions+weeks from `GET /api/teacher/sessions?courseId=`.
- **Grade publish**: students only see grades where `is_published = true`.
- **Roster scope**: an instructor only sees students registered in offerings explicitly assigned to them.

Related Diagrams

- user-scenarios.md
- use-cases.md
- activity-diagrams.md
- sequence-diagrams.md
- diagrams/ — rendered PNGs of every Mermaid diagram (auto-generated)

Instructor — User Scenarios

Instructor — User Scenarios

Scenario 1: Setting up Week 12 of CS101

Actor: Dr. John Carter (instructor of CS101 — Intro to Programming) **Precondition:** Spring 2026 semester active. CS101 is offering ID 1 assigned to John. Week 12 of the semester is the current week.

Steps: 1. John logs in; dashboard shows CS101. 2. Opens `/instructor/courses/1` (Course Detail). 3. Switches to **Week 12** tab. 4. Clicks **Edit Topic**, enters "Recursion and Tail Calls", saves. - `POST /api/teacher/course-offerings/1/weeks/12/topic { title, description }` 5. Uploads two materials: lecture slides PDF and example code link. - `POST /api/teacher/course-offerings/1/materials { week_number: 12, file_url, link_url, status: published }` 6. Creates assignment "Lab 3 — Recursion": - Selects class_session 22 (week 12) from the picker (loaded via `GET /api/teacher/sessions?courseId=1`). - Sets `start_at = now`, `end_at = now + 5 days`. - Submits → `POST /api/teacher/course-offerings/1/assignments { class_session_id: 22, week_number: 12, ... }`.

Postcondition: Week 12 topic, materials, and Lab 3 are visible to enrolled students at `/student/courses/1/weeks/12`.

Scenario 2: Marking attendance for today's class

Actor: Dr. John Carter **Precondition:** CS101 has a class session scheduled today.

Steps: 1. John opens `/instructor/attendance`. 2. Picks CS101 → today's session is listed. 3. Clicks **Mark Attendance** → roster appears with toggle (Present/Absent/Late) per student. 4. Marks Alice = Present, Brian = Absent, Daniel = Late. 5. Clicks **Save**. - `POST /attendance/sessions/{sid}/records [{student_id, status}, ...]` 6. Backend writes attendance_records; each student's absence-percentage is recomputed.

Postcondition: Students see their attendance status on `/student/attendance`. Brian's absence count increases — if ≥ 25% he sees a risk warning.

Alternate flow — past date: If John tries to mark a session from yesterday or earlier (e.g. someone forgot), bulk-save returns 400 "Attendance for previous dates is locked". He must create a fresh session via `POST /attendance/offering/1/sessions` with today's date.

Scenario 3: Grading submitted assignments

Actor: Dr. John Carter **Precondition:** 18 students submitted Lab 3.

Steps: 1. John opens `/instructor/courses/1` → **Assignments** tab → **Lab 3** → **Submissions**. 2. Sees list of 18 submissions sorted by submitted_at. 3. Clicks Alice's submission → reads content, enters score = 95, feedback = "Excellent recursion clarity". 4. Clicks **Save**. - `PUT /api/teacher/assignment-submissions/{id} { score: 95, feedback: "..." }` 5. Repeats for all 18 submissions. 6. When ready to publish: opens **Grades** tab, picks Lab 3 grade column, clicks **Publish**. - `POST /grades/publish { registration_ids: [...] }`

Postcondition: Alice sees her Lab 3 score on `/student/grades`. Component score contributes to her cumulative grade per the configured weights.

Scenario 4: Configuring grade components for CS101

Actor: Dr. John Carter **Precondition:** No grade configuration exists yet for CS101's Spring 2026 offering.

Steps: 1. John opens `/instructor/grades` → picks CS101. 2. Default configuration row exists (auto-created on first visit). John adjusts: - midterm = 25, final_exam = 35, project = 20, assignment = 10, quiz = 5, attendance = 5, participation = 0, lab = 0. 3. Total = 100 — system validates. 4. Saves. - `PUT /grades/config/offering/1 { ... }`

Postcondition: All grade entries use these weights. Component score validation rejects entries that exceed configured maxima.

Scenario 5: Replying to a student message

Actor: Dr. John Carter **Precondition:** Alice sent John a message "Question about Lab 3 recursion limit".

Steps: 1. John opens `/instructor/inbox` — sees Alice's message with **Unread** badge. 2. Clicks the message; content opens. 3. Clicks **Reply** → compose modal pre-fills Alice's name in **To** and `Re: Question about Lab 3 recursion limit` in subject. 4. Types his answer, clicks **Send**. - `POST /messages { recipient_id: 10, subject, body, broadcast: false }` 5. Marks Alice's original message as read. - `PUT /messages/{id}/read`

Postcondition: Alice sees John's reply in her inbox.

Scenario 6: Locked attendance and recovery

Actor: Dr. John Carter **Precondition:** John forgot to mark attendance for last Tuesday's class.

Steps: 1. John opens `/instructor/attendance` and selects last Tuesday's session. 2. He attempts to mark students — bulk-save returns "Attendance for previous dates is locked". 3. John reads the inline

help text explaining the date lock. 4. He creates a fresh ad-hoc session for today: - `POST /attendance/offering/1/sessions { session_date: today, ... }` 5. Marks the (still missing) absences in this new session.

Postcondition: Absence record is corrected through a new audit-trailed session, preserving the historical record of the original session.

Instructor — Use Cases

Instructor — Use Cases

Use Case Diagram

[Mermaid diagram — see PNG in diagrams/ folder]

Use Case Descriptions

UC1 — View assigned courses

Actor: Instructor **Main flow:** `GET /api/teacher/courses` returns offerings where `instructor_id = me`.

UC2 — View course roster

Actor: Instructor **Main flow:** Open course detail → `GET /api/teacher/course-offerings/{id}/registrations` returns active enrollments.

UC3 — Set weekly topic

Actor: Instructor **Main flow:** `POST /api/teacher/course-offerings/{id}/weeks/{w}/topic { title, description }`. **Postcondition:** Topic drives the student week view.

UC4 — Upload material

Actor: Instructor **Main flow:** `POST /api/teacher/course-offerings/{id}/materials { week_number, file_url|link_url|text_content, status }`. **Variant:** `status = 'scheduled'` with `publish_at` defers visibility until time arrives.

UC5 — Create assignment

Actor: Instructor **Preconditions:** A real `class_session_id` exists whose `week_id == form.week_number`. **Main flow:** `POST /api/teacher/course-offerings/{id}/assignments { class_session_id, week_number, title, description, max_score, start_at, end_at }`. **Exception:** If `class_session.week_id != week_number` → 400.

UC6 — View submissions

Actor: Instructor **Main flow:** `GET /api/teacher/assignments/{id}/submissions` returns list of student submissions.

UC7 — Grade submission

Actor: Instructor **Main flow:** `PUT /api/teacher/assignment-submissions/{id} { score, feedback }`. **Postcondition:** Submission contributes to the assignment component score for the registration.

UC8 — Create attendance session

Actor: Instructor **Main flow:** `POST /attendance/offering/{id}/sessions { session_date, week_number, start_time, end_time }`.

UC9 — Mark attendance

Actor: Instructor **Preconditions:** Session date is today (date-lock). **Main flow:** `POST /attendance/sessions/{sid}/records [{student_id, status, late_minutes}, ...]`. **Exception:** Past-date bulk save → 400 "Attendance for previous dates is locked".

UC10 — View attendance report

Actor: Instructor **Main flow:** `GET /api/teacher/course-offerings/{id}/attendance-report` returns per-student absence percentage.

UC11 — Configure grade components

Actor: Instructor **Main flow:** `PUT /grades/config/offering/{id} { midterm_points, final_exam_points, ..., attendance_points }`. **Validation:** Sum must total a configured cap (typically 100).

UC12 — Record component scores

Actor: Instructor **Main flow:** `PUT /grades/offering/{o}/registration/{r} { midterm_score, ... }`.

UC13 — Publish grades

Actor: Instructor **Main flow:** `POST /grades/publish { registration_ids }`. **Postcondition:** `is_published` flips to true; students see scores via `GET /grades/me`.

UC14 — Read inbox

Actor: Instructor **Main flow:** `GET /messages/inbox` filters to `recipient_id = me OR broadcast`.

UC15 — Reply to message

Actor: Instructor **Main flow:** Click Reply → `POST /messages { recipient_id, subject: "Re: ...", body, parent_id }`.

UC16 — Send direct message

Actor: Instructor **Main flow:** Same as Reply but with a fresh subject and recipient picked from contacts.

UC17 — Change password

Actor: Instructor **Main flow:** `POST /auth/change-password { current_password, new_password }`.

UC18 — Update profile

Actor: Instructor **Main flow:** `PUT /users/me { phone, bio, office_hours, ... }`.

Instructor — Activity Diagrams

Instructor — Activity Diagrams

A1 — Weekly course setup (topic + material + assignment)

[Mermaid diagram — see PNG in diagrams/ folder]

A2 — Attendance marking (with date lock)

[Mermaid diagram — see PNG in diagrams/ folder]

A3 — Grade workflow (config → entry → publish)

[Mermaid diagram — see PNG in diagrams/ folder]

A4 — Reply to message

[Mermaid diagram — see PNG in diagrams/ folder]

Instructor — Sequence Diagrams

Instructor — Sequence Diagrams

S1 — Create assignment (with class_session linkage)

[Mermaid diagram — see PNG in diagrams/ folder]

S2 — Attendance bulk save with date-lock

[Mermaid diagram — see PNG in diagrams/ folder]

S3 — Grade configuration + publish

[Mermaid diagram — see PNG in diagrams/ folder]

S4 — Grade assignment submission

[Mermaid diagram — see PNG in diagrams/ folder]

Academic Staff

Academic Staff Role

Academic Staff owns the academic structure: catalog, offerings, timetable, rooms, registration approvals, and semester windows. They are the operational backbone of every academic term.

> The backend role identifier is `academic_staff`. The Communications Staff (see ../communications-staff/) shares the same backend role but is documented separately for governance and acceptance review.

Responsibilities

- Maintain the course catalog (create / edit / archive courses, prerequisites)
- Create and edit course offerings (course → program → semester → instructor)
- Manage the timetable (weekly entries, classroom assignment, conflict avoidance)
- Maintain buildings and rooms (classrooms, labs)
- Approve or reject student course selection requests
- Set / adjust semester windows (registration_open_at, drop_deadline, etc.)
- View all student grades (read-only across the faculty)
- View student records within scope (faculty-scoped via `StaffFacultyScope`)
- Receive and reply to messages from students and instructors

Screens (Frontend Routes — both `/academic-staff` and `/staff` variants)

Route	Page
`/academic-staff`	Dashboard
`/academic-staff/profile`	Profile
`/academic-staff/courses`	Course Catalog
`/academic-staff/offerings`	Manage Offerings (course → program → instructor)
`/academic-staff/staff-course-offerings`	Course Offerings listing
`/academic-staff/staff-timetable`	Timetable manager
`/academic-staff/staff-buildings-rooms`	Buildings & Rooms
`/academic-staff/registrations`	Registration approvals
`/academic-staff/grades`	Grades view (all faculty grades)

Route	Page
`/academic-staff/students`	Student records
`/academic-staff/inbox`	Inbox
`/academic-staff/change-password`	Change Password

Key API Endpoints

Endpoint	Purpose
`GET/POST/PATCH /courses` + `/courses/catalog`	Course CRUD
`GET/POST/PATCH /offerings`	Offering CRUD
`GET /api/staff/offerings`	Faculty-scoped offerings
`GET /api/staff/instructors`	Pickers for assignment
`POST /api/staff/timetable-entries`	Schedule a class
`PATCH /api/staff/timetable-entries/{id}`	Reassign room / time
`POST /campus-resources/buildings` + `/classrooms`	Room inventory
`GET /api/staff/course-selections?status=requested`	Pending selections
`PATCH /api/staff/course-selections/{id}`	Approve / reject
`PATCH /semesters/{id}`	Adjust dates
`GET /grades/all` (faculty-scoped)	All grades within faculty

Permissions Matrix

Action	Allowed?
Create / edit course	✅
Create / edit offering	✅
Assign instructor to offering	✅
Edit timetable	✅ (no double-book of room/group)
Approve / reject registration requests	✅
Edit / publish grades	✅ (corrective)
Place finance hold	❌ (Finance Staff)
Issue invoice	❌

Action	Allowed?
Send direct message	✅
Create announcement / event	✅ (separated to Communications Staff docs)

Business Rules

- **Faculty scope.** Staff with `StaffFacultyScope` only see students, offerings, and selections within their assigned faculty.
- **Timetable integrity.** A `(classroom_id, weekday, start_time-end_time)` tuple is unique — no two offerings share a room/time. The UI surfaces conflicts before save.
- **Required vs elective selection.** Required-subject selection enforces strict program/year/prerequisite gating. Elective offerings bypass program/year (same-faculty only).
- **Drop deadline.** Set per semester via `PATCH /semesters/{id} { drop_deadline }`. Backend enforces against student drop attempts.
- **Approval gate.** Student selections enter as `requested`. Until staff updates to `approved`, no `registration` row exists.

Related Diagrams

- user-scenarios.md
- use-cases.md
- activity-diagrams.md
- sequence-diagrams.md
- diagrams/ — rendered PNGs of every Mermaid diagram (auto-generated)

Academic Staff — User Scenarios

Academic Staff — User Scenarios

Scenario 1: Approving a student's course selection

Actor: Rebecca Morgan (academic staff, FCSIT) **Precondition:** Alice Smith submitted a selection request for CS220 (elective, same faculty).

Steps: 1. Rebecca opens `/academic-staff/registrations`. 2. Filters to **Pending** + Spring 2026 → sees Alice's request. 3. Reviews student transcript inline (current GPA, semester load, prereqs). 4. No conflicts found. Clicks **Approve**. - `PATCH /api/staff/course-selections/{id} { status: 'approved' }` 5. Backend updates the selection and creates a `registrations` row with `status = 'active'`. 6. Alice gets a notification.

Postcondition: CS220 appears on Alice's timetable and on her **My Courses** page.

Alternate flow — reject: Rebecca sees Alice already at 30 credits (overload). She clicks **Reject** and enters reason "Exceeds maximum credit load". Selection moves to `status = 'rejected'`.

Scenario 2: Creating a new offering for Spring 2026

Actor: Rebecca Morgan **Precondition:** New course "CS305 — Advanced Algorithms" exists in catalog (created previously). Spring 2026 semester exists. Dr. John Carter teaches algorithms.

Steps: 1. Rebecca opens `/academic-staff/offerings`. 2. Clicks **New Offering**. 3. Form: Course = CS305, Semester = Spring 2026, Program = BSE (Software Engineering), Instructor = John Carter, Group = "Group A", Capacity = 30, Schedule = Tuesday 09:00–11:50 in Room A2. 4. Submits → `POST /offerings`. 5. Backend validates: course exists, semester active, instructor role = teacher, no timetable conflict for (room A2, Tue 09:00). 6. Offering created. Timetable entries auto-generated for each week of Spring 2026.

Postcondition: CS305 appears in the catalog browse for eligible students. Instructor sees it on `/instructor/courses`.

Scenario 3: Adjusting the drop deadline

Actor: Rebecca Morgan **Precondition:** Many students requested an extension; original `drop_deadline` was 2026-05-30.

Steps: 1. Rebecca opens `/staff/semester-windows` (or the equivalent `/academic-staff/semester-windows`). 2. Selects Spring 2026 → sees current windows. 3. Edits **Drop deadline** from 2026-05-30 → 2026-06-21. 4. Submits → `PATCH /semesters/2 { drop_deadline: '2026-06-21' }`. 5. Backend updates the row.

Postcondition: Until 2026-06-21, the backend will accept `POST /registrations/drop` requests. Past that date, drops return 400.

Scenario 4: Resolving a timetable conflict

Actor: Rebecca Morgan **Precondition:** New offering for IT201 was scheduled Tuesday 10:00 in Room B3, conflicting with existing CS101 in the same room/time.

Steps: 1. Rebecca opens `/academic-staff/staff-timetable`. 2. UI shows IT201 entry with a red conflict banner. 3. Rebecca clicks the entry → edit dialog. 4. Picks a different room (Room B4) → submits. - `PATCH /api/staff/timetable-entries/{id} { classroom_id: ... }` 5. Backend re-validates uniqueness, accepts.

Postcondition: No room/time double-booking. Both offerings remain scheduled. Affected students see the updated room on their timetable.

Scenario 5: Creating a new course in the catalog

Actor: Rebecca Morgan **Precondition:** Faculty board approved adding "DS210 — Statistics for Data Science" to the BDS program.

Steps: 1. Rebecca opens `/academic-staff/courses`. 2. Clicks **New Course**. 3. Enters: code = DS210, title = Statistics for Data Science, credits = 6, department = Data Science, prereqs = [DS110]. 4. Submits → `POST /courses { ... }`. 5. Backend stores course. UI lists it under the Data Science department. 6. Course is now available for offering creation.

Postcondition: DS210 exists in catalog. No offerings yet — those are created separately for each term.

Scenario 6: Adding a new classroom

Actor: Rebecca Morgan **Precondition:** Facilities team added a new lab room.

Steps: 1. Rebecca opens `/staff/buildings-rooms`. 2. Picks Building 3 → clicks **New Room**. 3. Enters: code = "Lab 3-205", type = "lab", capacity = 24. 4. Submits → `POST /campus-resources/classrooms { ... }`.

Postcondition: Room 3-205 appears in pickers when creating new timetable entries / offerings.

Academic Staff — Use Cases

Academic Staff — Use Cases

Use Case Diagram

[Mermaid diagram — see PNG in diagrams/ folder]

Use Case Descriptions

UC1 — Create course

Main flow: `POST /courses { code, title, credits, department_id, prerequisites }`.

UC4 — Create offering

Preconditions: Course exists, semester exists, instructor has teacher role. **Main flow:** `POST /offerings { course_id, semester_id, program_id, instructor_id, group_name, capacity, schedule, room }`. **Side effect:** Per-week timetable entries are generated.

UC7 — Add timetable entry

Preconditions: No `(classroom_id, weekday, start_time-end_time)` conflict. **Main flow:** `POST /api/staff/timetable-entries { offering_id, weekday, start_time, end_time, classroom_id }`.

UC11 — Review pending selection

Main flow: `GET /api/staff/course-selections?status=requested` returns the queue.

UC12 — Approve selection

Main flow: `PATCH /api/staff/course-selections/{id} { status: 'approved' }`. **Side effect:** Backend creates a `registrations` row.

UC13 — Reject selection

Main flow: `PATCH .../{id} { status: 'rejected', reason: '...' }`.

UC14 / UC15 / UC16 — Adjust semester windows

Main flow: `PATCH /semesters/{id} { registration_open_at, drop_deadline, total_weeks }`.

UC18 — View all grades (faculty-scoped)

Main flow: `GET /grades/all` filters by staff's faculty scope.

UC19 — Read inbox

Main flow: Same as Student / Instructor — `GET /messages/inbox`.

UC20 — Send / reply

Main flow: `POST /messages` (any recipient role; contacts dropdown includes everyone except self).

Academic Staff — Activity Diagrams

Academic Staff — Activity Diagrams

A1 — Course selection approval

[Mermaid diagram — see PNG in diagrams/ folder]

A2 — Create offering with timetable

[Mermaid diagram — see PNG in diagrams/ folder]

A3 — Adjust semester windows

[Mermaid diagram — see PNG in diagrams/ folder]

A4 — Resolve timetable conflict

[Mermaid diagram — see PNG in diagrams/ folder]

Academic Staff — Sequence Diagrams

Academic Staff — Sequence Diagrams

S1 — Approve student course selection

[Mermaid diagram — see PNG in diagrams/ folder]

S2 — Create offering

[Mermaid diagram — see PNG in diagrams/ folder]

S3 — Adjust drop deadline

[Mermaid diagram — see PNG in diagrams/ folder]

S4 — Timetable edit with conflict check

[Mermaid diagram — see PNG in diagrams/ folder]

Communications Staff

Communications Staff Role

Communications Staff handles institutional broadcast surfaces — announcements, events, clubs, and broadcast messaging. While the backend role identifier is `academic_staff` (the same as Academic Staff), the responsibilities are split here to reflect organisational governance: in larger institutions these are owned by a dedicated communications team.

Responsibilities

- Publish announcements (general or scoped to a faculty / program / cohort)
- Create and manage campus events (lectures, workshops, fairs, sports days)
- Approve / reject student club join requests
- Create club records, approve club proposals
- Manage event registration windows and capacity
- Broadcast institutional messages (system-wide messaging)
- Maintain news feed visible at `/student/news`

Screens (Frontend Routes)

Route	Page
`/academic-staff/communications`	Communications Hub (announcements / events / clubs tabs)
`/staff/communications`	Same hub (legacy `/staff` URL variant)

The hub aggregates:

- **Announcements** — create, edit, schedule, publish, withdraw
- **Events** — create, edit, view registrations
- **Clubs** — review join requests, approve / reject

Key API Endpoints

Endpoint	Purpose
`GET/POST/PATCH /communications/announcements`	Announcement CRUD
`POST /communications/announcements/{id}/publish`	Flip `is_published`
`GET/POST/PATCH /communications/events`	Event CRUD

Endpoint	Purpose
`GET /communications/events/{id}/registrations`	List registrants
`GET/POST /communications/clubs`	Club records
`PATCH /communications/club-memberships/{id}`	Approve / reject join request
`POST /messages` (with `broadcast: true`)	Broadcast message (allowed for `academic_staff`, `system_admin`)

Permissions Matrix

Action	Allowed?
Publish announcement	✅
Withdraw announcement	✅
Create event	✅
View event registration list	✅
Approve / reject club join	✅
Create club	✅
Broadcast message	✅ (`broadcast: true` requires `academic_staff` or `system_admin` role)
Send direct messages	✅
Issue invoice	❌
Edit grades	❌

Business Rules

- **Publication metadata.** Every announcement / event carries `created_by_user_id`, `created_at`, and (on publish) `published_at`.
- **Scope.** Announcements can target all students, a specific faculty, a specific program, or a cohort. Scope is enforced server-side at fetch.
- **Broadcast restriction.** Broadcast messages (`is_broadcast = true`) only allowed for `academic_staff` and `system_admin`. Other roles get 403.
- **Club approval.** Until communications staff approves, `club_membership.status` stays `requested`. Approval flips to `active`. Rejection flips to `rejected` with optional reason.
- **Event capacity.** When `capacity` is set, the system rejects registrations beyond it.

Related Diagrams

- user-scenarios.md
- use-cases.md
- activity-diagrams.md
- sequence-diagrams.md
- diagrams/ — rendered PNGs of every Mermaid diagram (auto-generated)

Communications Staff — User Scenarios

Communications Staff — User Scenarios

Scenario 1: Publishing a semester-start announcement

Actor: Rebecca Morgan (academic_staff role, communications duties) **Precondition:** Spring 2026 semester starts in 5 days. Faculty deans approved the announcement text.

Steps: 1. Rebecca opens `/academic-staff/communications` → **Announcements** tab. 2. Clicks **New Announcement**. 3. Fills: - Title = "Spring 2026 starts Monday — schedule live now" - Body = "..." - Scope = "All students" - Publish at = now (or schedule for tomorrow 08:00) 4. Saves draft → `POST /communications/announcements { title, body, scope, publish_at }`. 5. Clicks **Publish** → `POST /communications/announcements/{id}/publish`. 6. `is_published` flips, `published_at = now()`.

Postcondition: Every student sees the announcement at `/student/news`.

Scenario 2: Creating a campus event

Actor: Rebecca Morgan **Precondition:** Career Fair scheduled for 2026-06-15.

Steps: 1. Opens `/academic-staff/communications` → **Events** tab → **New Event**. 2. Fills: Title = "2026 Career Fair", Date = 2026-06-15, Time = 10:00–16:00, Location = "Hall A", Capacity = 300, Registration window = open until 2026-06-13. 3. Submits → `POST /communications/events { ... }`. 4. Event appears in the **Events** list and on student `/student/news`.

Postcondition: Students can register. Backend rejects new registrations once `count >= capacity` or after `registration_close_at`.

Scenario 3: Approving a club join request

Actor: Rebecca Morgan **Precondition:** Alice submitted a join request for Robotics Club.

Steps: 1. Opens `/academic-staff/communications` → **Clubs** tab → **Membership requests**. 2. Sees Alice's request with **Pending** badge. 3. Reviews her profile (program, year). Clicks **Approve**. - `PATCH /communications/club-memberships/{id} { status: 'active' }`. 4. Alice gets a notification.

Postcondition: Alice's club card on `/student/clubs` switches from **Pending** to **Member**.

Alternate flow — reject: Rebecca picks **Reject** and enters reason "Club at capacity". Membership flips to `rejected`.

Scenario 4: Broadcasting a system-wide message

Actor: Rebecca Morgan **Precondition:** Library hours are changing temporarily; needs urgent broadcast.

Steps: 1. Opens Inbox → **New Message** → toggles **Broadcast** option. 2. Subject = "Library hours change — read by Friday". 3. Body = "..." 4. Sends → `POST /messages { broadcast: true, subject, body }`. 5. Backend permits because Rebecca's role ∈ `{academic_staff, system_admin}`. 6. Message row stored with `is_broadcast = true`, no recipient_id.

Postcondition: Every active user sees the broadcast in their inbox with a **Broadcast** badge.

Scenario 5: Withdrawing a published announcement

Actor: Rebecca Morgan **Precondition:** Announcement contained incorrect date.

Steps: 1. Opens **Announcements** tab → finds the published row. 2. Clicks **Withdraw** → confirmation. 3. `PATCH /communications/announcements/{id} { is_published: false, withdrawn_at: now }`.

Postcondition: Announcement disappears from `/student/news`. Audit log entry records who withdrew it and when. A revised version can be created and published separately.

Communications Staff — Use Cases

Communications Staff — Use Cases

Use Case Diagram

[Mermaid diagram — see PNG in diagrams/ folder]

Use Case Descriptions

UC1 — Draft announcement

Main flow: `POST /communications/announcements { title, body, scope, publish_at }`; `is_published = false`.

UC2 — Publish announcement

Main flow: `POST /communications/announcements/{id}/publish` → flips `is_published`, sets `published_at`.

UC3 — Schedule announcement

Main flow: Same as UC1 with future `publish_at`. A background job (or on-demand fetch logic) treats it as visible only when `publish_at <= now`.

UC4 — Withdraw announcement

Main flow: `PATCH /communications/announcements/{id} { is_published: false }`. Audit log entry recorded.

UC5 — Scope to faculty / program

Main flow: Announcement carries `scope` field; backend fetch filters by viewer.

UC6 — Create event

Main flow: `POST /communications/events { title, starts_at, ends_at, location, capacity, registration_close_at }`.

UC9 — View event registrations

Main flow: `GET /communications/events/{id}/registrations`.

UC11 — Create club

Main flow: `POST /communications/clubs { name, description, category_id }`.

UC13 — Review join requests

Main flow: `GET /communications/club-memberships?status=requested`.

UC14 / UC15 — Approve / reject

Main flow: `PATCH /communications/club-memberships/{id} { status: 'active'|'rejected', reason? }`.

UC16 — Broadcast message

Preconditions: Sender role ∈ {academic_staff, system_admin}. **Main flow:** `POST /messages { broadcast: true, subject, body }`. Server enforces role.

UC17 — Reply to user message

Main flow: Same as inbox reply (UC15 in Instructor docs).

Communications Staff — Activity Diagrams

Communications Staff — Activity Diagrams

A1 — Announcement lifecycle (draft → publish → withdraw)

[Mermaid diagram — see PNG in diagrams/ folder]

A2 — Event creation and registration management

[Mermaid diagram — see PNG in diagrams/ folder]

A3 — Club join approval

[Mermaid diagram — see PNG in diagrams/ folder]

A4 — Broadcast message

[Mermaid diagram — see PNG in diagrams/ folder]

Communications Staff — Sequence Diagrams

Communications Staff — Sequence Diagrams

S1 — Publish announcement

[Mermaid diagram — see PNG in diagrams/ folder]

S2 — Create + register event

[Mermaid diagram — see PNG in diagrams/ folder]

S3 — Approve club membership

[Mermaid diagram — see PNG in diagrams/ folder]

S4 — Broadcast message

[Mermaid diagram — see PNG in diagrams/ folder]

Finance Staff

Finance Staff Role

Finance Staff manages the student billing and collection workflow: issues invoices, records payments, places and releases holds, and reviews per-student balances. Finance scope is restricted to the staff member's assigned faculty via `FinanceFacultyScope`.

Responsibilities

- Issue student invoices (tuition, lab fees, late fees, custom charges)
- Record payments against invoices
- Place finance holds on accounts (preventing further registration, transcripts, etc.)
- Release holds once obligations are met
- Review per-student finance accounts (balance, history)
- View finance dashboard (faculty totals, outstanding amounts)
- Receive and reply to messages

Screens (Frontend Routes)

Route	Page
`/finance-staff`	Dashboard (totals: billed / collected / outstanding)
`/finance-staff/profile`	Profile
`/finance-staff/students`	Student Accounts (per-student balance + status)
`/finance-staff/invoices`	Invoice ledger
`/finance-staff/payments`	Payment ledger
`/finance-staff/holds`	Active holds + history
`/finance-staff/change-password`	Change password

> Finance Staff does **not** currently have an Inbox screen — messaging in/out from finance staff is intentionally out of scope per the README role matrix.

Key API Endpoints

Endpoint	Purpose
`GET /finance/dashboard`	Aggregate stats (faculty-scoped)

Endpoint	Purpose
`GET /finance/students`	Student accounts with balance
`POST /finance/invoices`	Issue invoice `{student_id, amount, item, due_date}`
`GET /finance/invoices`	Invoice ledger
`POST /finance/payments`	Record payment `{invoice_id, amount, method, reference}`
`GET /finance/payments`	Payment ledger
`POST /finance/holds`	Place hold `{student_id, reason}`
`PATCH /finance/holds/{id}`	Release hold

Permissions Matrix

Action	Allowed?
Issue invoice	✅
Record payment	✅
Place hold	✅
Release hold	✅
View student finance account	✅ (faculty-scoped)
View student academic grades	❌
Edit grades	❌
Approve registrations	❌
Send / receive direct messages	❌ (out of scope)

Business Rules

- **Faculty scope.** `FinanceFacultyScope` limits visibility — a finance staff member only sees students within their faculty.
- **Currency.** Amounts are stored as DECIMAL; UI displays the configured institutional currency code (currently "ALL").
- **Hold side-effects.** When a hold is active, the registration backend may reject `POST /api/student/course-selections` for that student (configurable policy).
- **Audit.** Every invoice issued, payment recorded, hold placed/released is audit-logged.
- **Balance computation.** `outstanding = sum(invoices.amount) - sum(payments.amount)` for an account, scoped to a semester or all-time.

Related Diagrams

- user-scenarios.md
- use-cases.md
- activity-diagrams.md
- sequence-diagrams.md
- diagrams/ — rendered PNGs of every Mermaid diagram (auto-generated)

Finance Staff — User Scenarios

Finance Staff — User Scenarios

Scenario 1: Issuing a tuition invoice

Actor: FCSIT Finance Officer (`finance.csit@cis.edu`) **Precondition:** Alice Smith is enrolled for Spring 2026. Tuition due is ALL 80,000.

Steps: 1. Officer opens `/finance-staff/invoices` → **New Invoice**. 2. Picks student Alice Smith (search by name or student code). 3. Enters: item = "Spring 2026 Tuition", amount = 80,000.00, due_date = 2026-06-30. 4. Submits → `POST /finance/invoices { student_id, amount, item, due_date }`. 5. Backend creates invoice with `status = 'unpaid'`.

Postcondition: Alice sees the invoice on her student finance summary.

Scenario 2: Recording a payment

Actor: FCSIT Finance Officer **Precondition:** Alice paid ALL 40,000 by bank transfer; reference number provided.

Steps: 1. Officer opens `/finance-staff/payments` → **Record Payment**. 2. Picks Alice's invoice (or by reference). 3. Enters: amount = 40,000, method = "bank_transfer", reference = "TXN-5429". 4. Submits → `POST /finance/payments { invoice_id, amount, method, reference }`. 5. Backend inserts payment; invoice `balance` recomputed.

Postcondition: Invoice shows partially paid; outstanding = 40,000.

Scenario 3: Placing a hold for overdue balance

Actor: FCSIT Finance Officer **Precondition:** Student Brian Taylor has an invoice 30+ days overdue.

Steps: 1. Officer opens `/finance-staff/students` → finds Brian → clicks his row. 2. Account detail shows outstanding ALL 60,000, last payment 45 days ago. 3. Clicks **Place Hold** → enters reason = "Overdue balance (>30 days)". 4. `POST /finance/holds { student_id, reason }`.

Postcondition: Brian sees a "Finance hold active" banner on his dashboard. Backend rejects new course-selection requests from Brian with 400 "Finance hold active".

Scenario 4: Releasing a hold after payment

Actor: FCSIT Finance Officer **Precondition:** Brian paid in full. Hold is still active.

Steps: 1. Officer records the payment (Scenario 2). 2. Opens `/finance-staff/holds` → finds Brian's active hold. 3. Clicks **Release** → confirmation. 4. `PATCH /finance/holds/{id} { released_at: now, released_by_user_id: me }`. 5. Audit log entry recorded.

Postcondition: Brian's hold banner clears. He can register for courses again.

Scenario 5: Reviewing faculty dashboard

Actor: FCSIT Finance Officer (start of month) **Steps:** 1. Officer opens `/finance-staff` dashboard. 2. Tiles show: - Total billed (faculty-scoped) - Total collected - Outstanding (billed − collected) - Number of active holds 3. Officer drills into outstanding to identify top debtors → opens each account.

Postcondition: Operational data informs collection prioritisation.

Finance Staff — Use Cases

Finance Staff — Use Cases

Use Case Diagram

[Mermaid diagram — see PNG in diagrams/ folder]

Use Case Descriptions

UC1 — Issue invoice

Main flow: `POST /finance/invoices { student_id, amount, item, due_date }`. Validation: student exists, amount > 0.

UC5 — Record payment

Main flow: `POST /finance/payments { invoice_id, amount, method, reference }`. Validation: amount ≤ remaining balance.

UC8 — Place hold

Main flow: `POST /finance/holds { student_id, reason }`. Audit log entry.

UC9 — Release hold

Main flow: `PATCH /finance/holds/{id} { released_at, released_by_user_id }`. Audit log entry.

UC11 — View student finance account

Main flow: `GET /finance/students/{id}` → invoices + payments + balance + active holds. Faculty-scoped.

UC12 — Review aggregate dashboard

Main flow: `GET /finance/dashboard` → faculty totals (billed / collected / outstanding / active holds count).

UC13 — Change password

Main flow: `POST /auth/change-password`.

UC14 — Update profile

Main flow: `PUT /users/me { phone, etc. }`.

Finance Staff — Activity Diagrams

Finance Staff — Activity Diagrams

A1 — Invoice → payment → balance update

[Mermaid diagram — see PNG in diagrams/ folder]

A2 — Faculty dashboard review

[Mermaid diagram — see PNG in diagrams/ folder]

A3 — Place / release hold

[Mermaid diagram — see PNG in diagrams/ folder]

A4 — Recording a refund (rare)

[Mermaid diagram — see PNG in diagrams/ folder]

Finance Staff — Sequence Diagrams

Finance Staff — Sequence Diagrams

S1 — Issue invoice + record payment

[Mermaid diagram — see PNG in diagrams/ folder]

S2 — Place hold blocks registration

[Mermaid diagram — see PNG in diagrams/ folder]

S3 — Faculty-scoped dashboard

[Mermaid diagram — see PNG in diagrams/ folder]

System Admin

System Admin Role

System Admin has full operational control of the platform: provisions users, configures semesters, manages global settings, monitors analytics, and acts as break-glass authority for any role-restricted operation.

Responsibilities

- Provision and lifecycle accounts (create / suspend / reactivate / reset password)
- Create and configure semesters (start / end / total_weeks / windows)
- Manage course catalog (parallel authority with academic staff)
- View system-wide analytics (users, academic, finance, engagement)
- Configure system settings (institutional currency, JWT expiry, etc.)
- View all student records and grades (no faculty scope)
- Override any audit-trailed action when institutionally justified

Screens (Frontend Routes)

Route	Page
`/admin`	Dashboard
`/admin/profile`	Profile
`/admin/users`	Users Manager (search, filter, create, edit, reset)
`/admin/students`	Students roster (global)
`/admin/courses`	Course Catalog manager
`/admin/semesters`	Semesters (create, dates, windows)
`/admin/registrations`	Registrations (read across faculties)
`/admin/analytics`	System Analytics (totals, KPIs, charts)
`/admin/settings`	Global settings
`/admin/change-password`	Change password

Key API Endpoints

Endpoint	Purpose
`POST /users`	Create user `{email, full_name, role, faculty_id, ...}`
`PATCH /users/{id}`	Activate, suspend, edit
`POST /users/{id}/reset-password`	Generate new temporary password
`GET /users`	List with filters
`POST /semesters`	Create semester
`PATCH /semesters/{id}`	Adjust windows
`GET /admin/analytics`	Aggregate stats
`GET /admin/settings` + `PATCH /admin/settings`	Read / write system settings

Permissions Matrix

Action	Allowed?
Provision user	✅
Suspend / reset / reactivate user	✅
View / edit ANY user's profile	✅
Create semester	✅
Edit semester windows	✅
Edit course catalog	✅
Approve registrations	✅ (parallel with academic staff)
Issue invoices / record payments	❌ (Finance role; admin would create a finance staff account instead)
View all grades (no faculty scope)	✅
Publish / edit grades	✅ (corrective)
Send broadcast message	✅
Send / receive direct messages	❌ (out of scope per role matrix)

Business Rules

- **Lifecycle audit.** Every user lifecycle action (CREATE_USER, SUSPEND_USER, RESET_PASSWORD, REACTIVATE_USER) writes to `audit_log` with actor, target, timestamp, and IP.

- **Token rotation.** Password reset issues a new temporary password and forces `is_first_login = true`.
- **Role assignment.** Setting a user's role is admin-only. Demotions are audit-logged with reason.
- **System settings**. Changes to `JWT_EXPIRE_MINUTES`, allowed origins, etc. are validated against safe ranges.
- **Semester windows.** Admin can edit any semester; academic staff can only edit windows of the current/upcoming term.

Related Diagrams

- user-scenarios.md
- use-cases.md
- activity-diagrams.md
- sequence-diagrams.md
- diagrams/ — rendered PNGs of every Mermaid diagram (auto-generated)

System Admin — User Scenarios

System Admin — User Scenarios

Scenario 1: Provisioning a new student account

Actor: System Administrator (`admin@cis.edu`) **Precondition:** New student admitted to BSE program, ready for Spring 2026.

Steps: 1. Admin opens `/admin/users` → clicks **Create User**. 2. Form: email = `new.student@cis.edu` , full_name = "Jane Walker", role = "student", program_id = BSE, faculty_id = FCSIT, degree_level = "Bachelor". 3. Optional: temporary password (auto-generated if blank). 4. Submits → `POST /users { ... }` . 5. Backend: - Hashes the temp password with bcrypt - Creates `users` + `students` rows - Sets `is_first_login = true` - Sends notification + email (if SMTP configured) - Audit log entry `CREATE_USER` 6. Admin receives the temp password to share securely with the student.

Postcondition: Jane can log in with the temp password and will be redirected to `/change-password` immediately. Once she changes it, she gets normal access.

Scenario 2: Resetting a user's password

Actor: Admin **Precondition:** Student forgot her password and the email-based reset isn't reaching her institutional inbox.

Steps: 1. Admin opens `/admin/users` → searches "Alice Smith" → clicks her row. 2. Clicks **Reset Password**. 3. Confirmation dialog with new temporary password displayed. 4. `POST /users/{id}/reset-password` . 5. Backend: - Generates new temp password - Re-hashes user.password_hash - Sets `is_first_login = true` - Audit log `RESET_PASSWORD` 6. Admin securely shares the temp password with Alice.

Postcondition: Alice's next login forces a password change.

Scenario 3: Suspending a user

Actor: Admin **Precondition:** Account compromised; need to immediately block access.

Steps: 1. Opens user detail page → clicks **Suspend**. 2. Enters reason "Compromised credentials — pending investigation". 3. `PATCH /users/{id} { is_active: false, status: 'suspended' }` . 4. Backend: - Sets `is_active = false` , `status = 'suspended'` - Invalidates existing sessions (token still valid until expiry; can be paired with JWT secret rotation for hard kill) - Audit log `SUSPEND_USER`

Postcondition: Subsequent logins return 401. Any existing JWTs return 401 because routes recheck `user.is_active` on every request.

Scenario 4: Creating a new semester

Actor: Admin **Precondition:** Spring 2026 ending; Fall 2026 needs to be set up.

Steps: 1. Opens `/admin/semesters` → **New Semester**. 2. Fills: name = "Fall 2026", code = "F26", start_date = 2026-09-01, end_date = 2026-12-22, total_weeks = 14, registration_open_at = 2026-07-15, drop_deadline = 2026-10-10. 3. Submits → `POST /semesters { ... }`.

Postcondition: Fall 2026 visible in pickers. Academic staff can begin creating offerings for the new term.

Scenario 5: Reviewing system analytics

Actor: Admin **Steps:** 1. Opens `/admin/analytics`. 2. Page renders tiles: - Total users (active, pending, suspended) - Total students by faculty - Active semesters - Total billed / collected (finance summary across all faculties) - Engagement (logins last 30 days, message volume, club activity) 3. Drills into anomalies (e.g. spike in failed logins → security review).

Postcondition: Admin has institutional KPIs at a glance. Identified anomalies trigger further action (e.g. user investigation, support ticket).

Scenario 6: Adjusting system settings

Actor: Admin **Precondition:** Need to extend JWT expiry from 8h to 12h for a workshop event where staff stay logged in.

Steps: 1. Opens `/admin/settings`. 2. Edits **JWT expiry minutes** from 480 → 720. 3. Validates against safe range (60..1440). 4. Saves → `PATCH /admin/settings { JWT_EXPIRE_MINUTES: 720 }`. 5. Audit log entry.

Postcondition: New tokens issued after this change live 12h. Existing tokens are unaffected.

System Admin — Use Cases

System Admin — Use Cases

Use Case Diagram

[Mermaid diagram — see PNG in diagrams/ folder]

Use Case Descriptions

UC1 — Create user

Main flow: `POST /users { email, full_name, role, faculty_id, program_id, degree_level }`.
Side effect: Creates `users` + role-specific row (`students` / `staff_profiles` / `instructors`).
Postcondition: `is_first_login = true` ; user must change password on first login.

UC2 — Edit user

Main flow: `PATCH /users/{id} { fields }` . Cannot change email on existing user (institutional policy).

UC3 — Reset password

Main flow: `POST /users/{id}/reset-password` . Generates temp password; sets `is_first_login = true` .

UC4 — Suspend user

Main flow: `PATCH /users/{id} { is_active: false, status: 'suspended' }` . Backend re-checks `is_active` on every API call.

UC5 — Reactivate user

Main flow: `PATCH /users/{id} { is_active: true, status: 'active' }` .

UC6 — Assign role / faculty

Main flow: Same as UC2 but with role change. Demotions audit-logged with reason.

UC8 — Create semester

Main flow: `POST /semesters { name, code, dates, total_weeks }` .

UC9 — Edit semester windows

Main flow: `PATCH /semesters/{id} { registration_open_at, drop_deadline }`.

UC13 — View analytics

Main flow: `GET /admin/analytics` aggregates across all faculties.

UC15 — Edit system settings

Main flow: `PATCH /admin/settings { ... }`. Each setting has safe-range validation.

UC16 — View audit logs

Main flow: `GET /admin/audit-log?action=...&actor=...&since=...&until=...`.

UC17 — Send broadcast

Main flow: `POST /messages { broadcast: true, ... }`. Backend allows because admin role.

System Admin — Activity Diagrams

System Admin — Activity Diagrams

A1 — User provisioning

[Mermaid diagram — see PNG in diagrams/ folder]

A2 — Password reset

[Mermaid diagram — see PNG in diagrams/ folder]

A3 — Suspend / reactivate

[Mermaid diagram — see PNG in diagrams/ folder]

A4 — Create semester

[Mermaid diagram — see PNG in diagrams/ folder]

A5 — System settings change

[Mermaid diagram — see PNG in diagrams/ folder]

System Admin — Sequence Diagrams

System Admin — Sequence Diagrams

S1 — Provision new user

[Mermaid diagram — see PNG in diagrams/ folder]

S2 — Reset password

[Mermaid diagram — see PNG in diagrams/ folder]

S3 — Suspend (with effect on existing token)

[Mermaid diagram — see PNG in diagrams/ folder]

S4 — Edit semester windows

[Mermaid diagram — see PNG in diagrams/ folder]

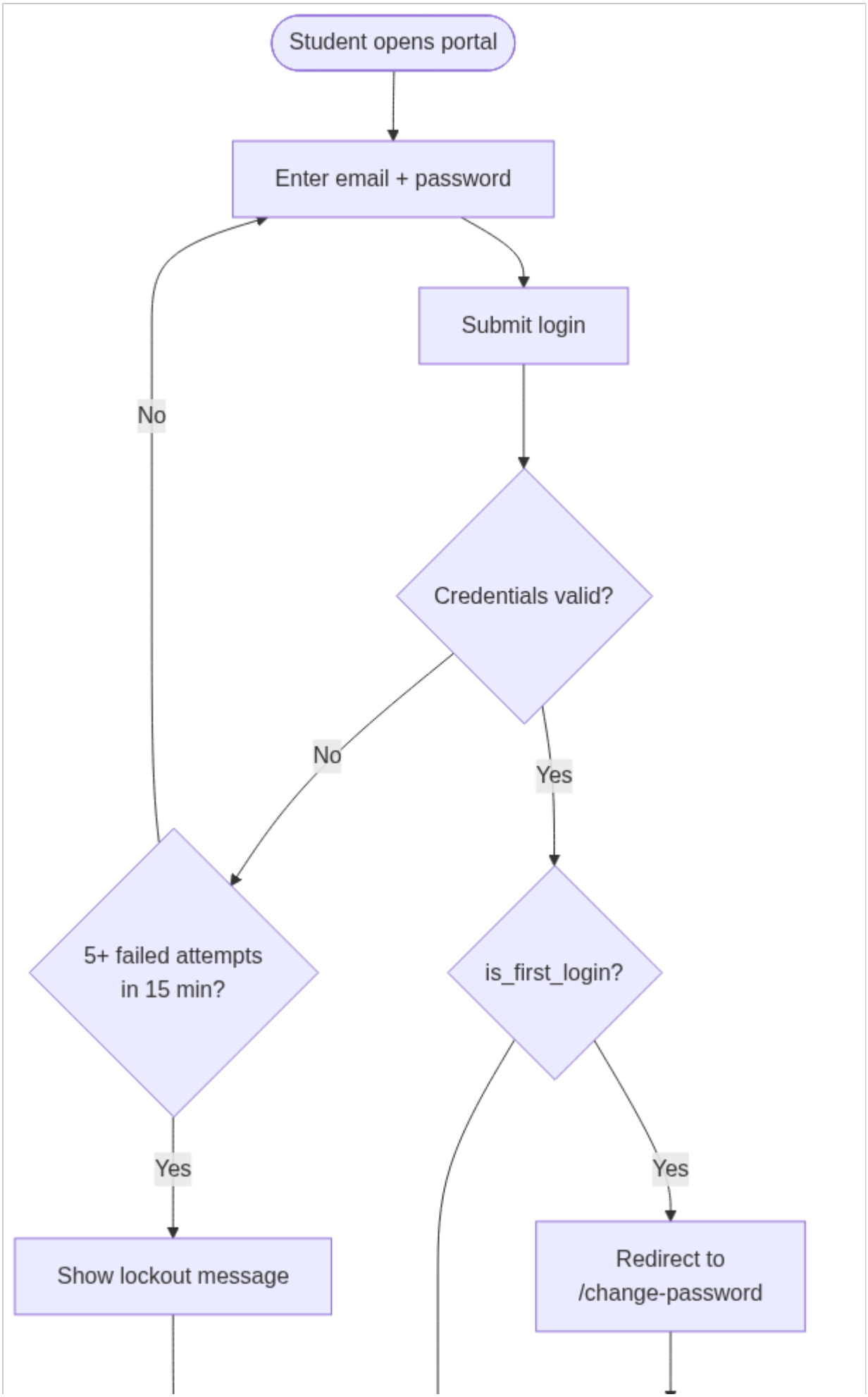
S5 — Analytics snapshot

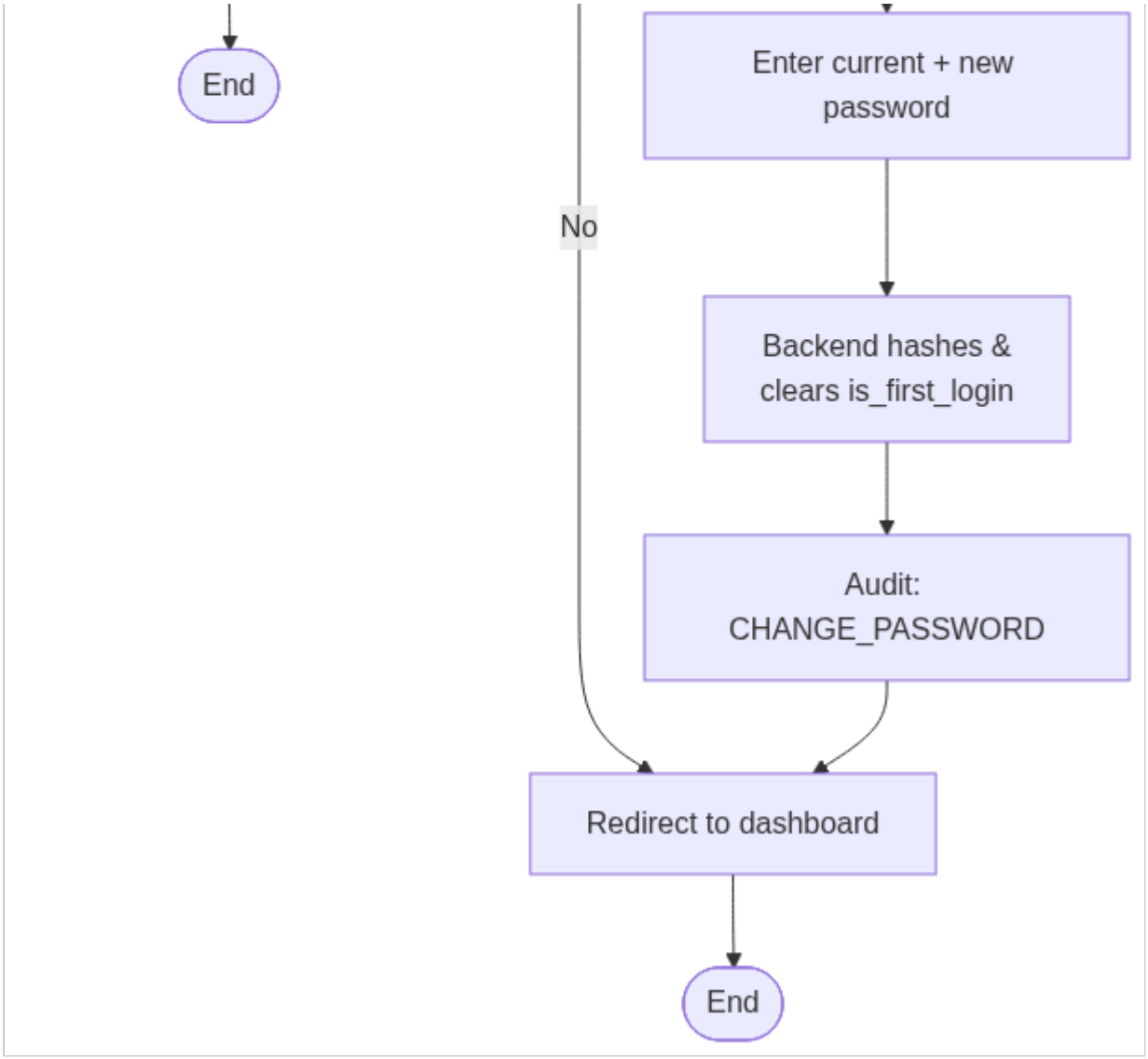
[Mermaid diagram — see PNG in diagrams/ folder]

Appendix A — Rendered Role Diagrams

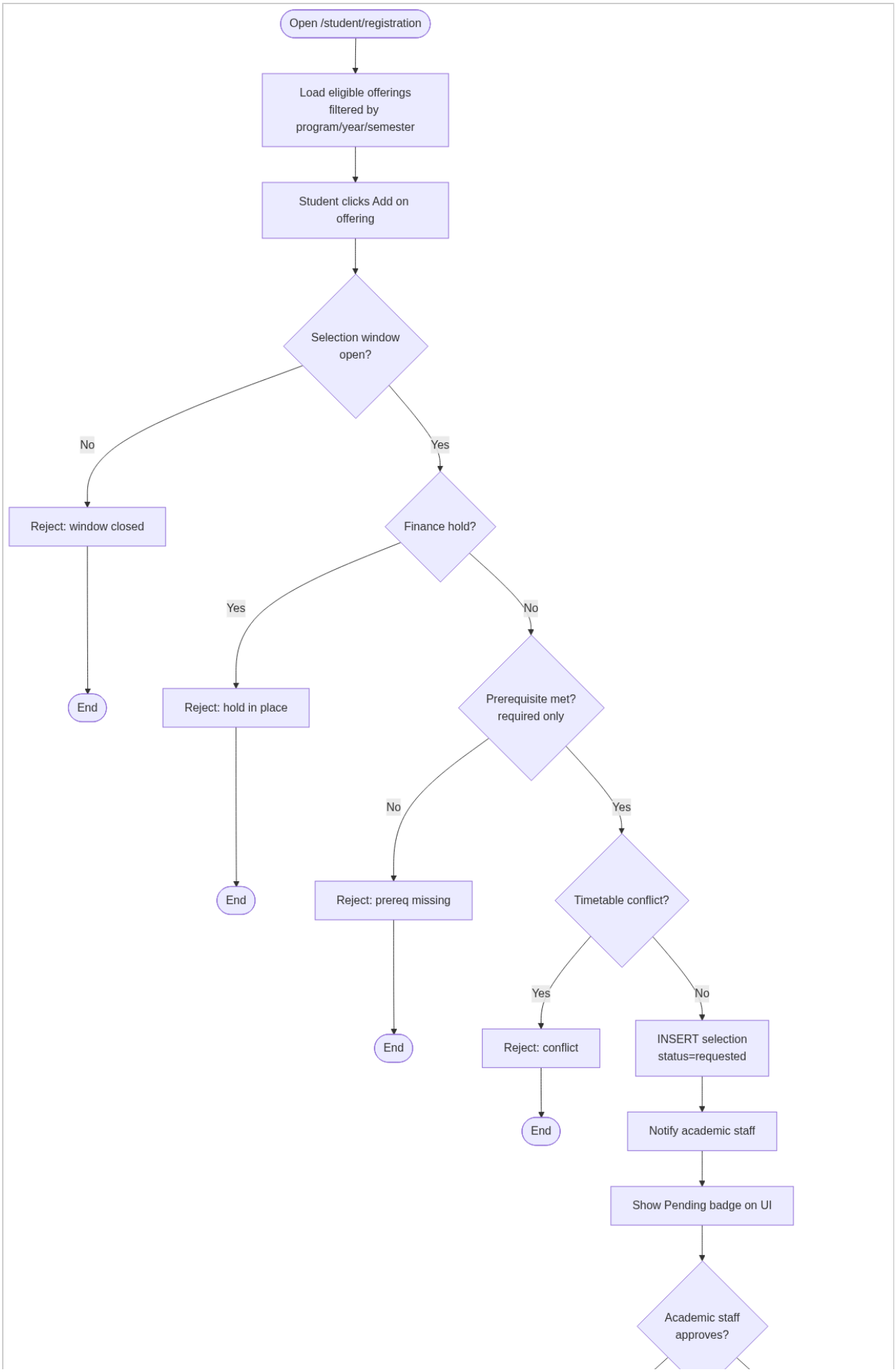
Student

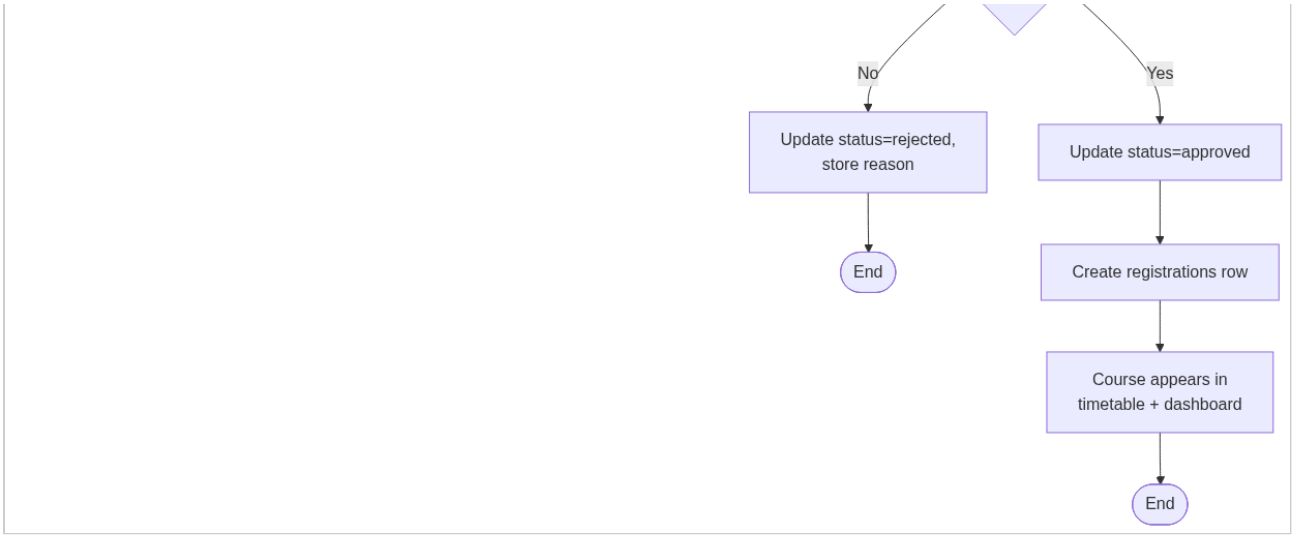
activity-diagrams-01


Student opens portal
Enter email + password
Submit login
Credentials valid?
No
Yes
5+ failed attempts in 15 min?
No
is_first_login?
Yes
Yes
Show lockout message
Redirect to /change-password

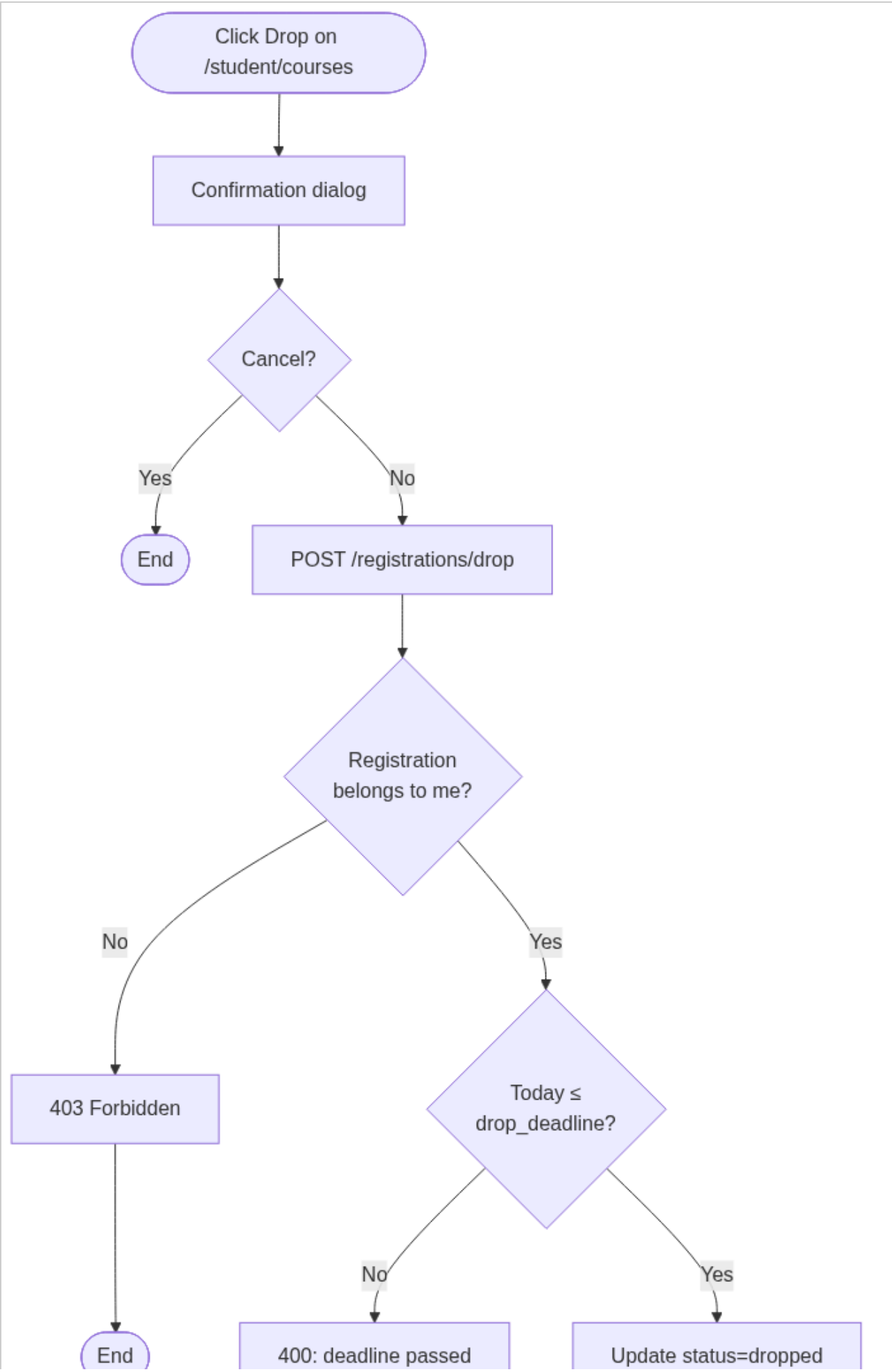


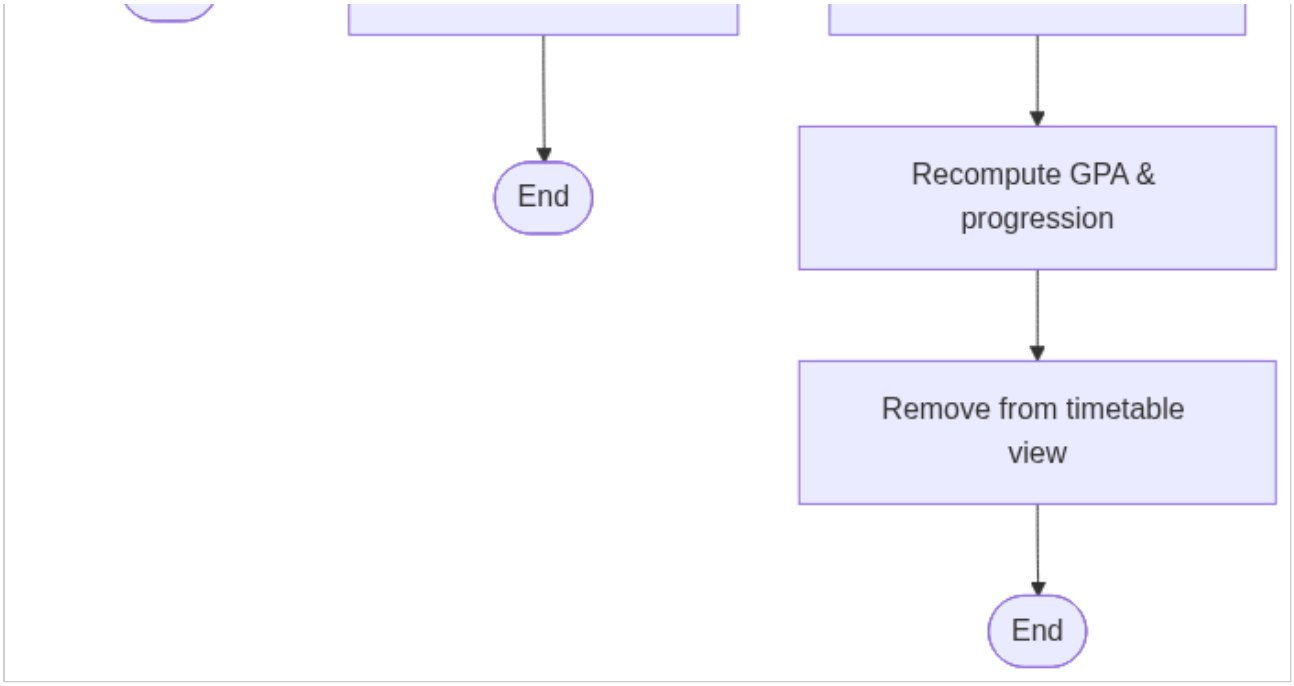
activity-diagrams-02


Open /student/registration
Load eligible offerings filtered by program/year/semester
Student clicks Add on offering
Selection window open?
No
Yes
Reject: window closed
End
Finance hold?
Yes
No
Reject: hold in place
End
Prerequisite met? required only
No
Yes
Reject: prereq missing
End
Timetable conflict?
Yes
No
Reject: conflict
End
INSERT selection status=requested
Notify academic staff
Show Pending badge on UI
Academic staff approves?

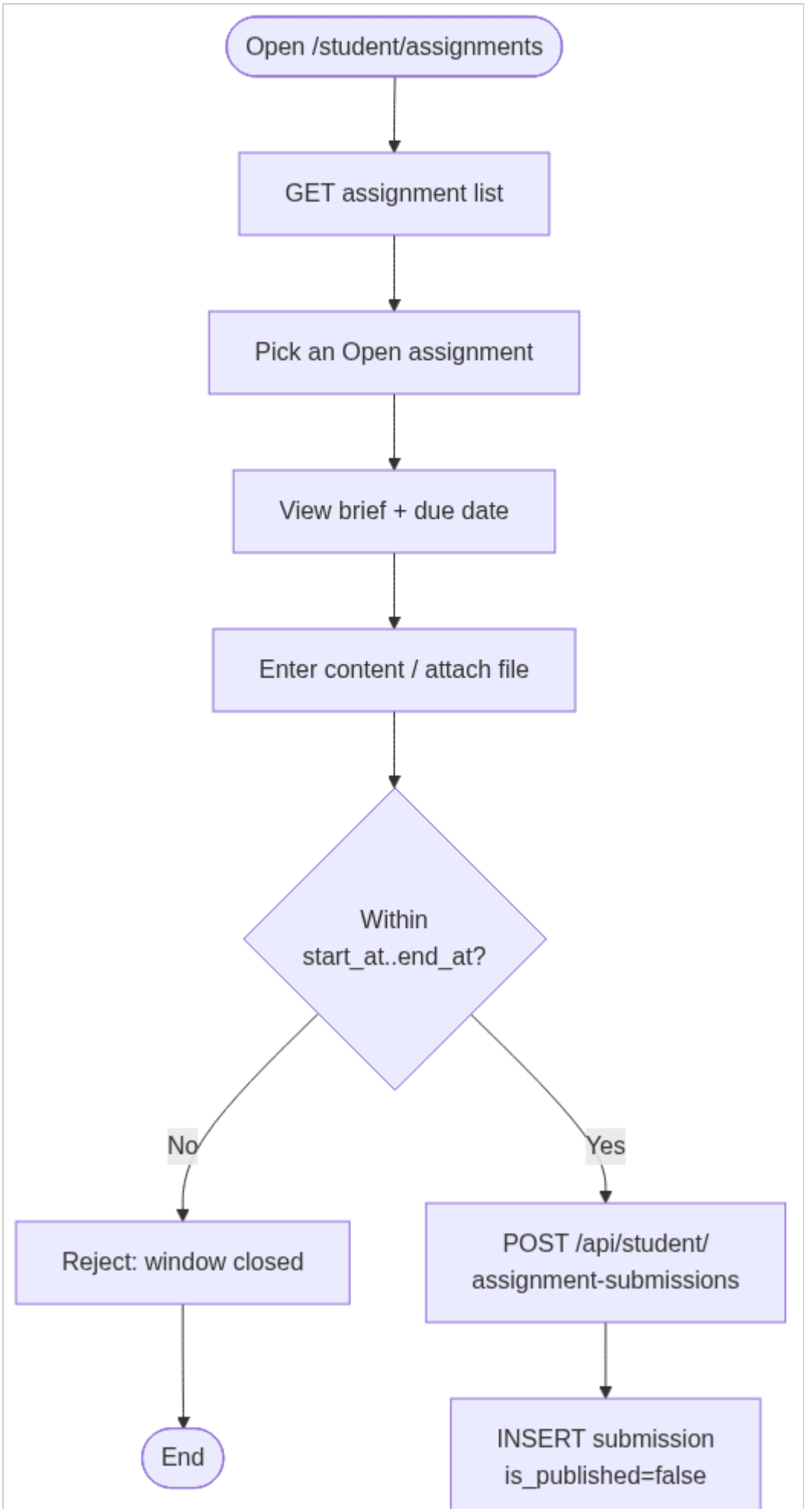


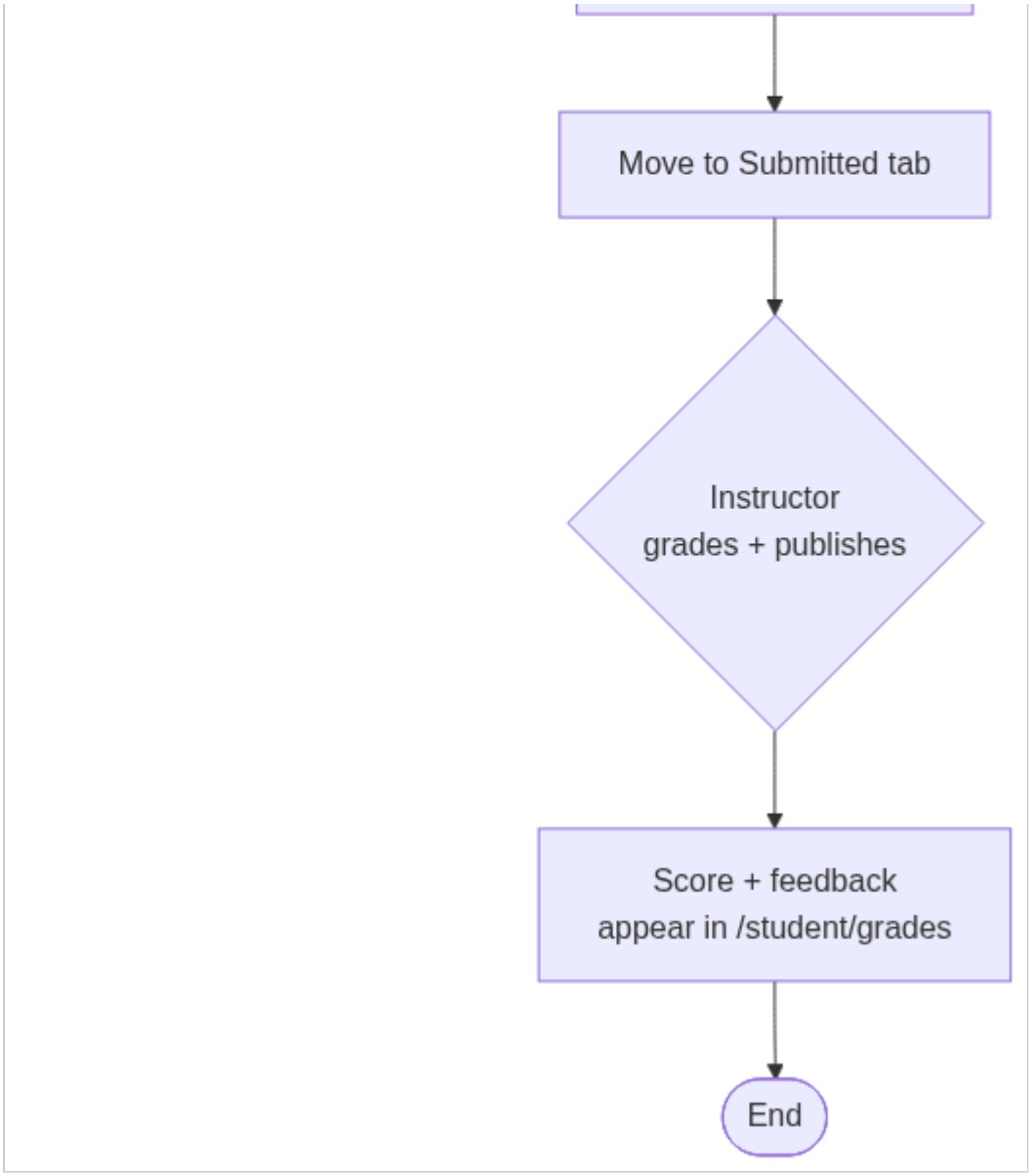
activity-diagrams-03


Click Drop on /student/courses
Confirmation dialog
Cancel?
Yes
No
End
POST /registrations/drop
Registration belongs to me?
No
Yes
403 Forbidden
Today ≤ drop_deadline?
End
No
Yes
400: deadline passed
Update status=dropped

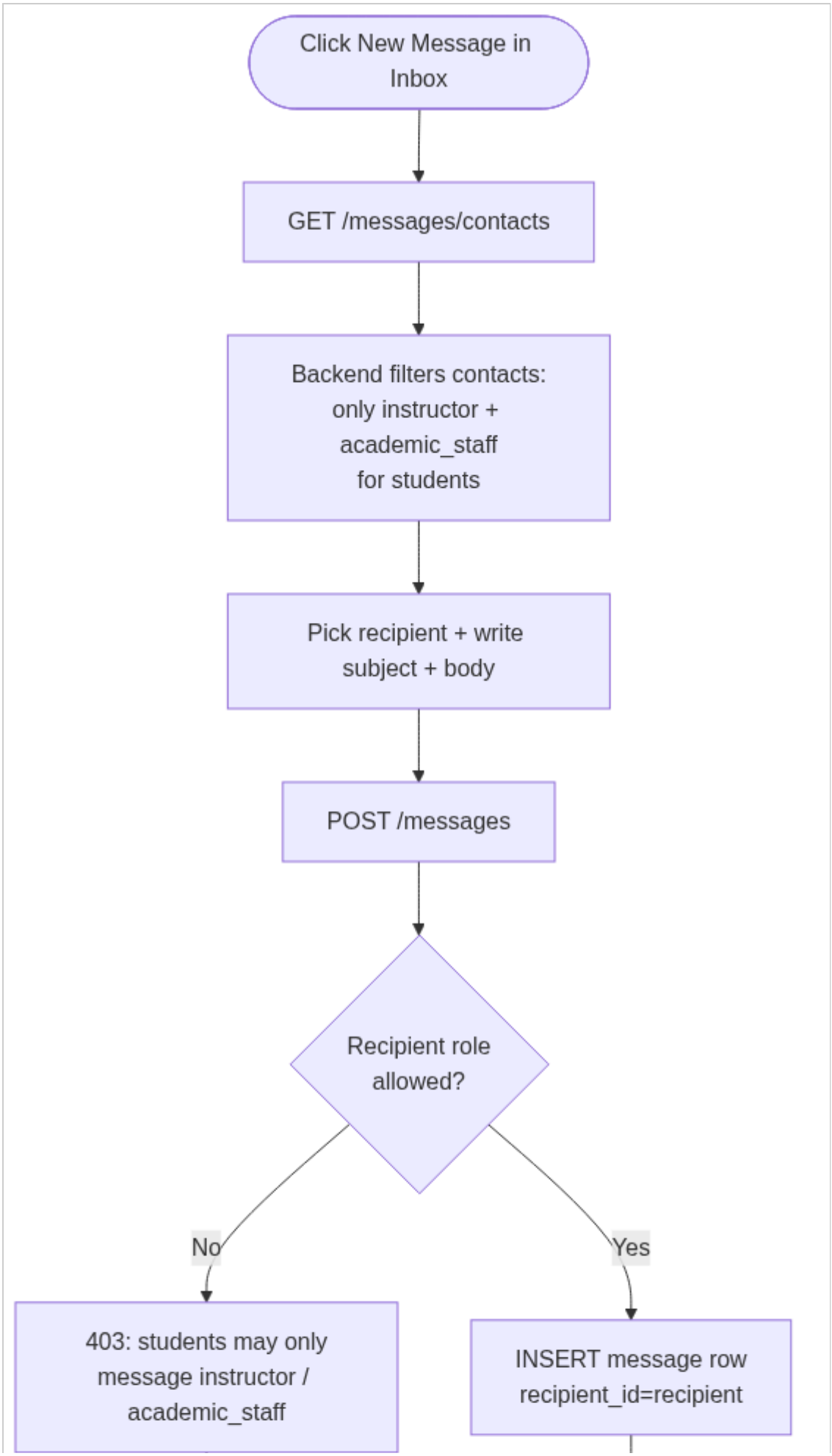


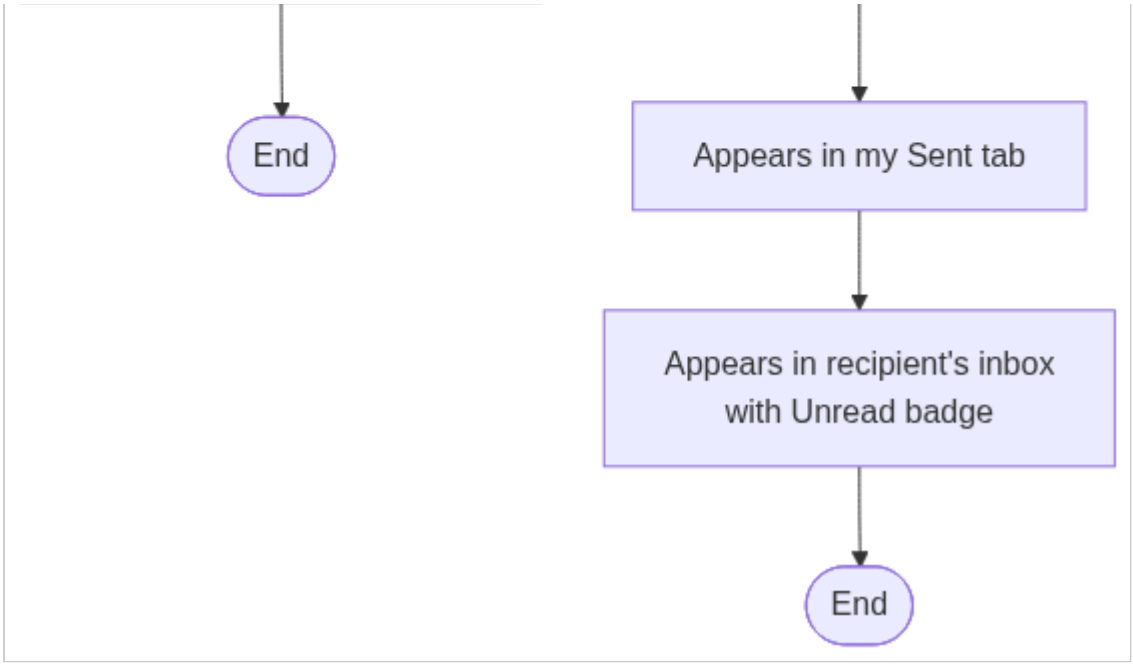
activity-diagrams-04


Open /student/assignments
GET assignment list
Pick an Open assignment
View brief + due date
Enter content / attach file
Within start_at..end_at?
No
Yes
Reject: window closed
End
POST /api/student/ assignment-submissions
INSERT submission is_published=false

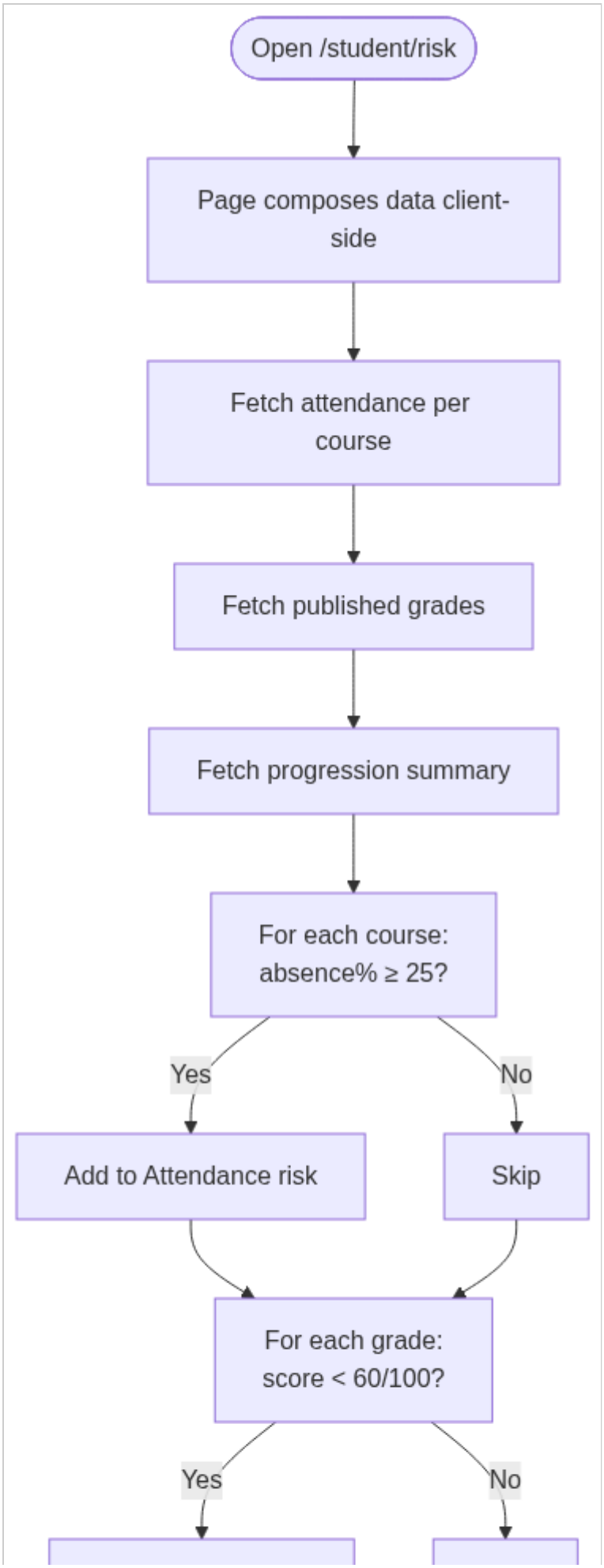


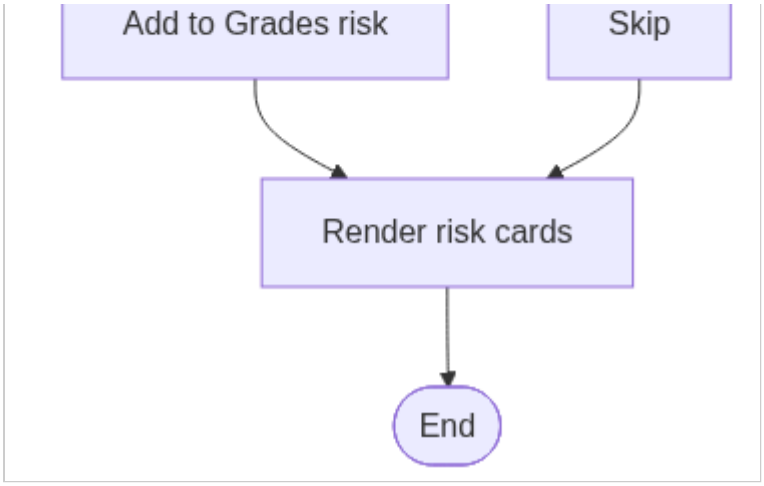
activity-diagrams-05


Click New Message in Inbox
GET /messages/contacts
Backend filters contacts: only instructor + academic_staff for students
Pick recipient + write subject + body
POST /messages
Recipient role allowed?
No
Yes
403: students may only message instructor / academic_staff
INSERT message row recipient_id=recipient

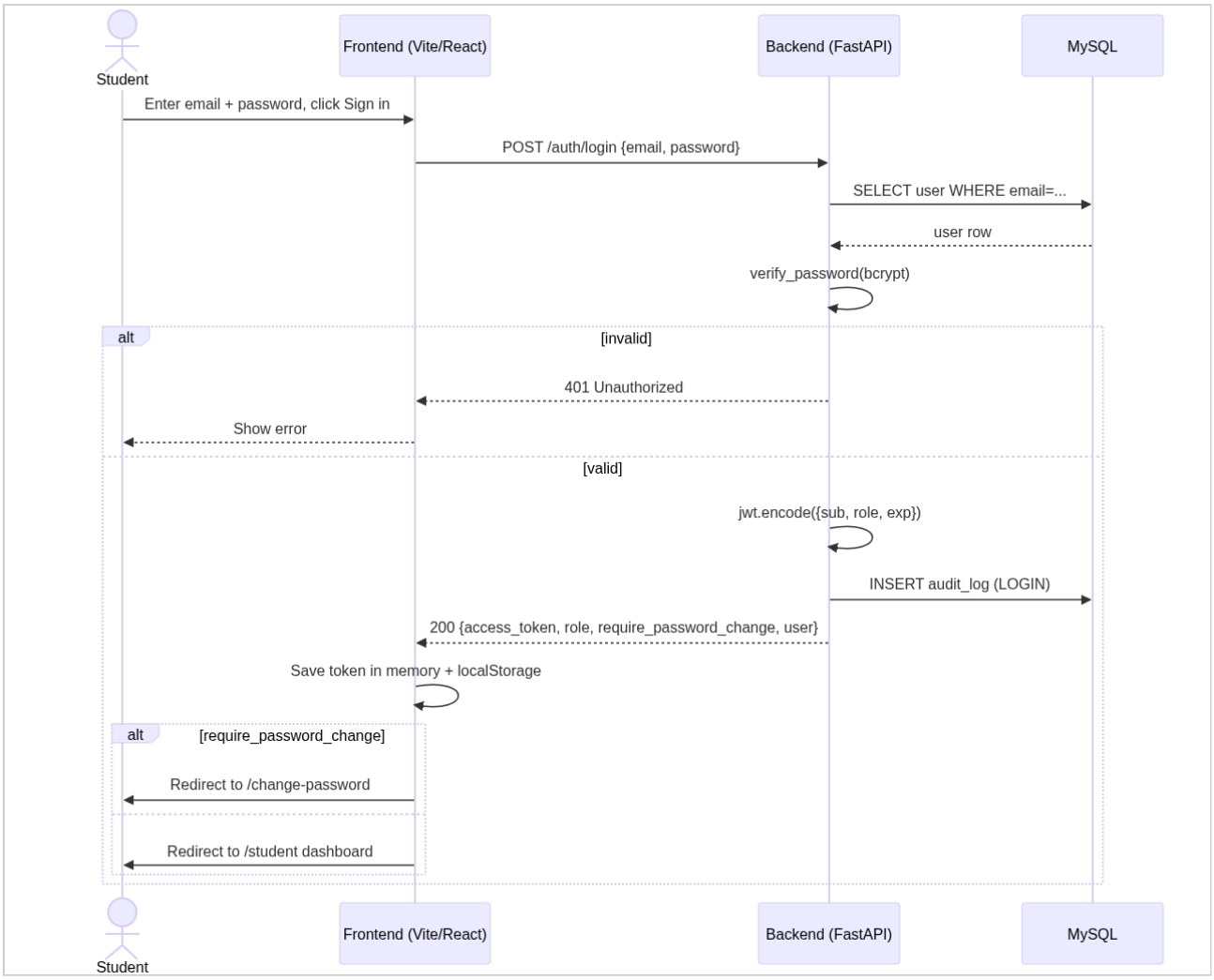


activity-diagrams-06

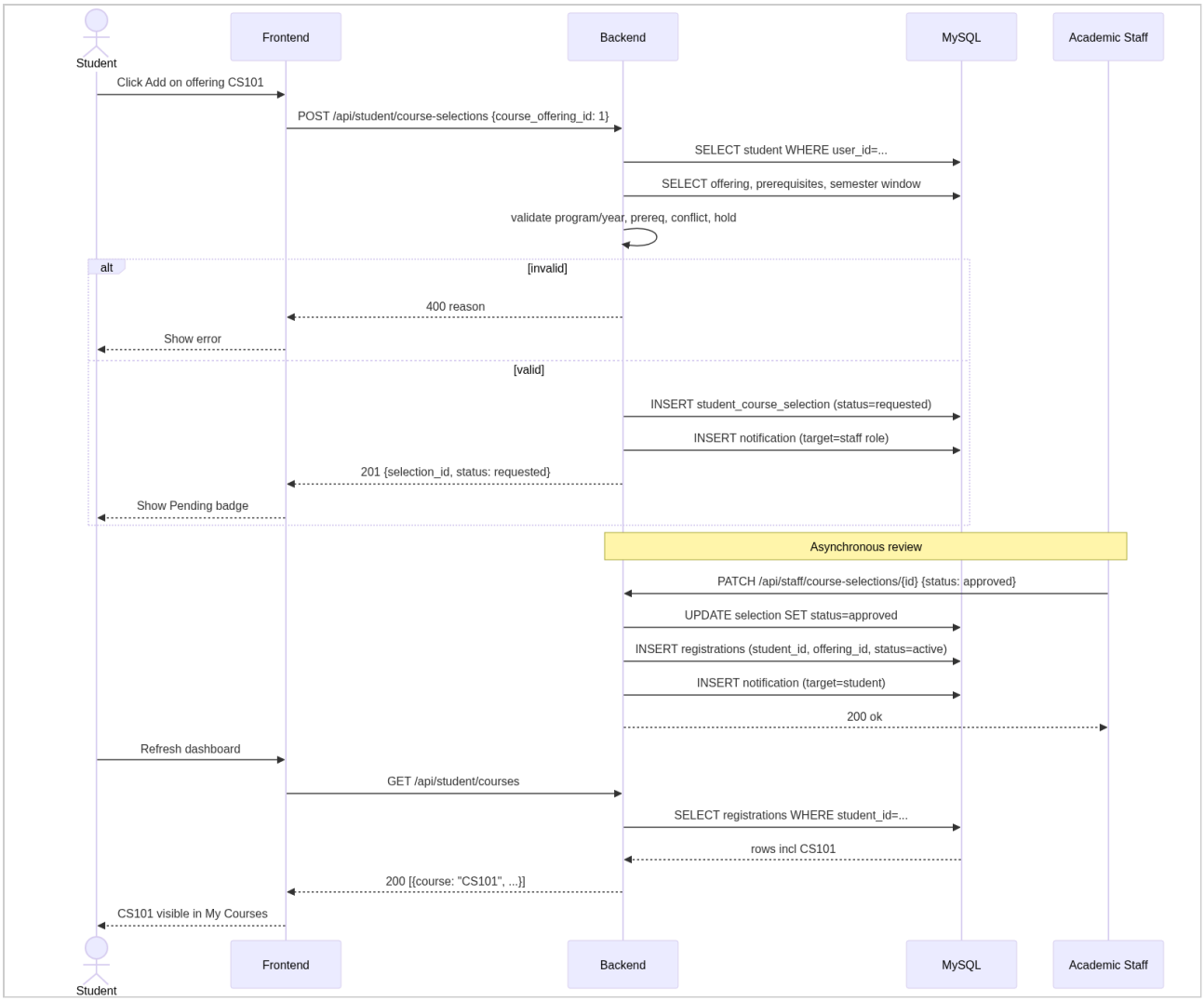

Open /student/risk
Page composes data client-side
Fetch attendance per course
Fetch published grades
Fetch progression summary
For each course: absence% ≥ 25?
Yes
No
Add to Attendance risk
Skip
For each grade: score < 60/100?
Yes
No



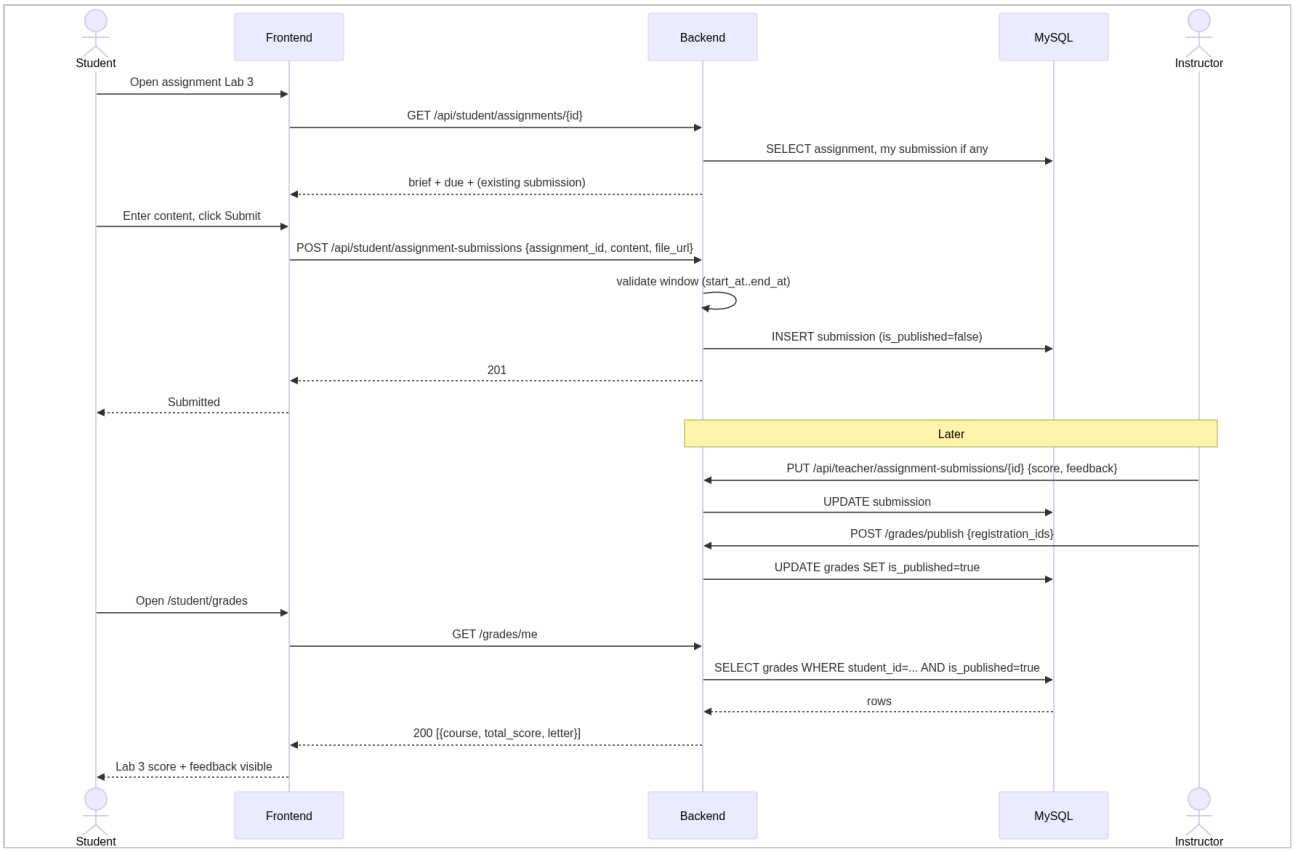
sequence-diagrams-01



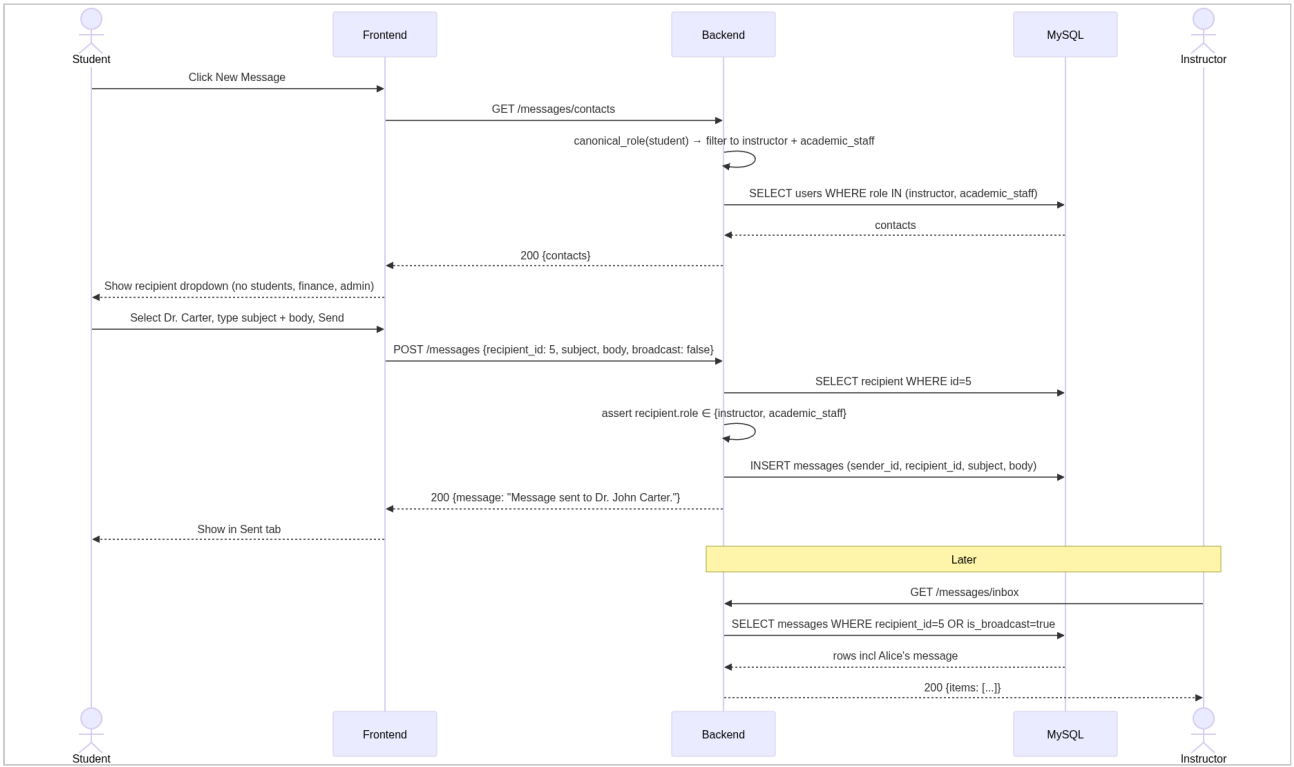
sequence-diagrams-02



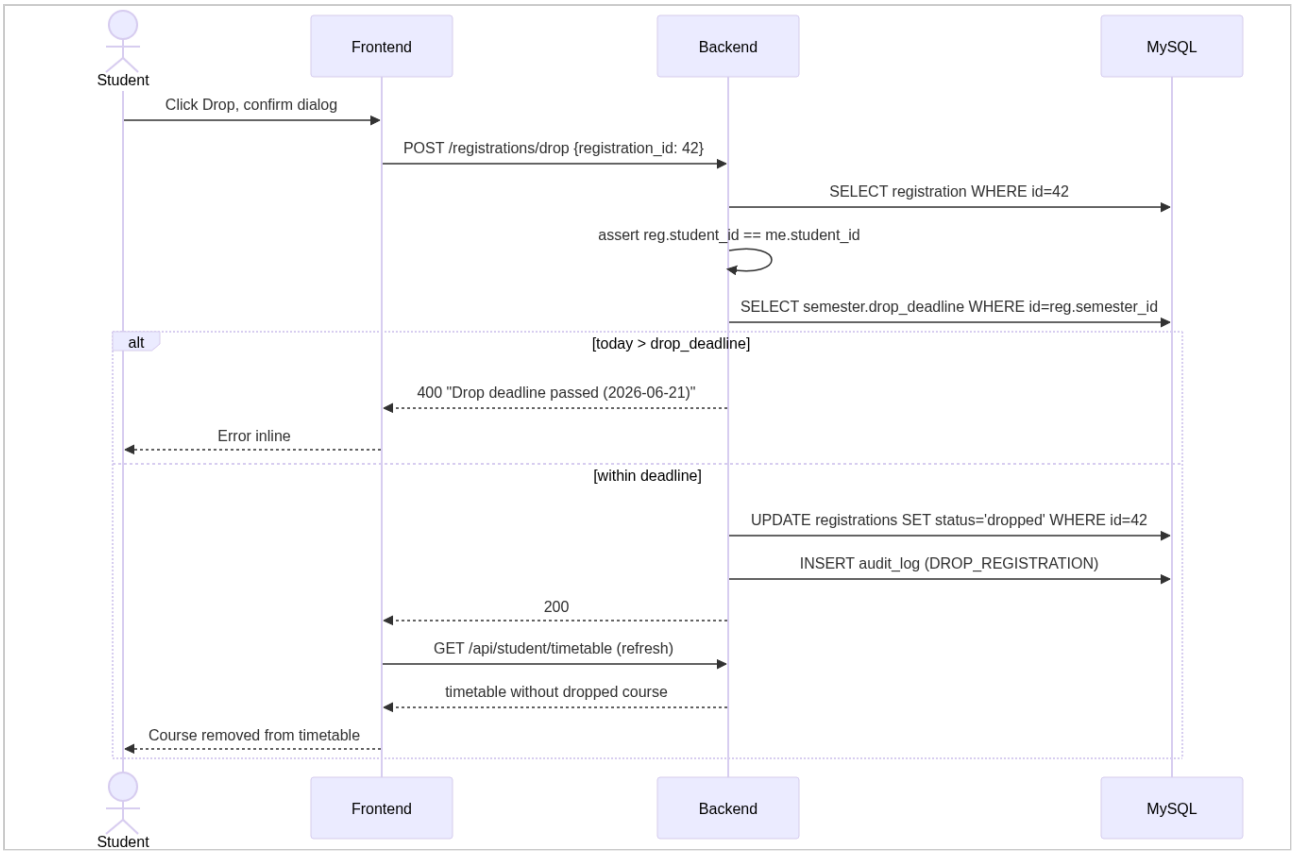
sequence-diagrams-03



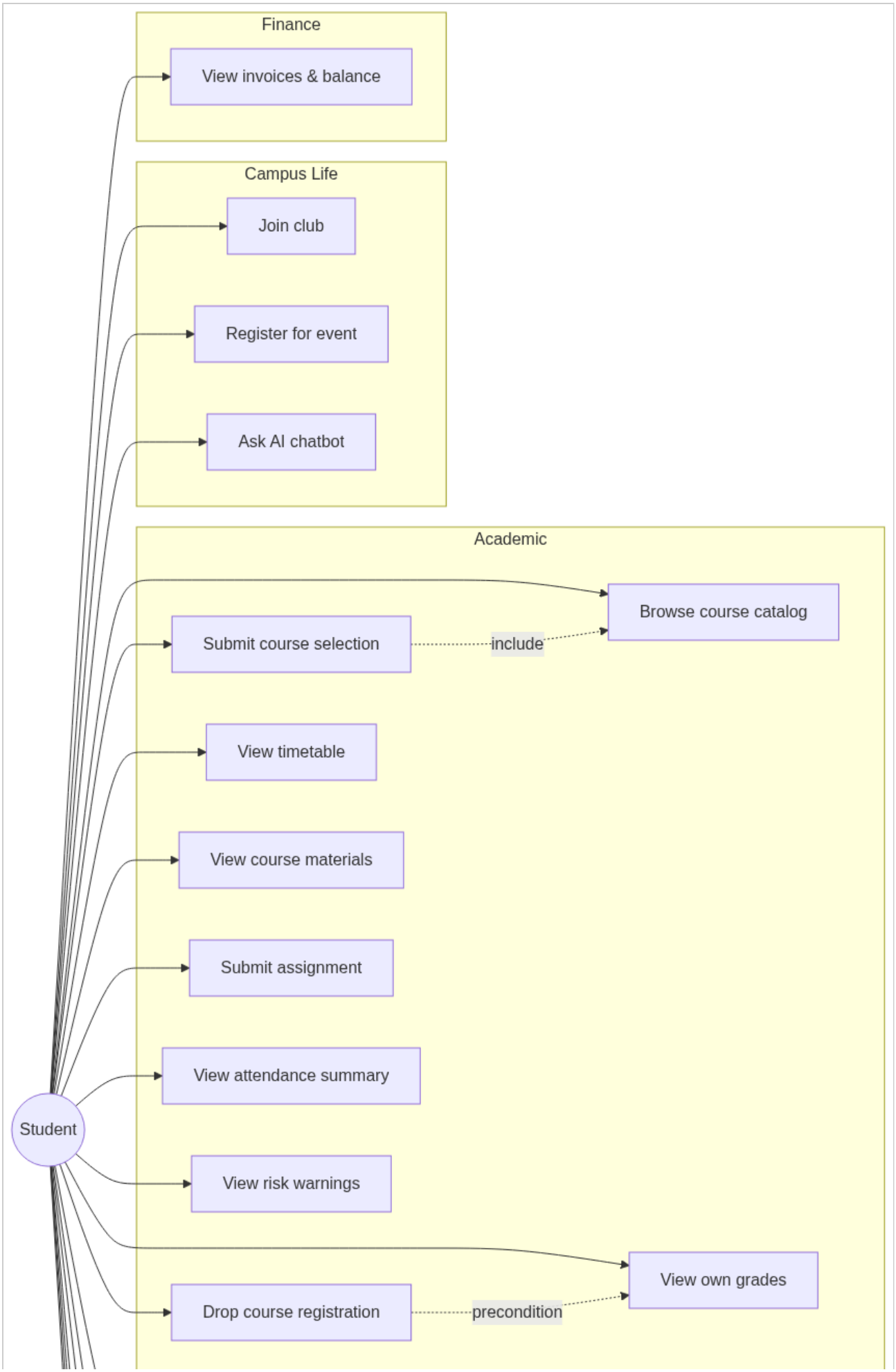
sequence-diagrams-04

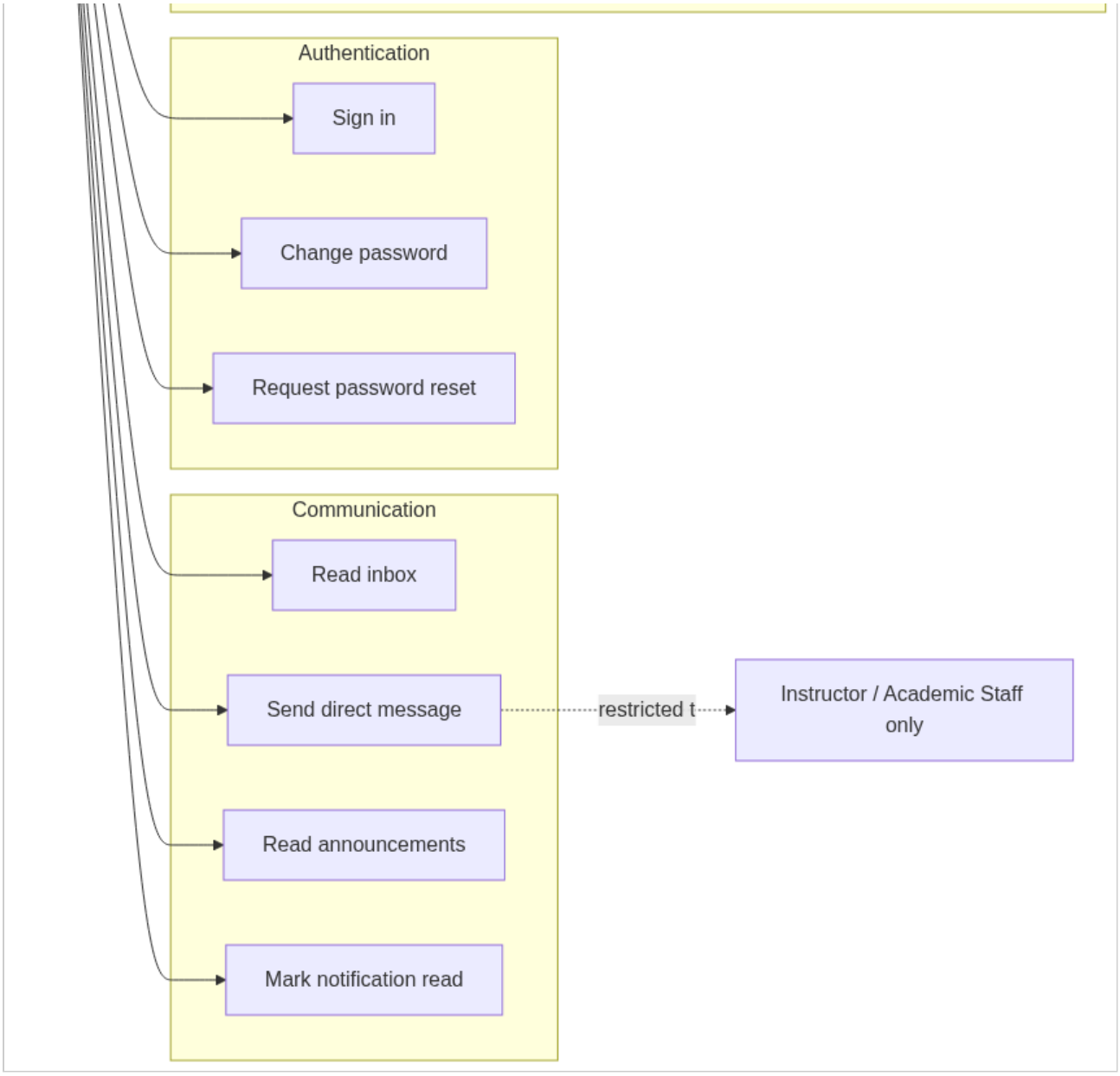


sequence-diagrams-05


Student
Frontend
Backend
MySQL
Click Drop, confirm dialog
POST /registrations/drop {registration_id: 42}
SELECT registration WHERE id=42
assert reg.student_id == me.student_id
SELECT semester.drop_deadline WHERE id=reg.semester_id
alt
[today > drop_deadline]
400 "Drop deadline passed (2026-06-21)"
Error inline
[within deadline]
UPDATE registrations SET status='dropped' WHERE id=42
INSERT audit_log (DROP_REGISTRATION)
200
GET /api/student/timetable (refresh)
timetable without dropped course
Course removed from timetable
Student
Frontend
Backend
MySQL

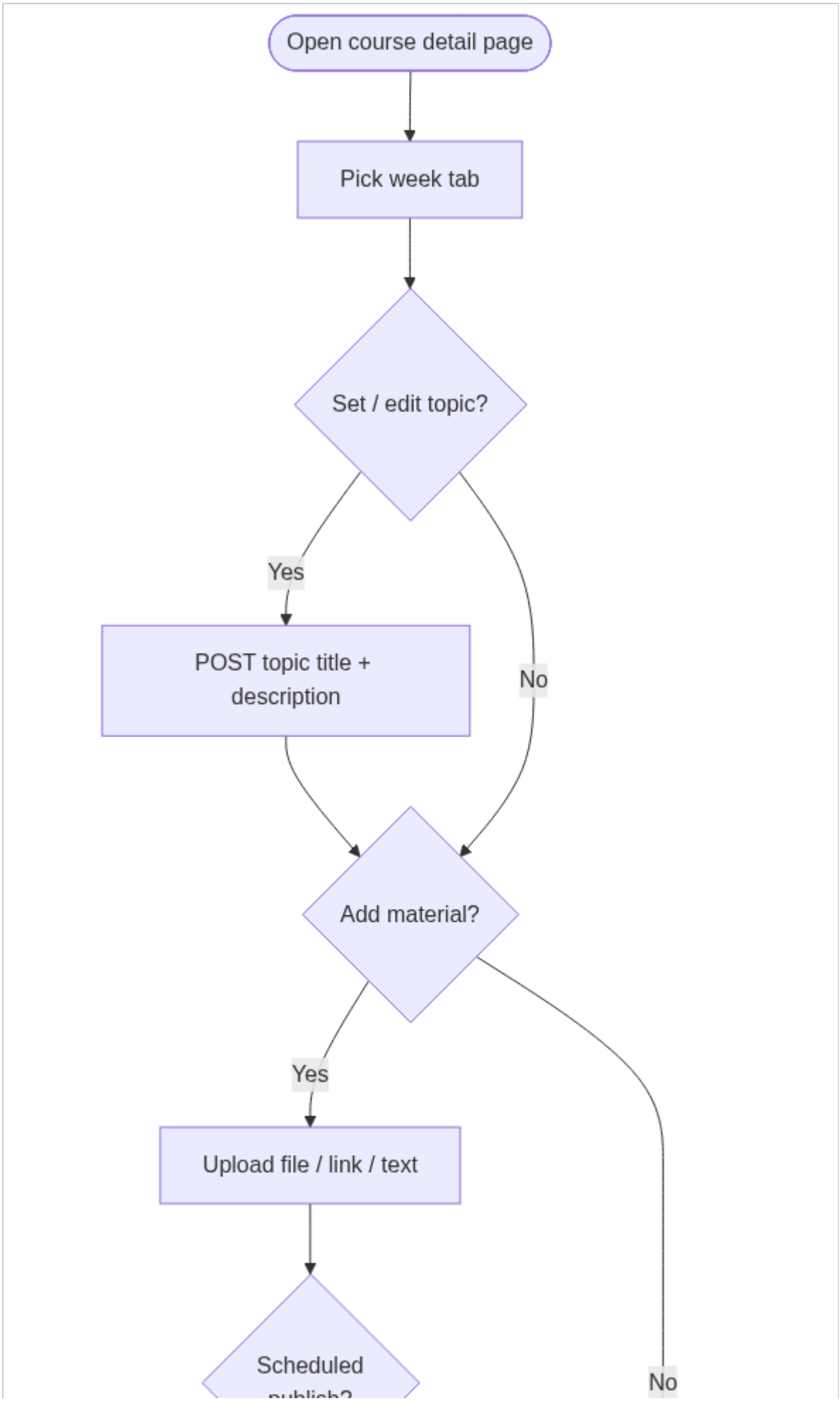
use-cases-01

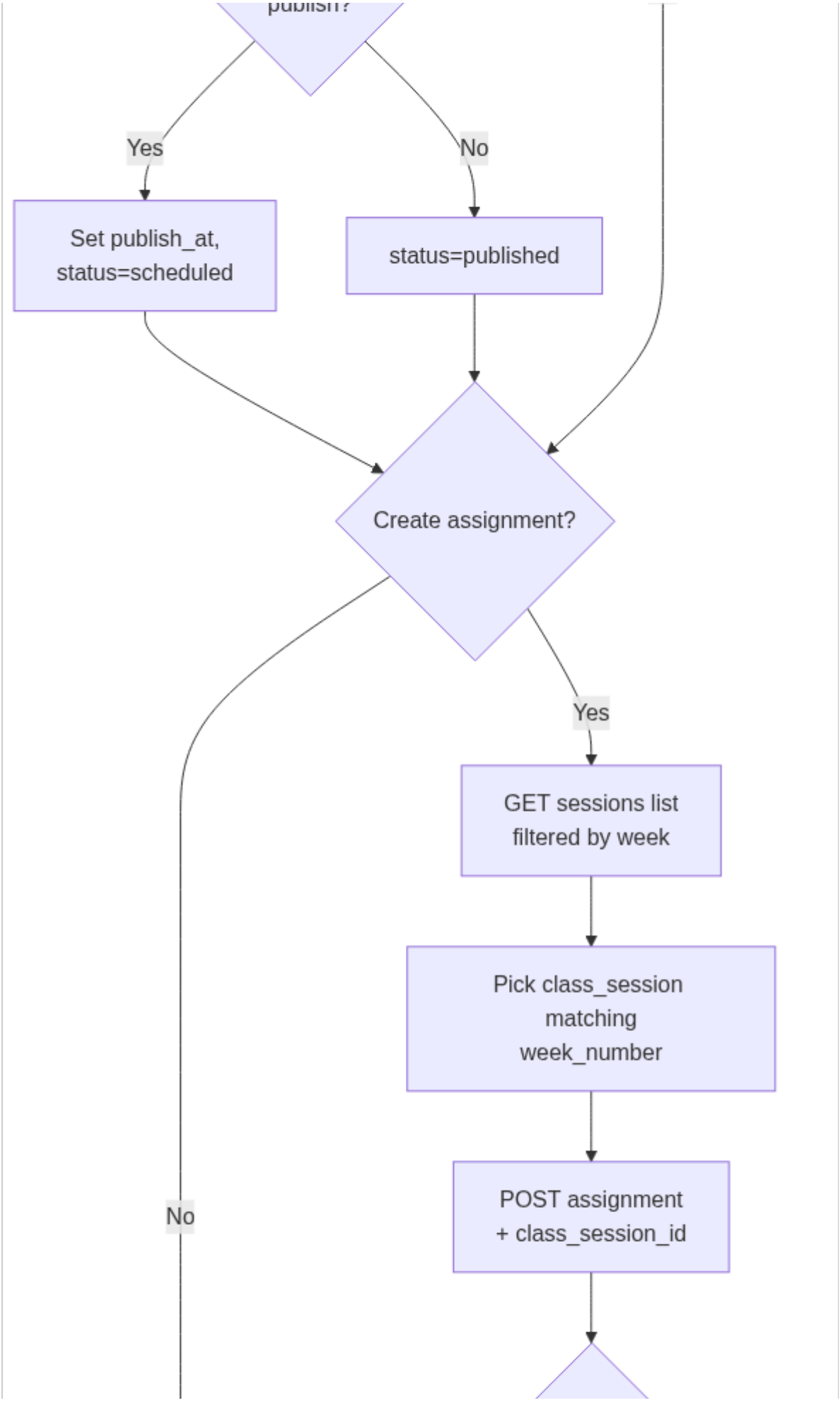

Finance
View invoices & balance
Campus Life
Join club
Register for event
Ask AI chatbot
Academic
Browse course catalog
Submit course selection
include
View timetable
View course materials
Submit assignment
View attendance summary
Student
View risk warnings
View own grades
Drop course registration
precondition

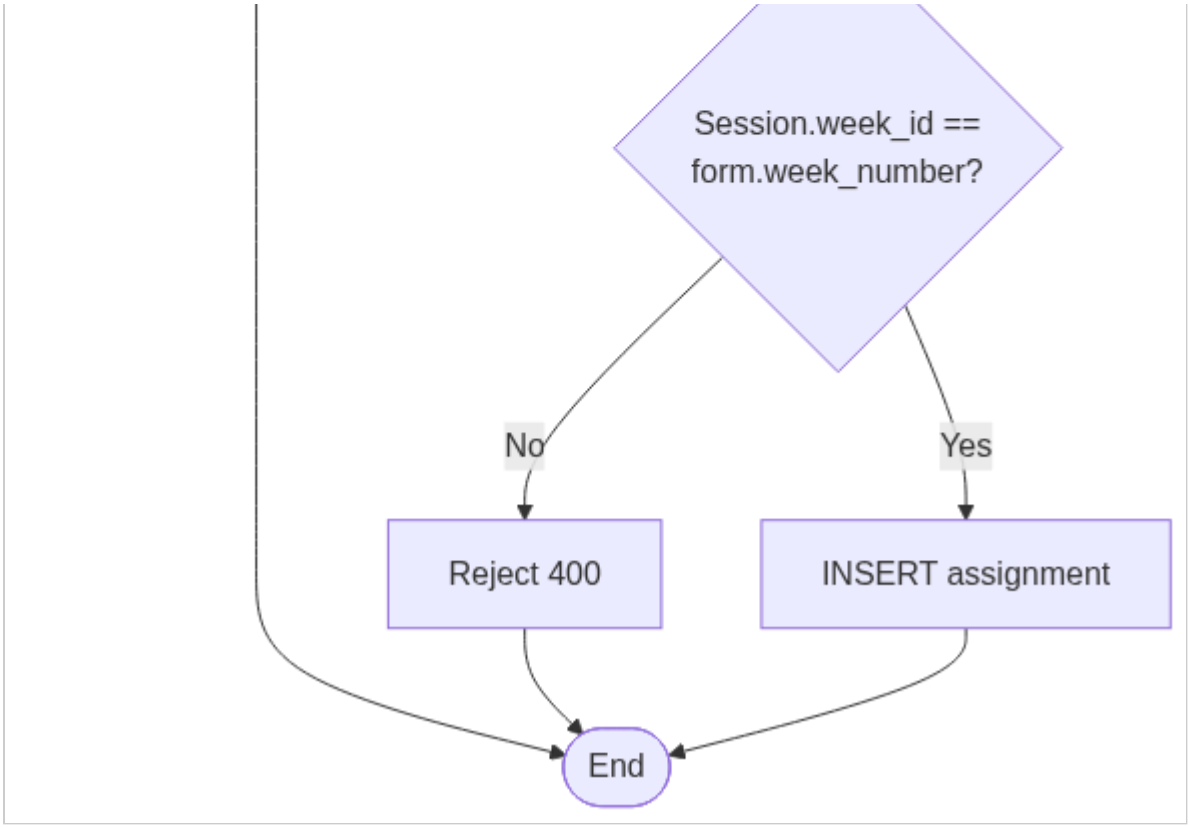


Instructor

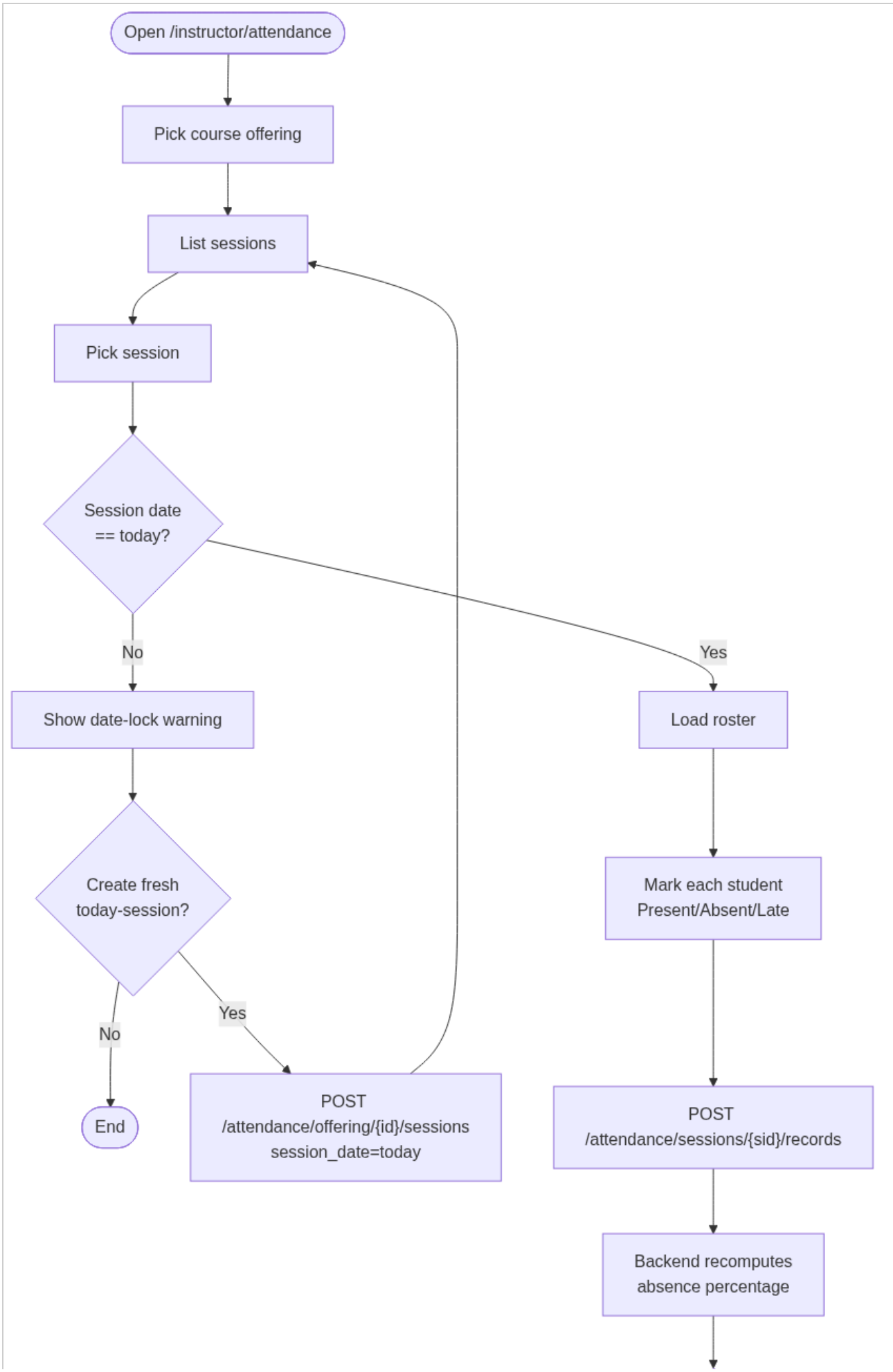
activity-diagrams-01

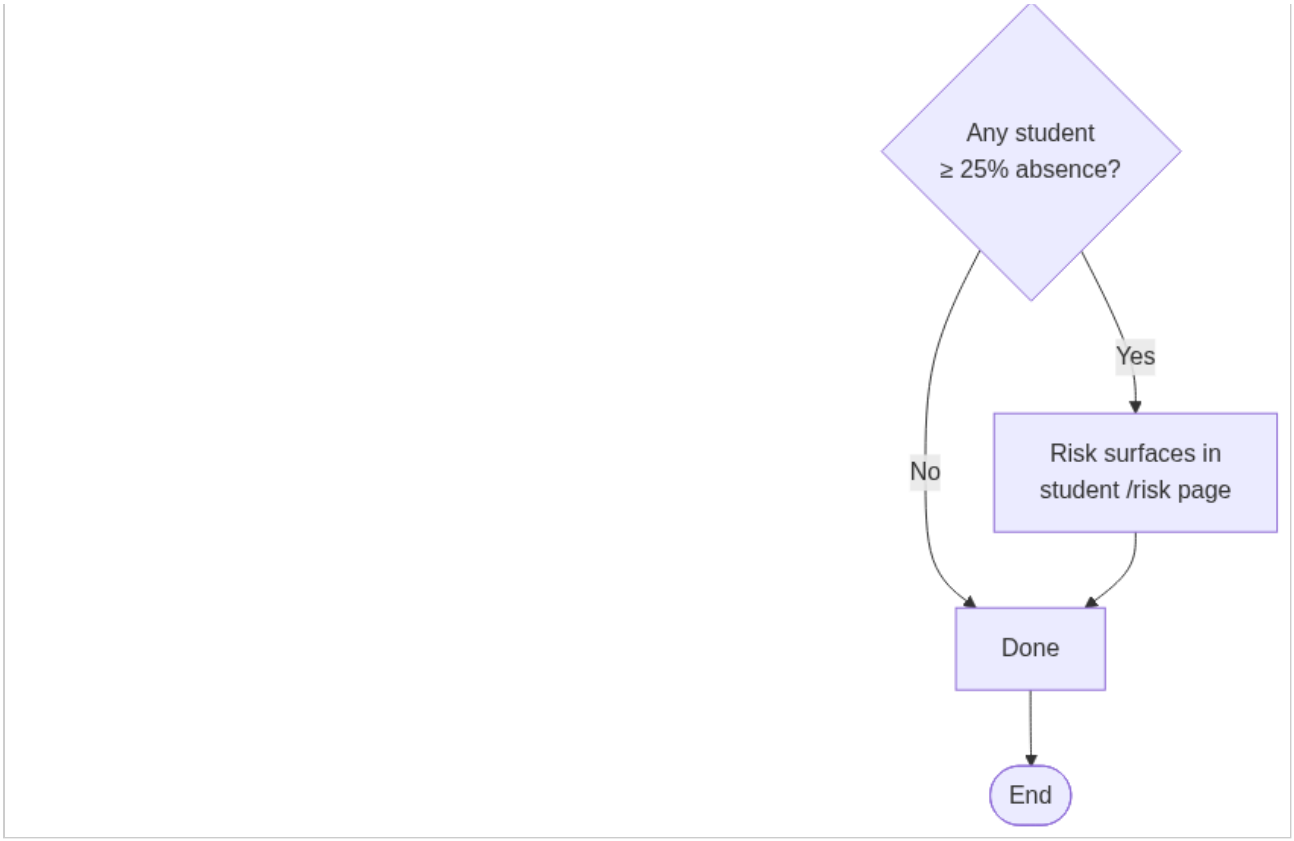

Open course detail page
Pick week tab
Set / edit topic?
Yes
No
POST topic title + description
Add material?
Yes
No
Upload file / link / text
Scheduled


publish?
Yes
No
Set publish_at,
status=scheduled
status=published
Create assignment?
Yes
GET sessions list
filtered by week
Pick class_session
matching
week_number
POST assignment
+ class_session_id
No

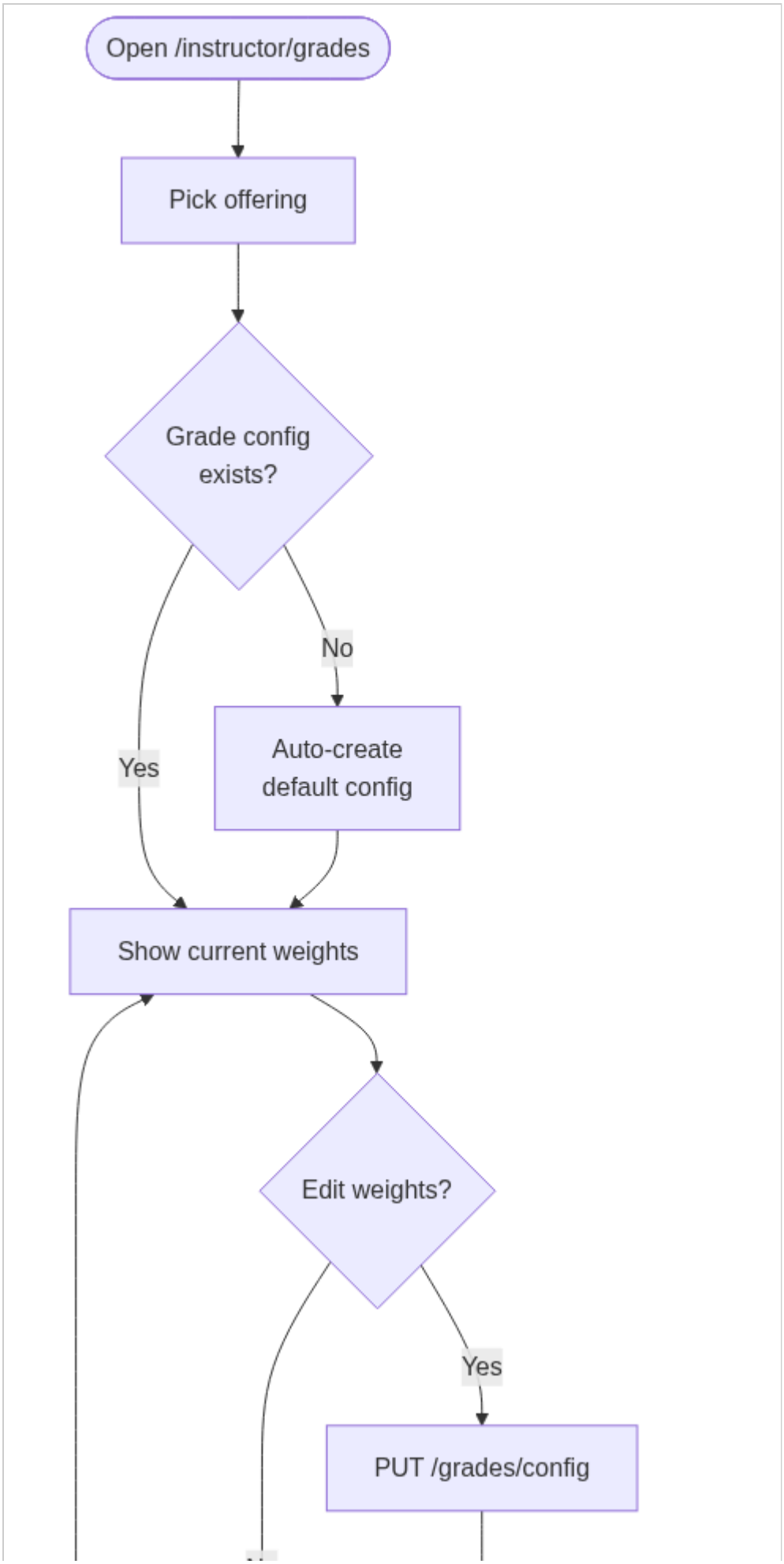


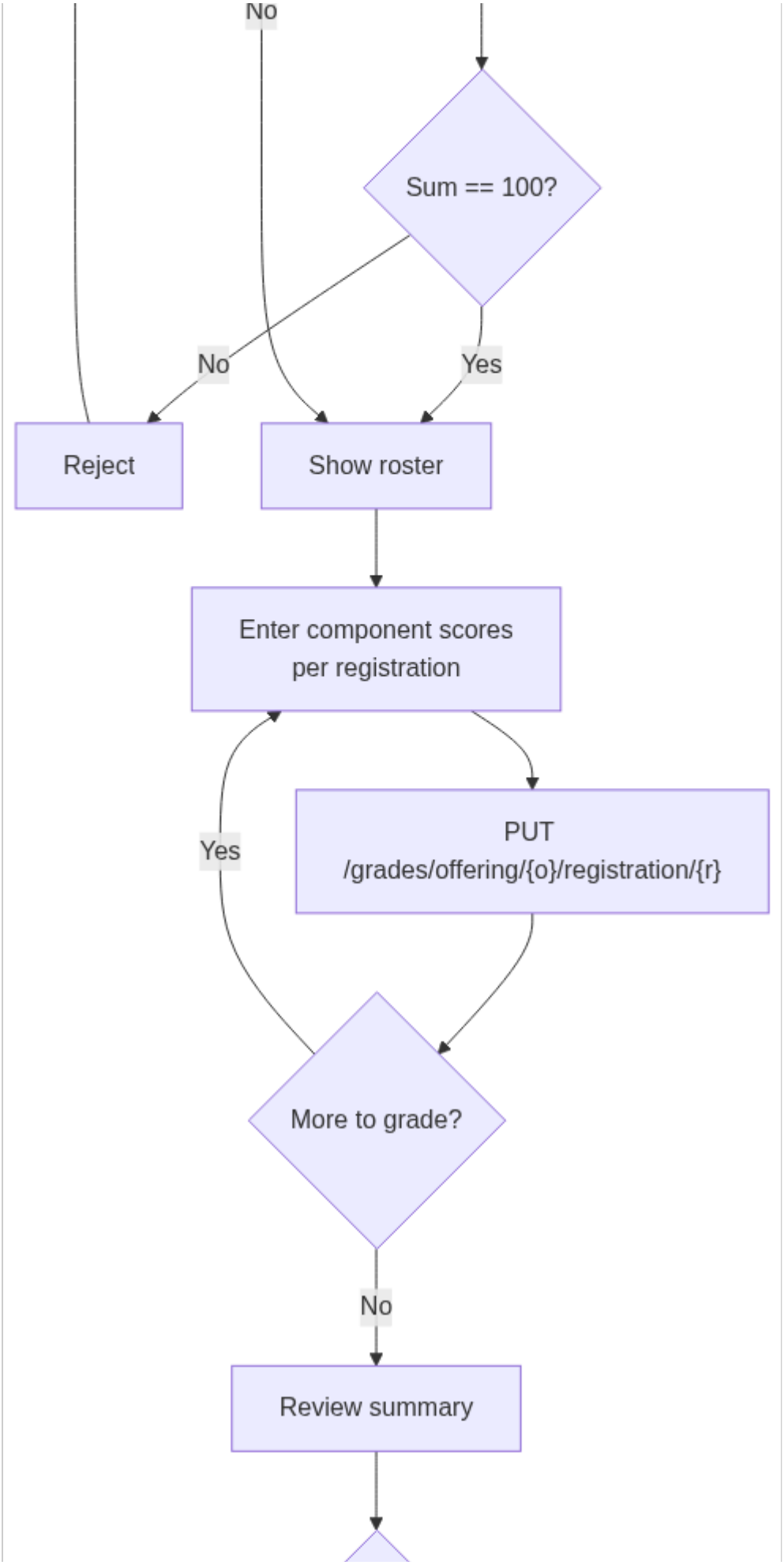
activity-diagrams-02

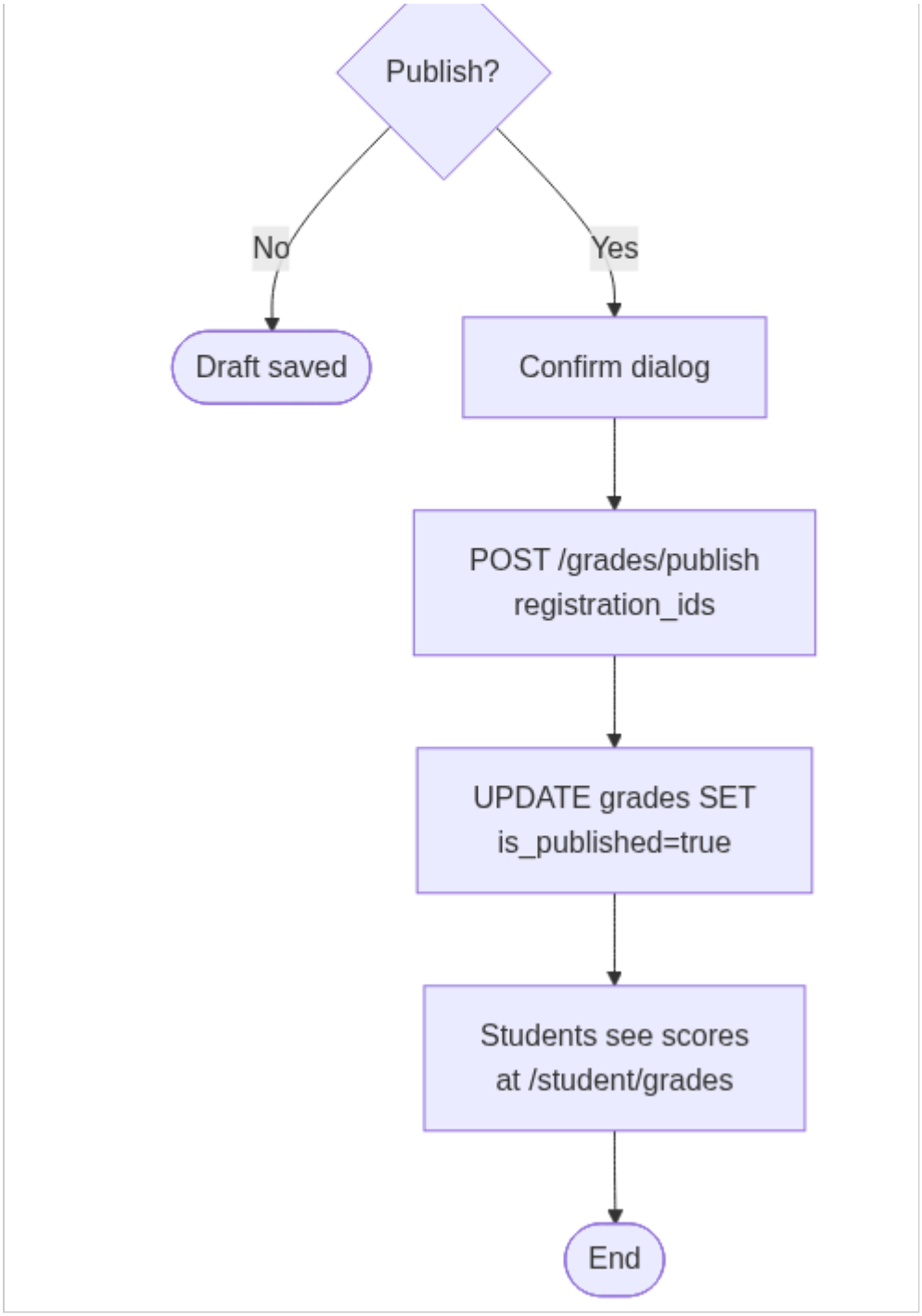

Open /instructor/attendance
Pick course offering
List sessions
Pick session
Session date == today?
No
Yes
Show date-lock warning
Load roster
Create fresh today-session?
Mark each student Present/Absent/Late
No
Yes
End
POST /attendance/offering/{id}/sessions session_date=today
POST /attendance/sessions/{sid}/records
Backend recomputes absence percentage



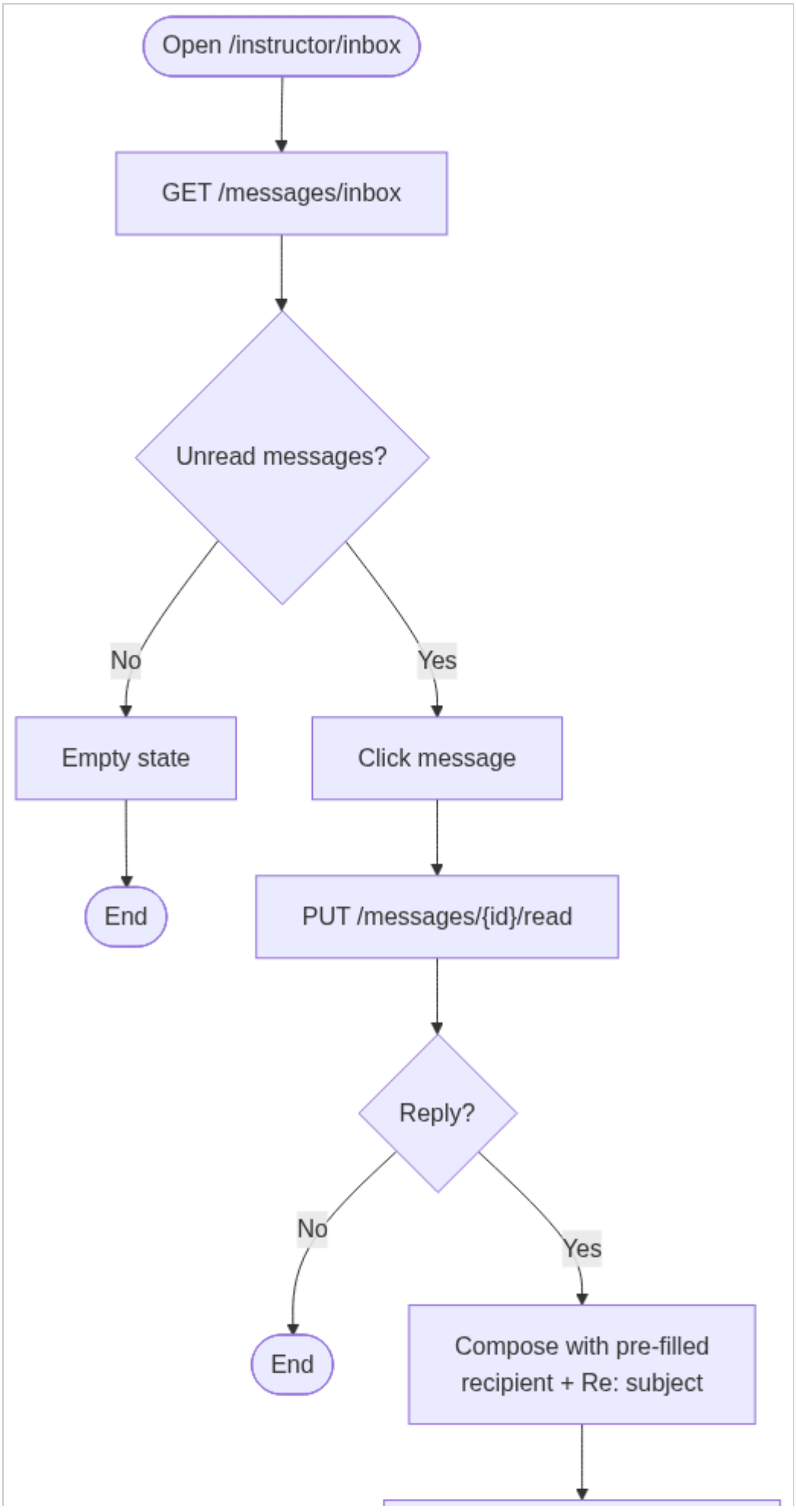
activity-diagrams-03

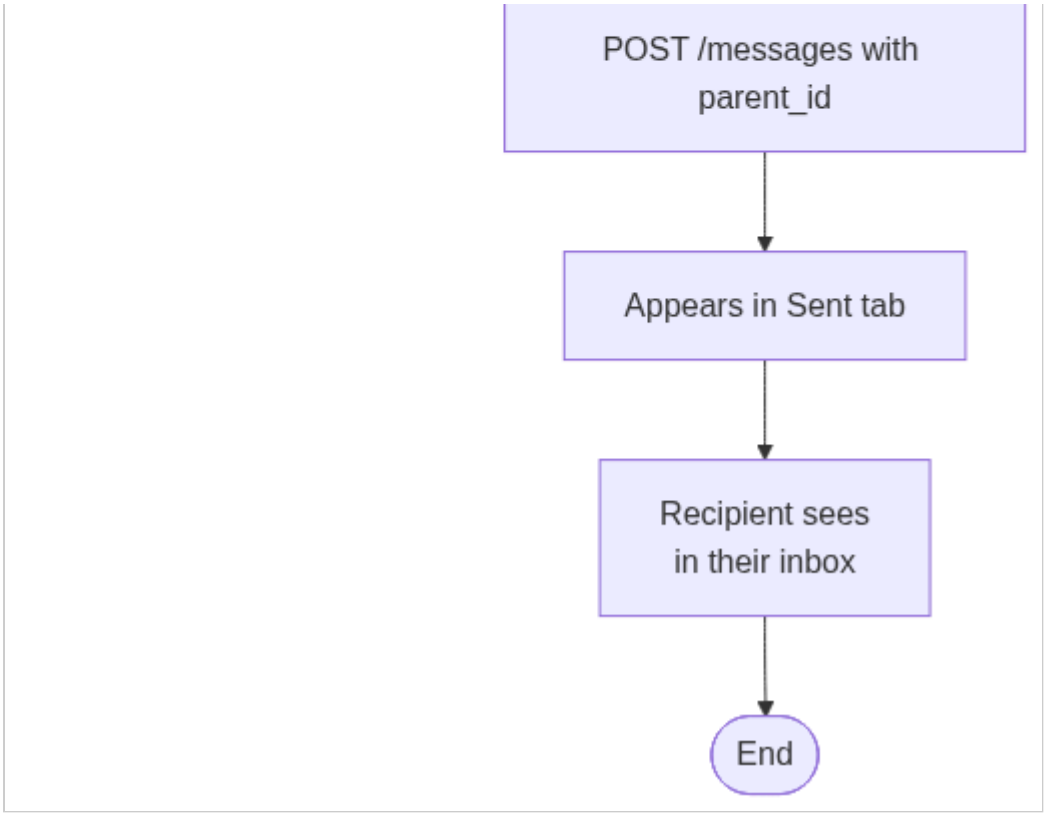

Open /instructor/grades
Pick offering
Grade config exists?
No
Yes
Auto-create default config
Show current weights
Edit weights?
Yes
PUT /grades/config


No
Sum == 100?
No
Yes
Reject
Show roster
Enter component scores per registration
PUT /grades/offering/{o}/registration/{r}
Yes
More to grade?
No
Review summary

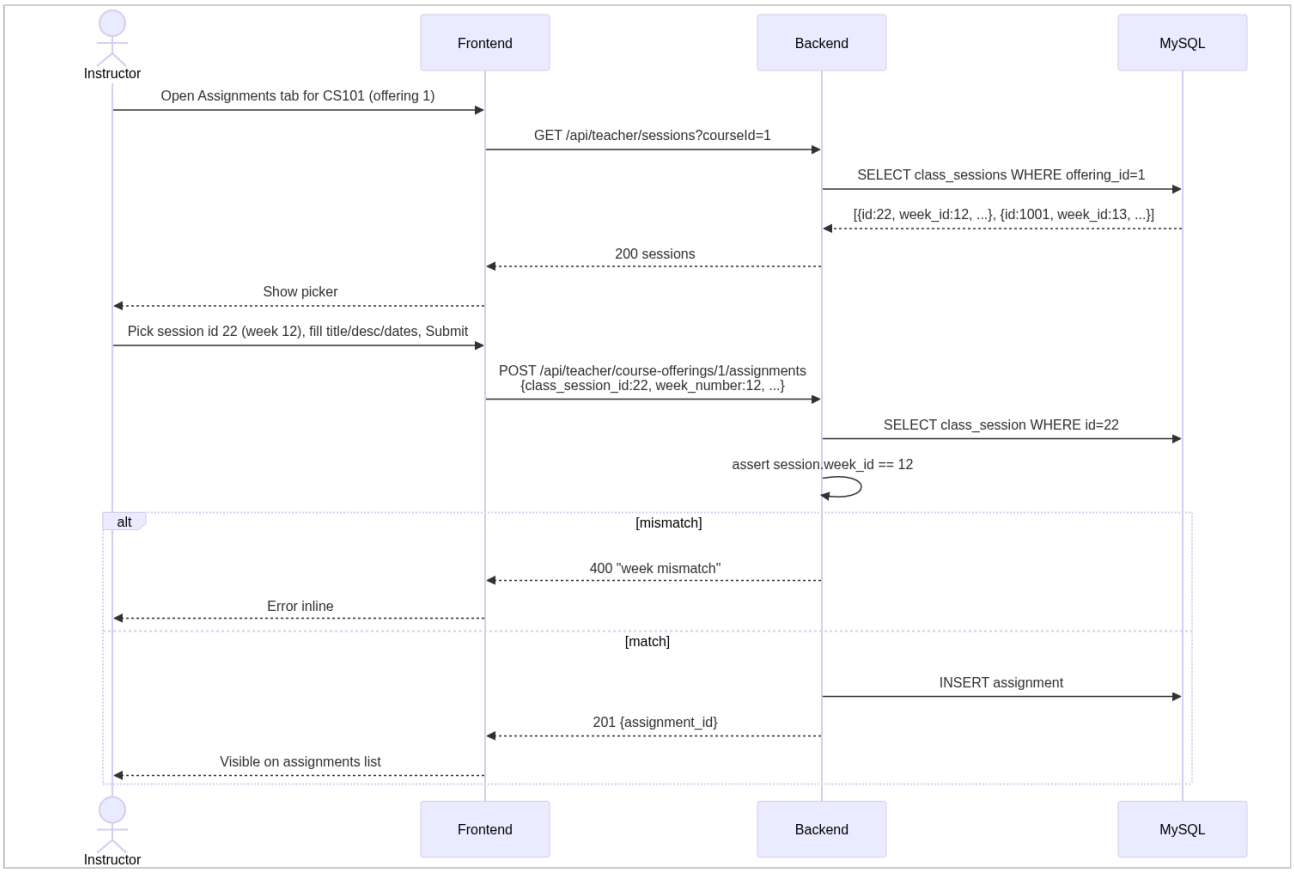


activity-diagrams-04

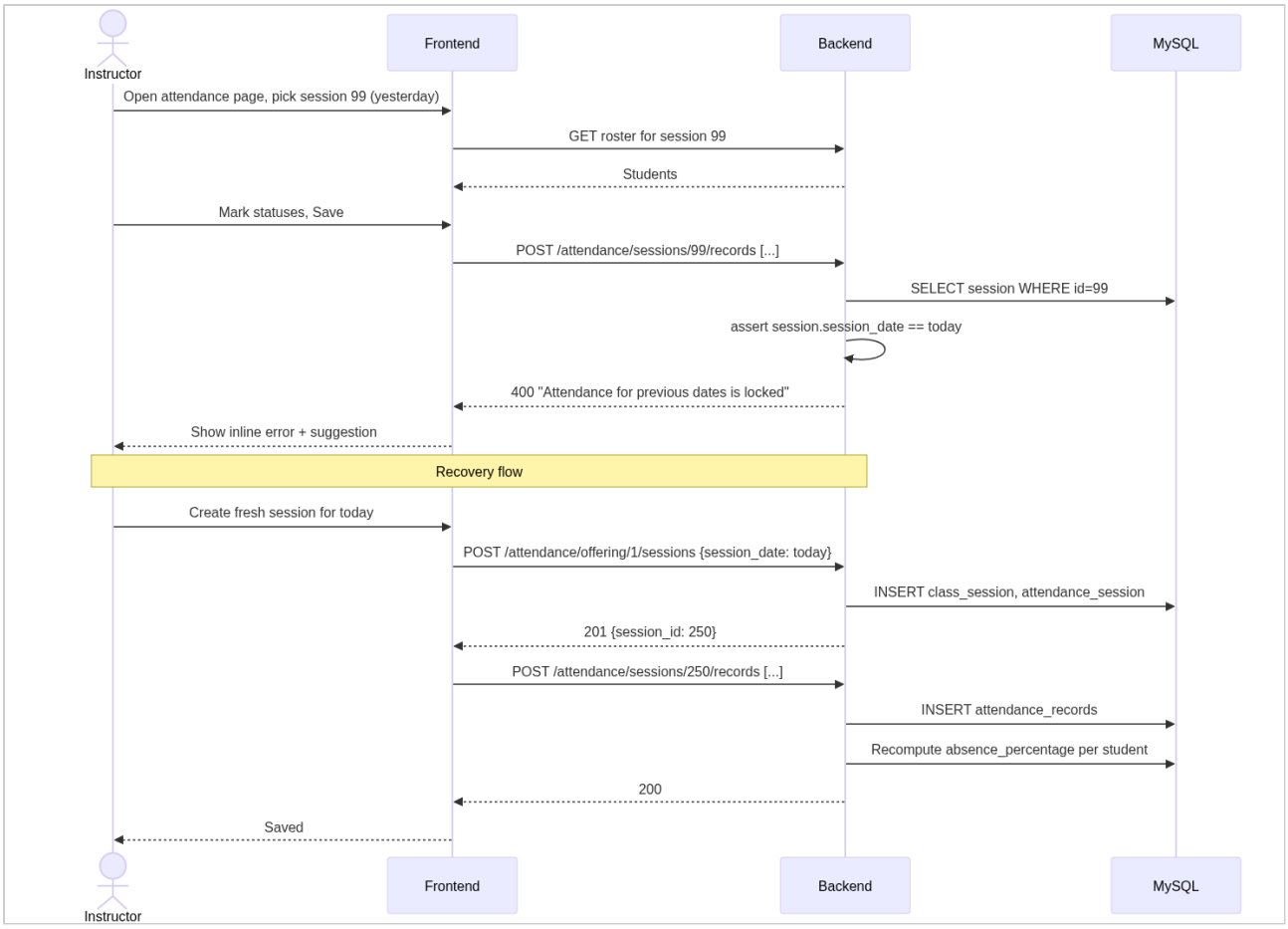

Open /instructor/inbox
GET /messages/inbox
Unread messages?
No
Yes
Empty state
Click message
End
PUT /messages/{id}/read
Reply?
No
Yes
End
Compose with pre-filled recipient + Re: subject



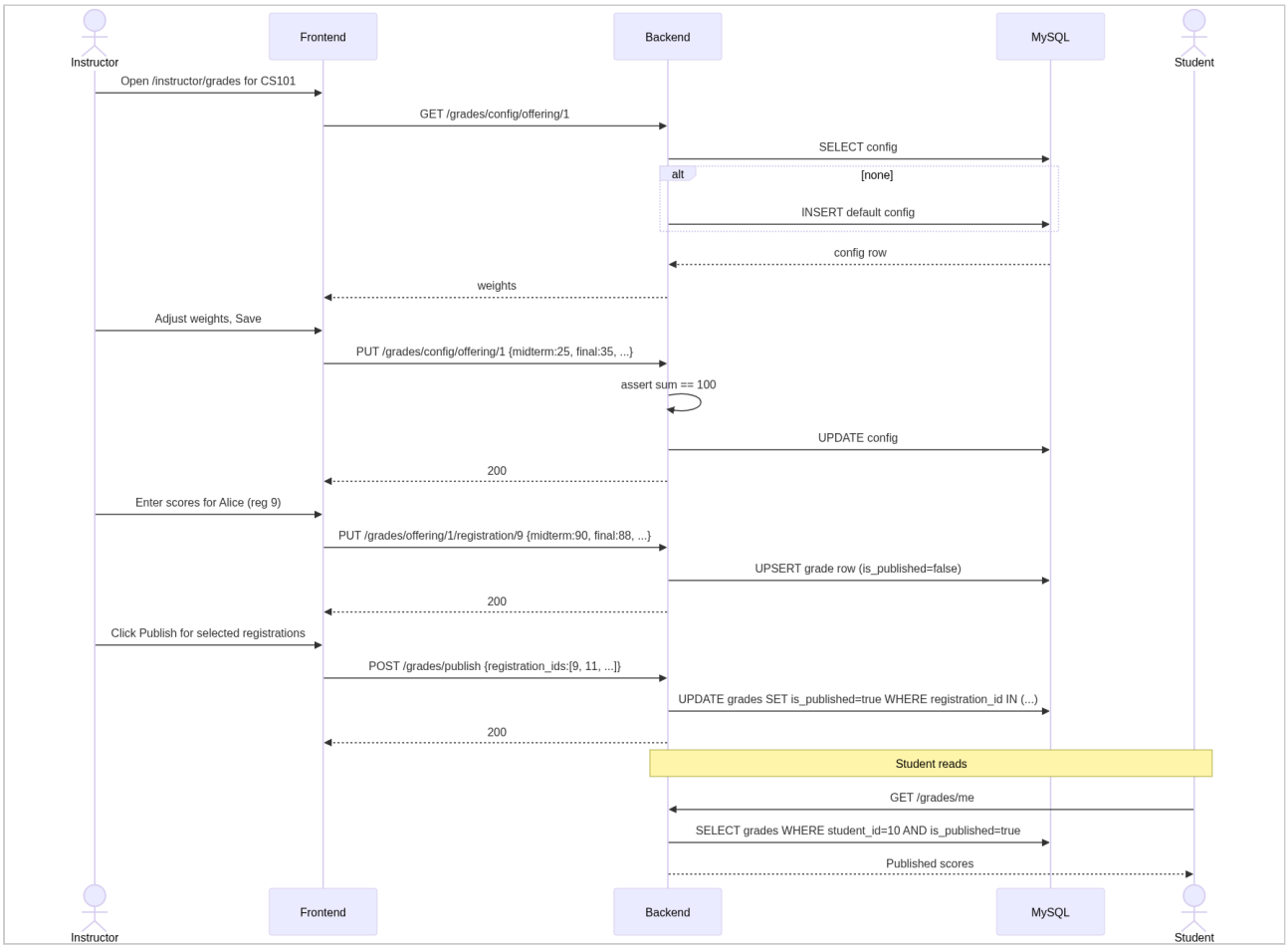
sequence-diagrams-01



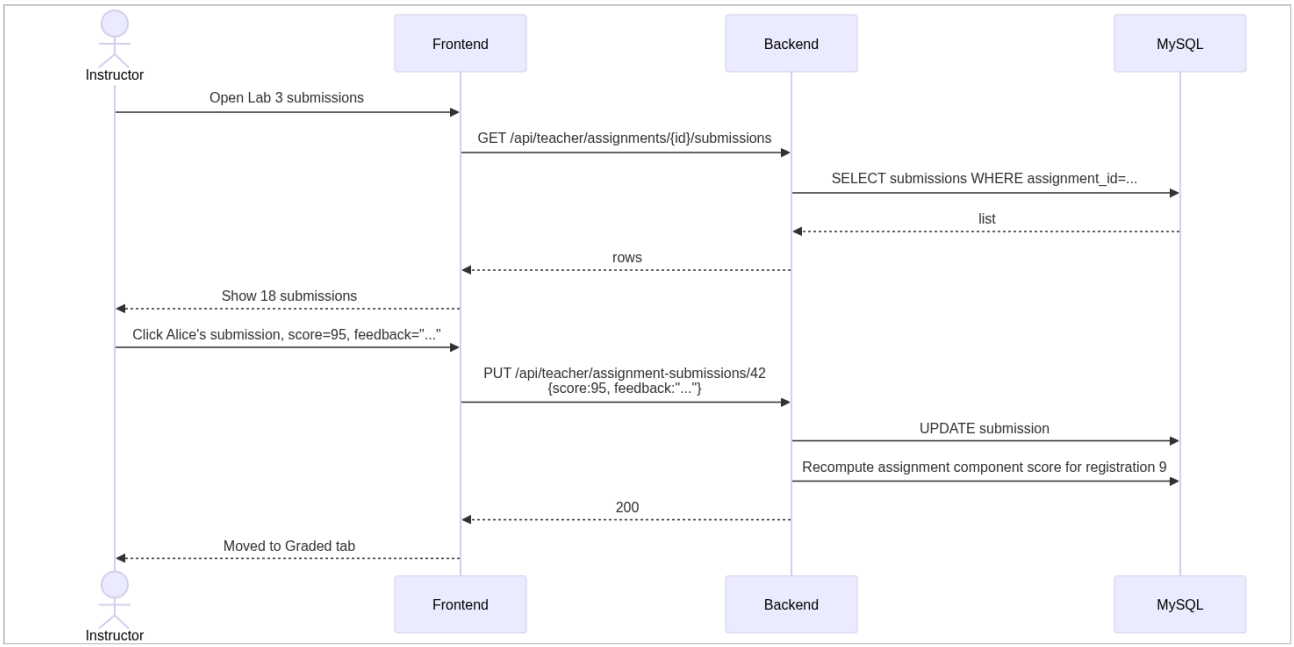
sequence-diagrams-02



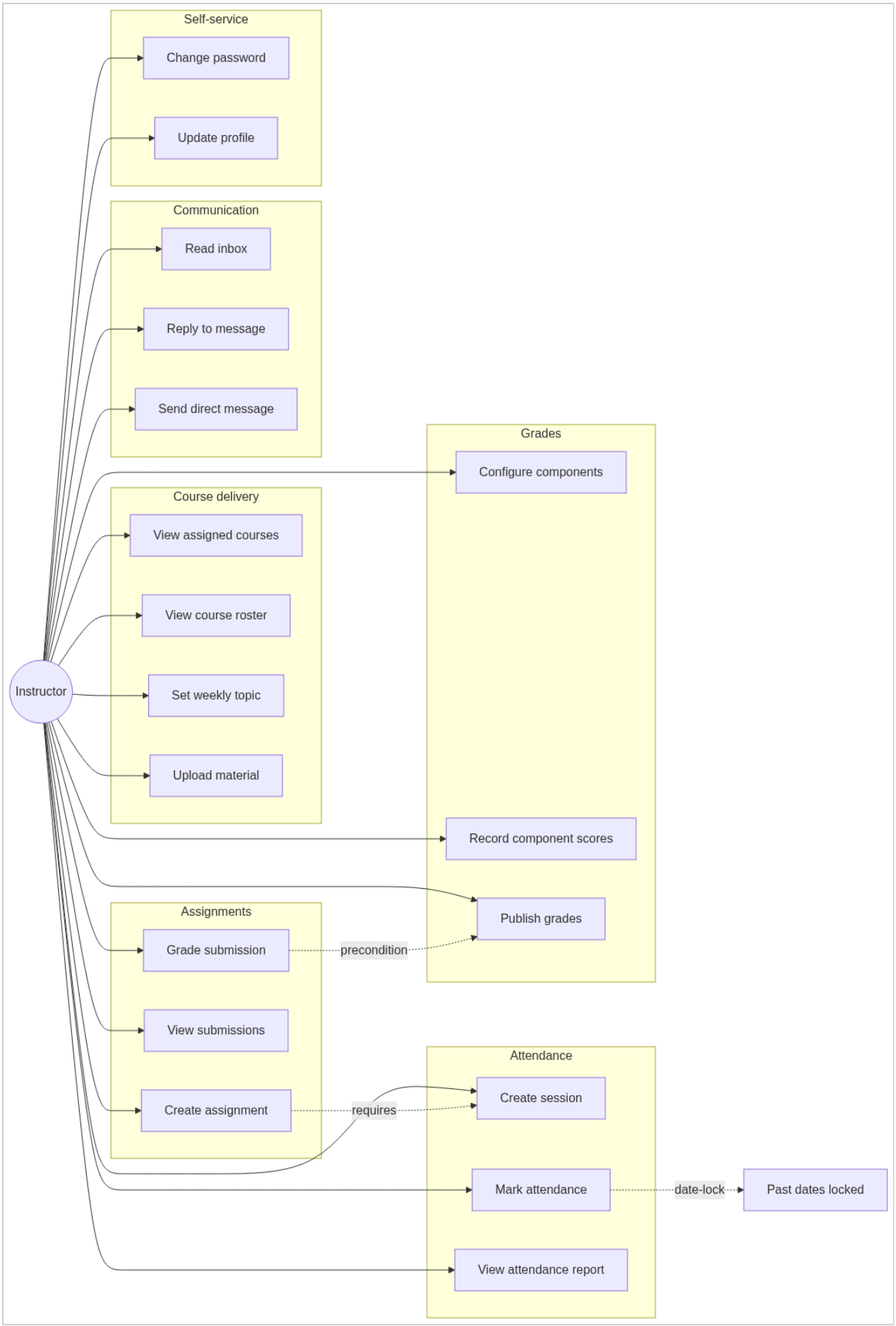
sequence-diagrams-03



sequence-diagrams-04

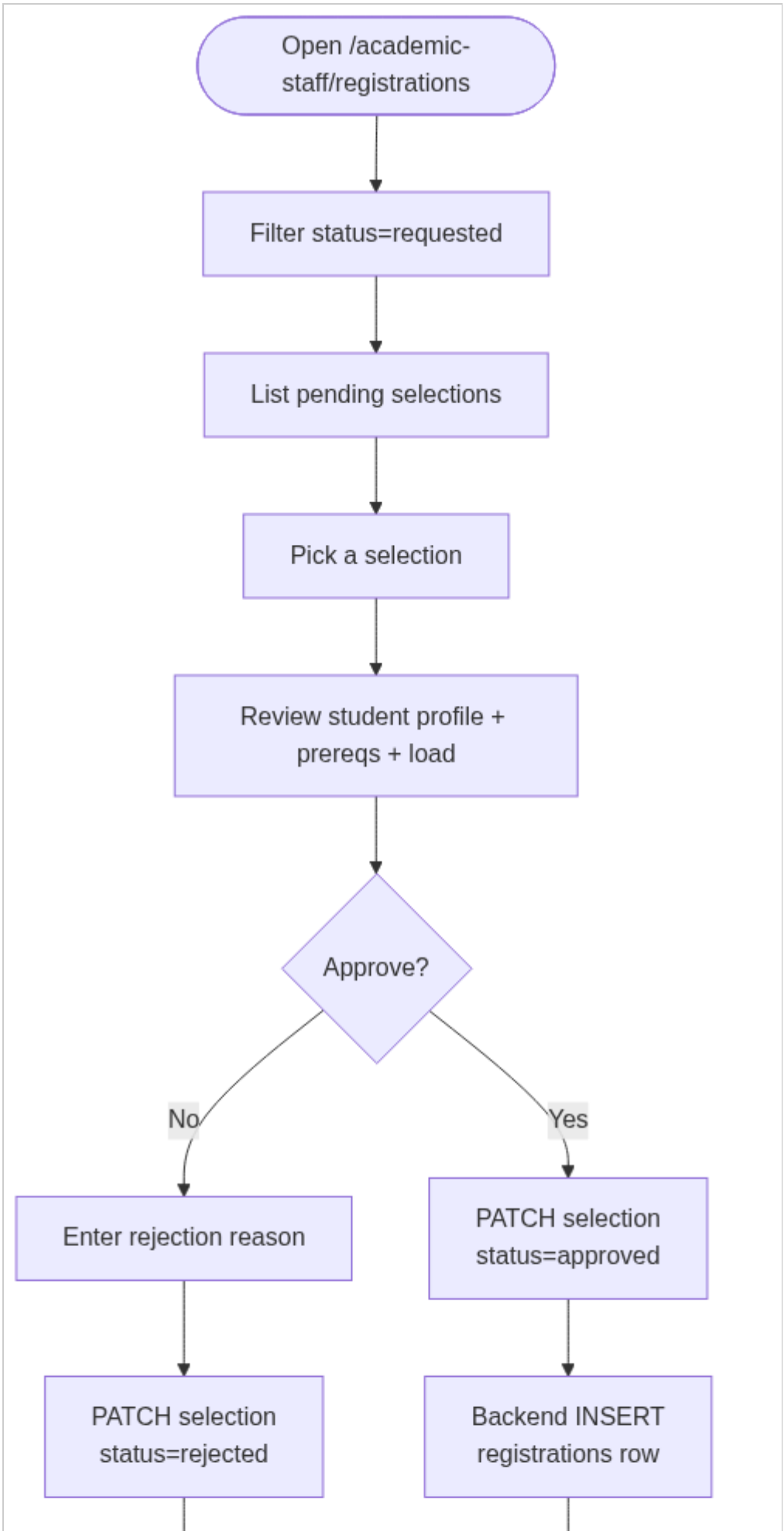


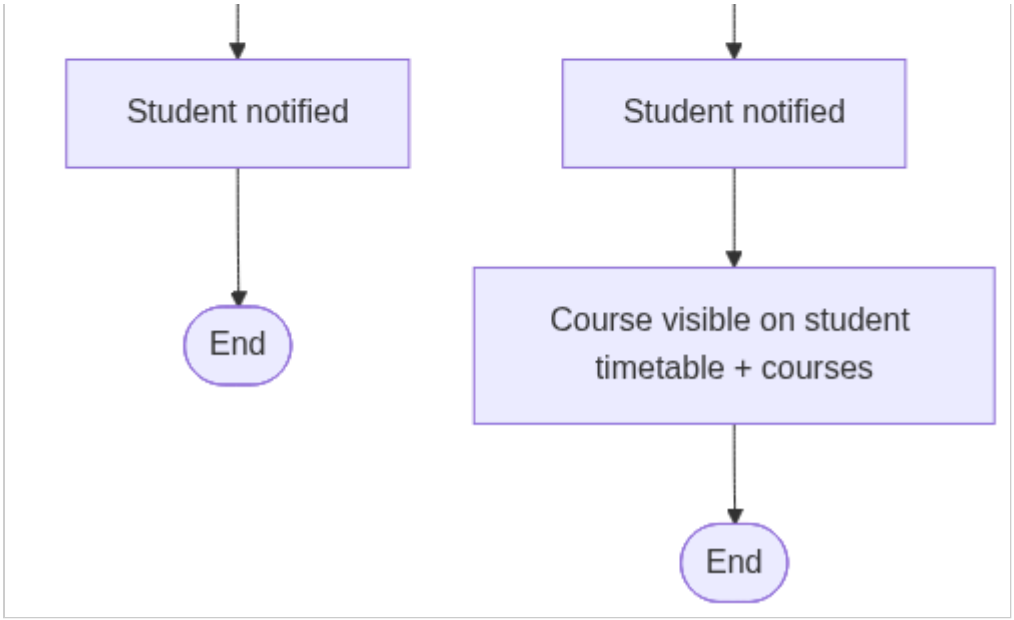
use-cases-01


Self-service
Change password
Update profile
Communication
Read inbox
Reply to message
Send direct message
Grades
Configure components
Course delivery
View assigned courses
View course roster
Instructor
Set weekly topic
Upload material
Record component scores
Publish grades
Assignments
Grade submission
precondition
View submissions
Attendance
Create assignment
requires
Create session
Mark attendance
date-lock
Past dates locked
View attendance report

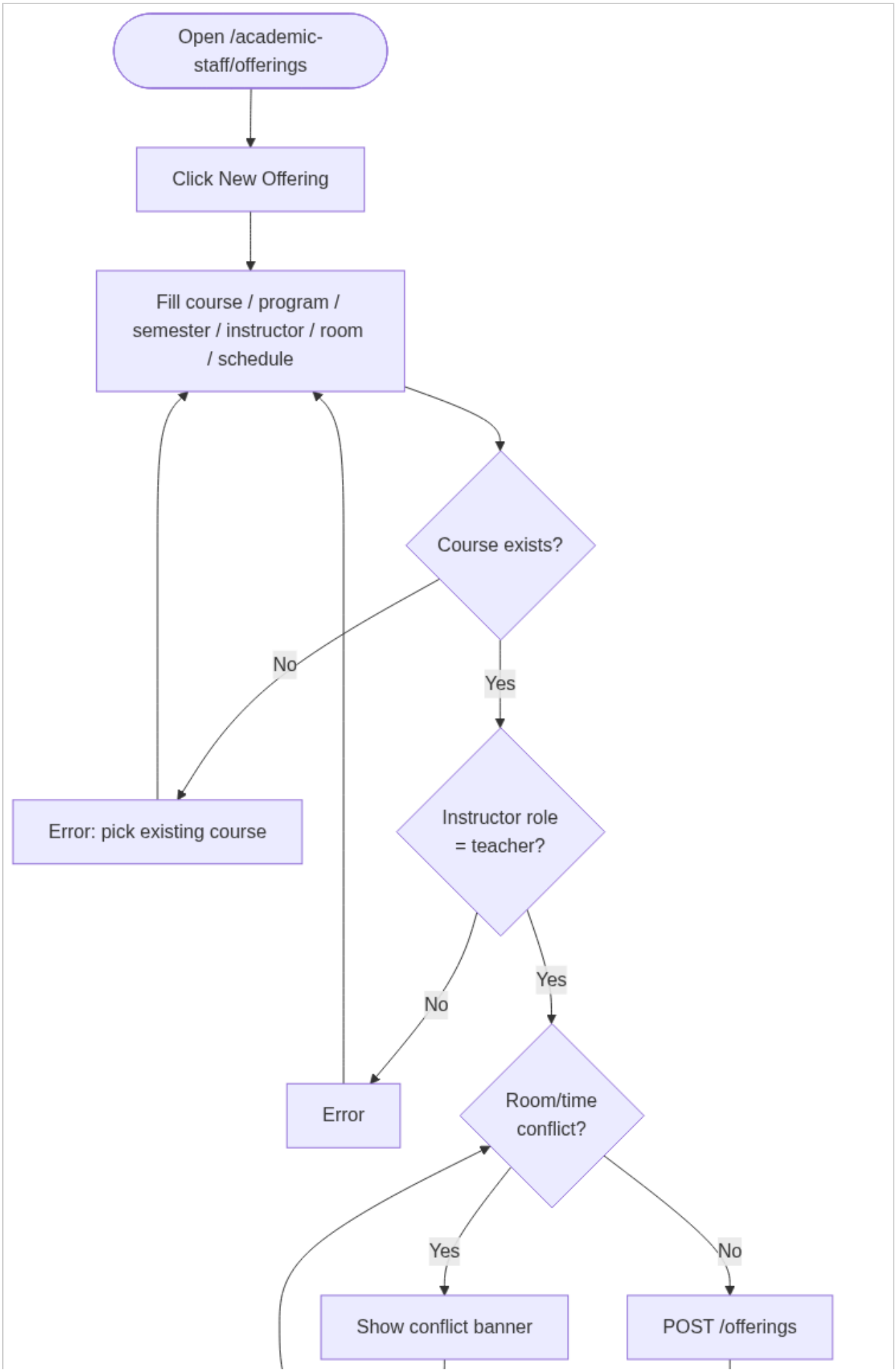
Academic Staff

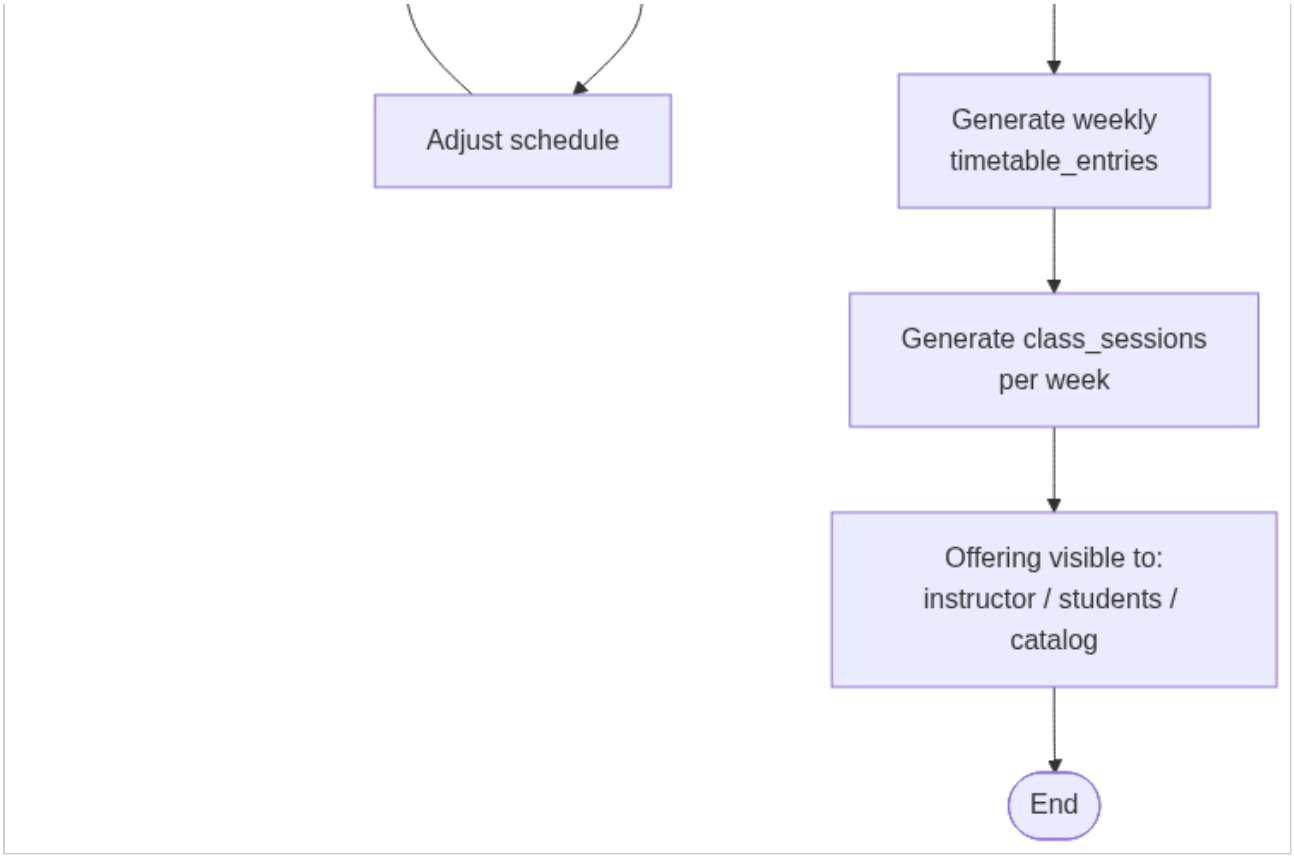
activity-diagrams-01


Open /academic-staff/registrations
Filter status=requested
List pending selections
Pick a selection
Review student profile + prereqs + load
Approve?
No
Yes
Enter rejection reason
PATCH selection status=approved
PATCH selection status=rejected
Backend INSERT registrations row

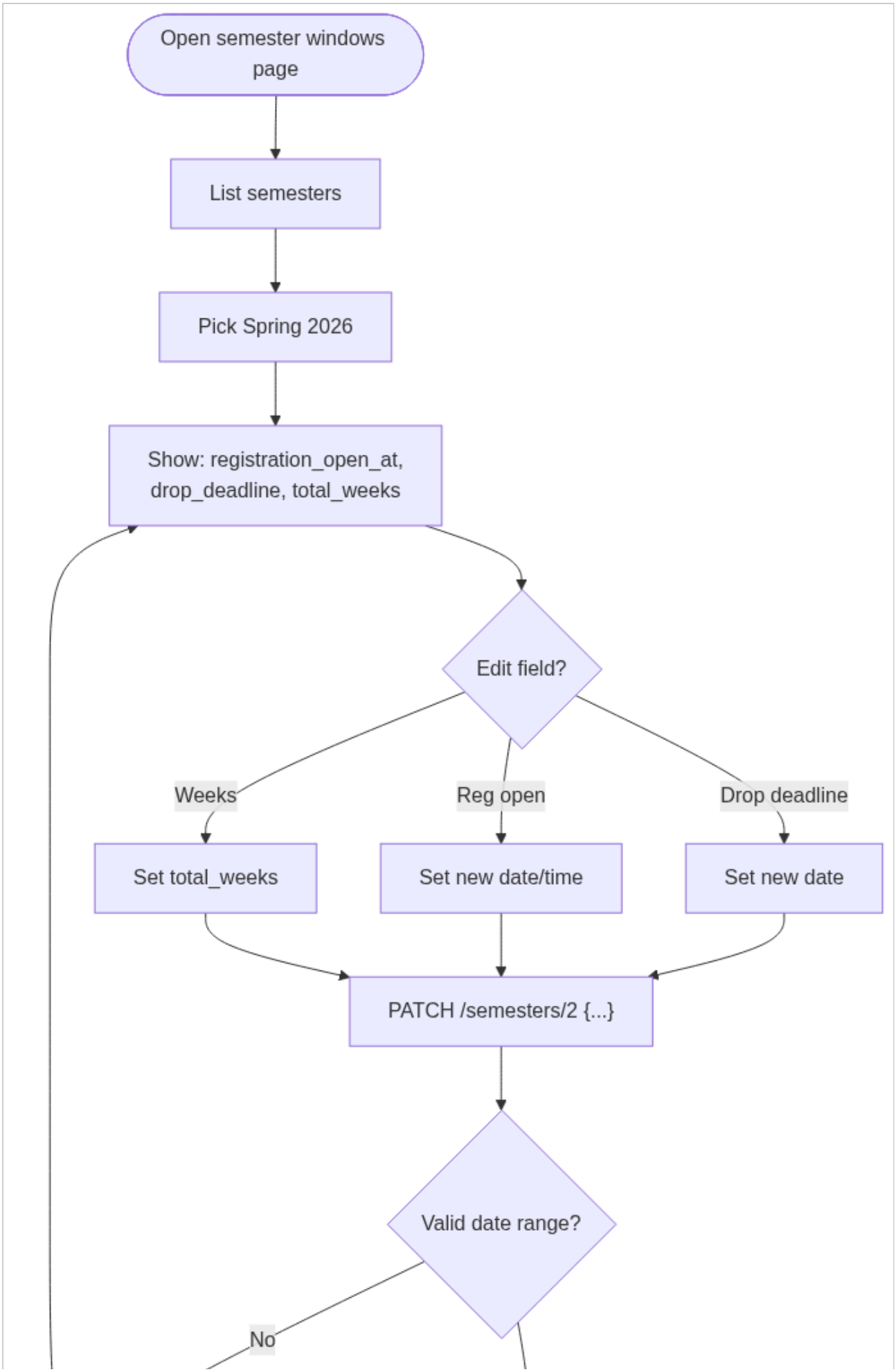


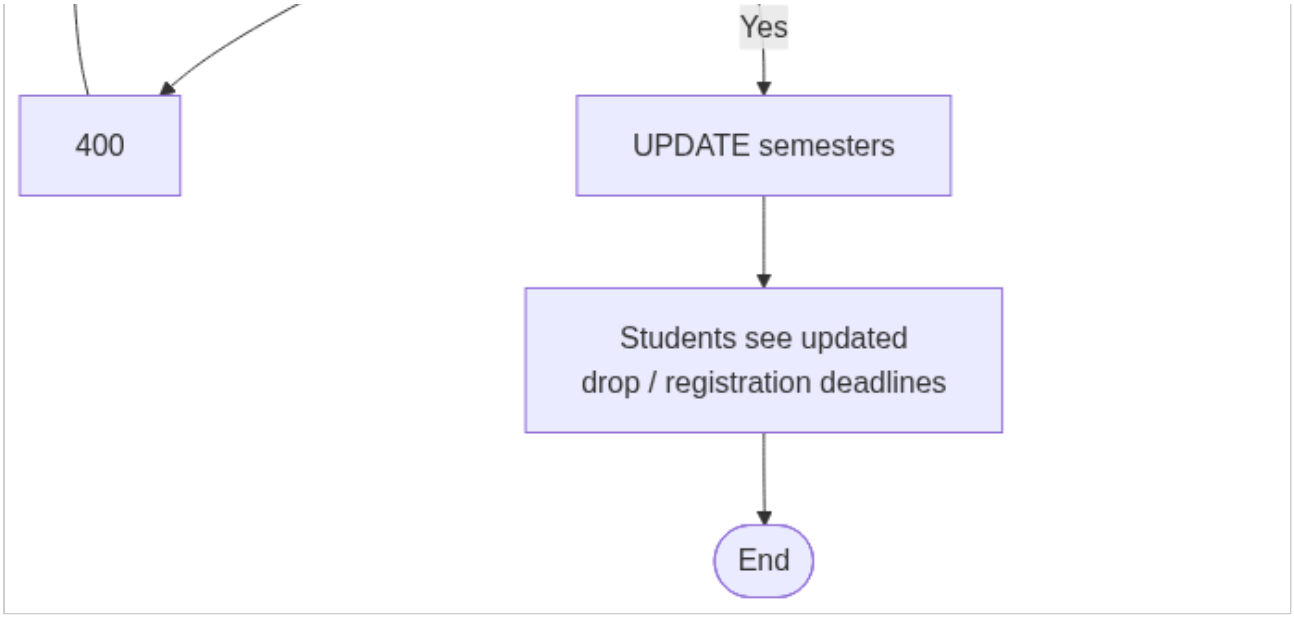
activity-diagrams-02


Open /academic-staff/offerings
Click New Offering
Fill course / program / semester / instructor / room / schedule
Course exists?
No
Yes
Error: pick existing course
Instructor role = teacher?
No
Yes
Error
Room/time conflict?
Yes
No
Show conflict banner
POST /offerings

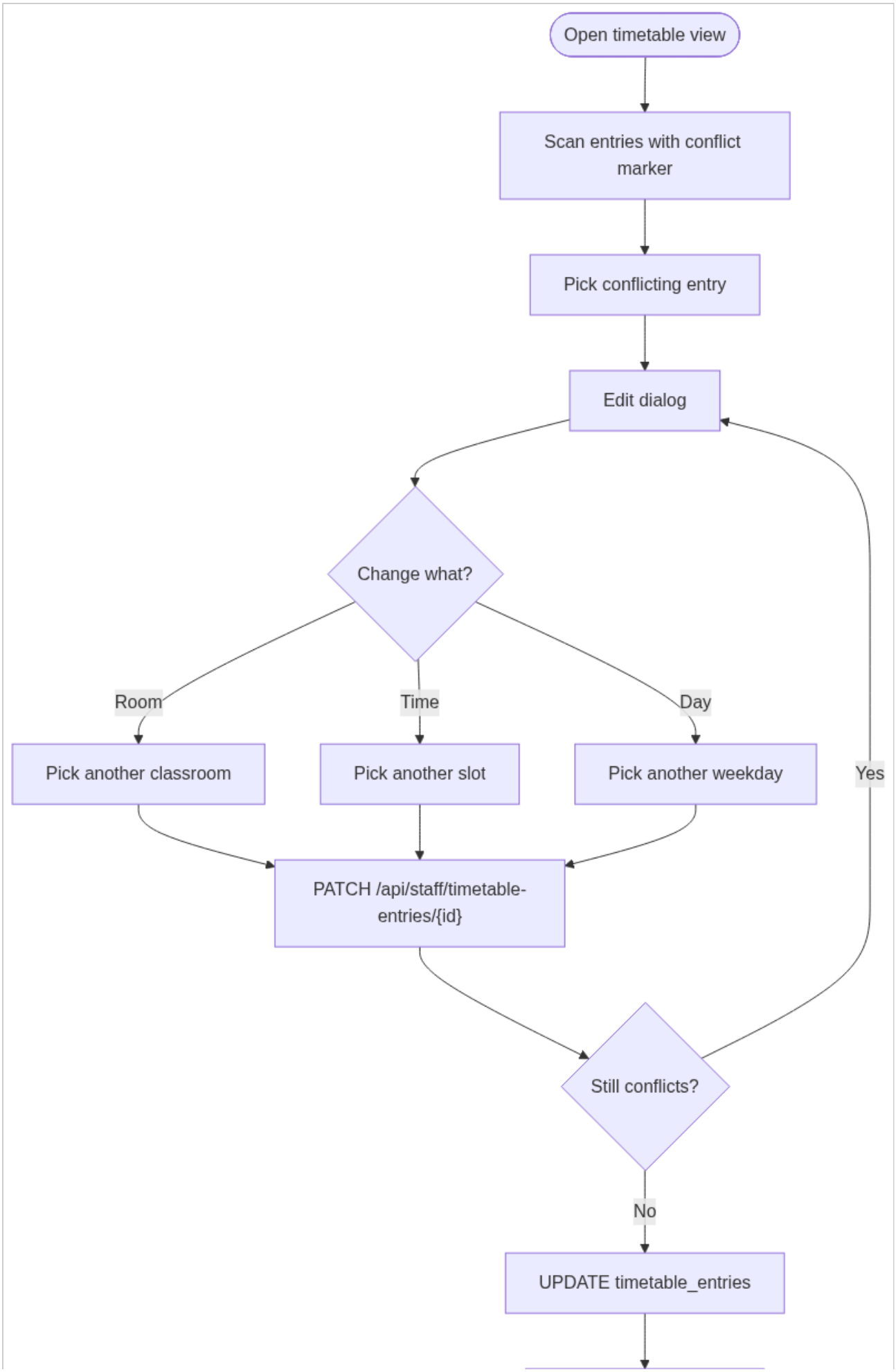


activity-diagrams-03

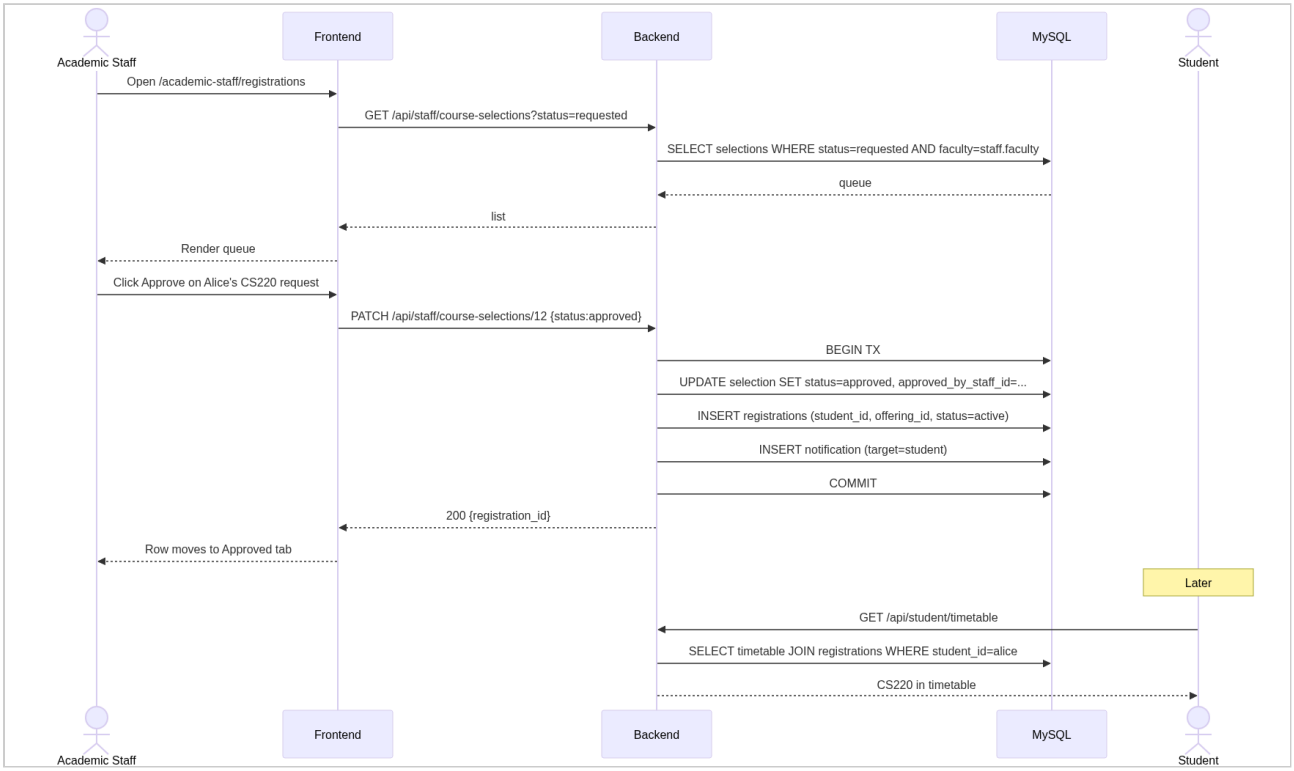

Open semester windows page
List semesters
Pick Spring 2026
Show: registration_open_at, drop_deadline, total_weeks
Edit field?
Weeks
Reg open
Drop deadline
Set total_weeks
Set new date/time
Set new date
PATCH /semesters/2 {...}
Valid date range?
No



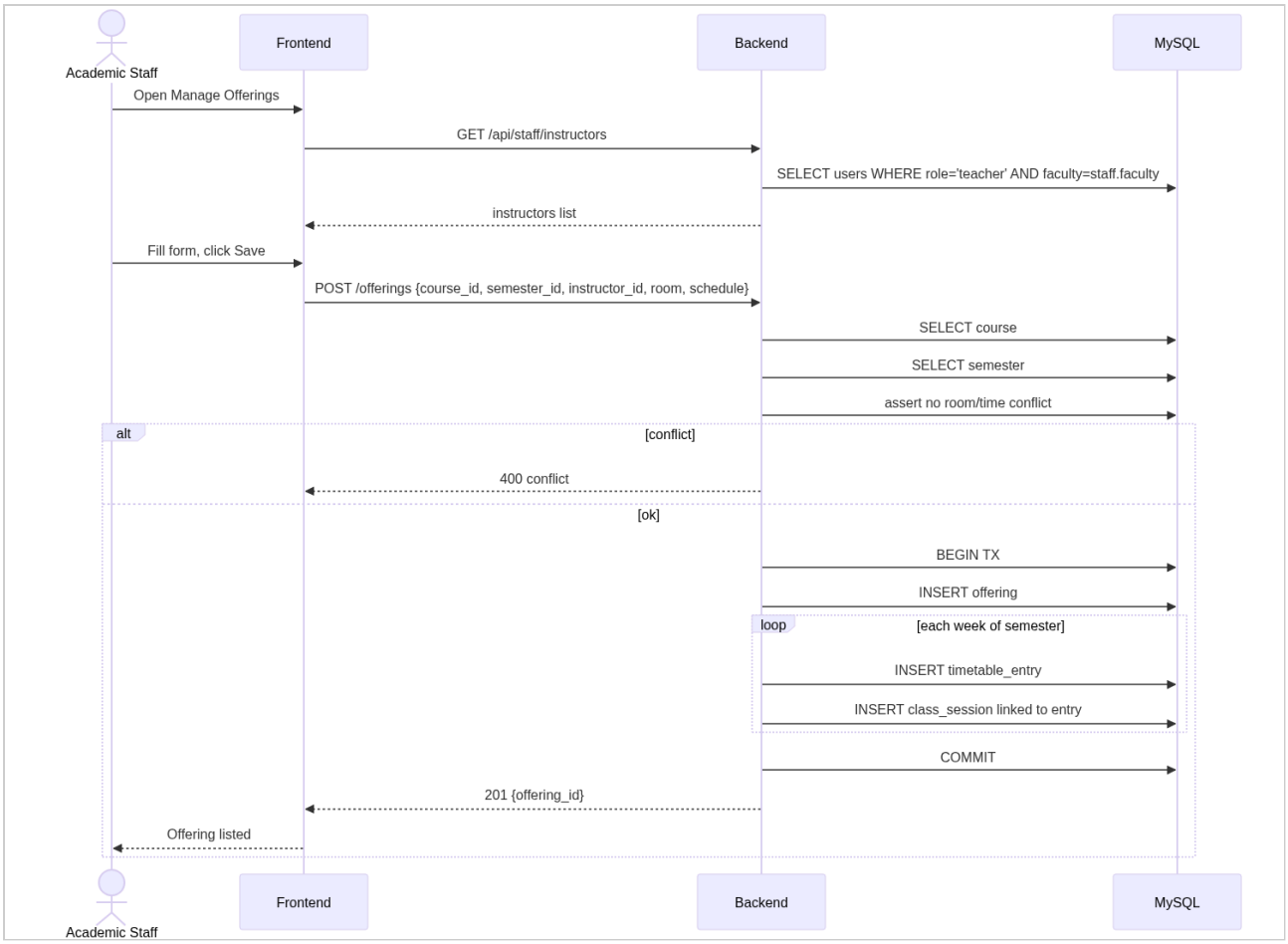
activity-diagrams-04


Open timetable view
Scan entries with conflict marker
Pick conflicting entry
Edit dialog
Change what?
Room
Time
Day
Pick another classroom
Pick another slot
Pick another weekday
PATCH /api/staff/timetable-entries/{id}
Still conflicts?
Yes
No
UPDATE timetable_entries

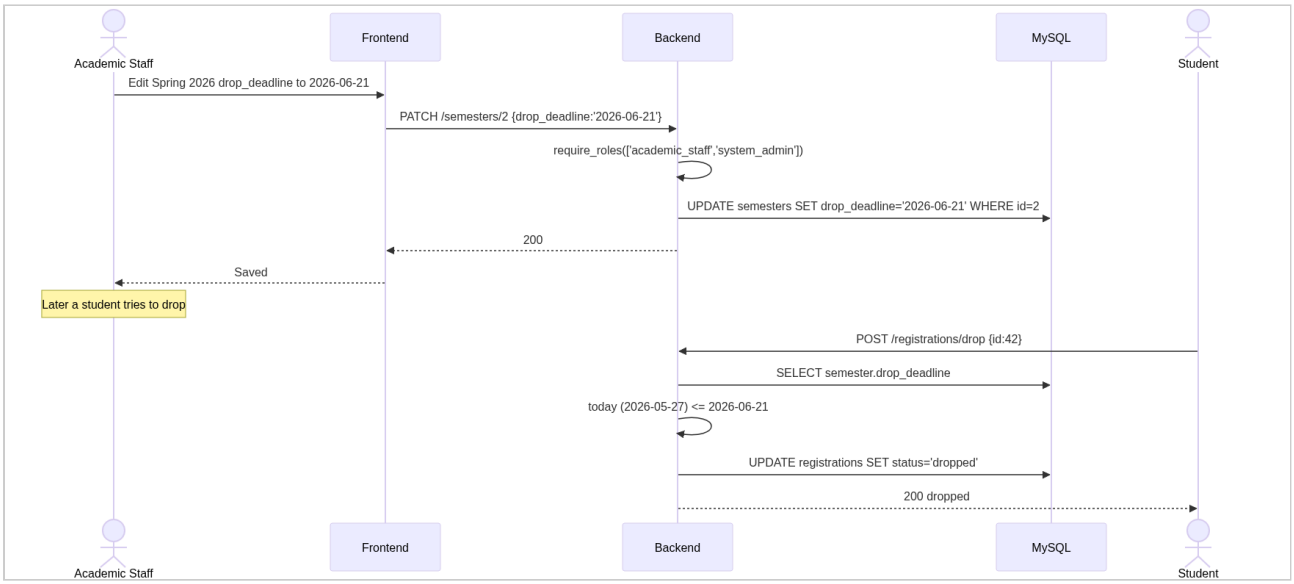
sequence-diagrams-01



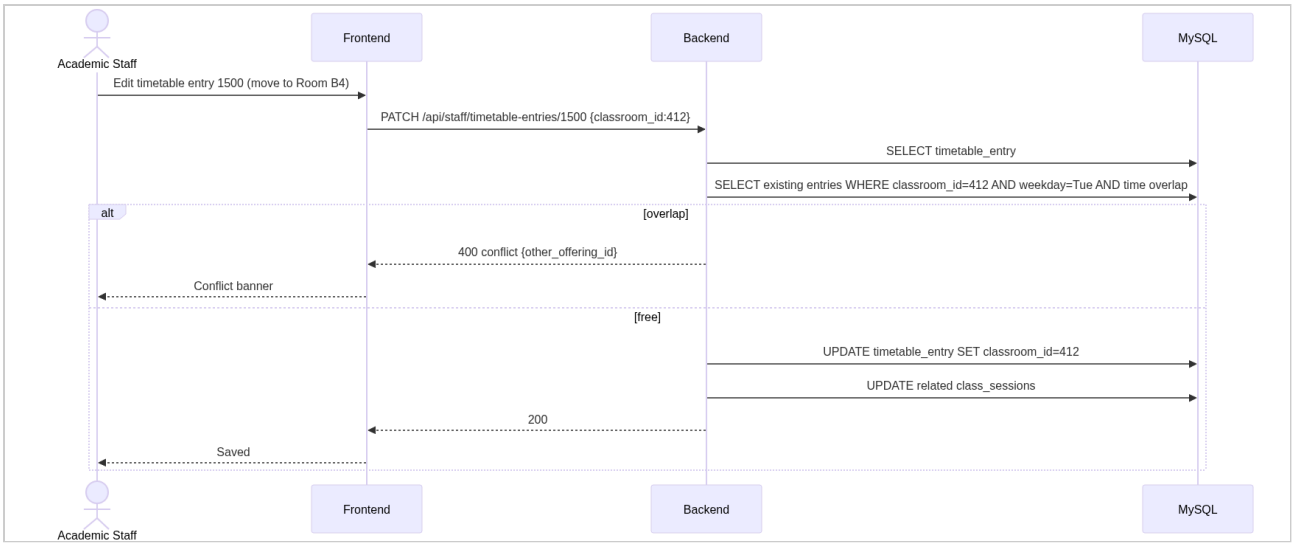
sequence-diagrams-02



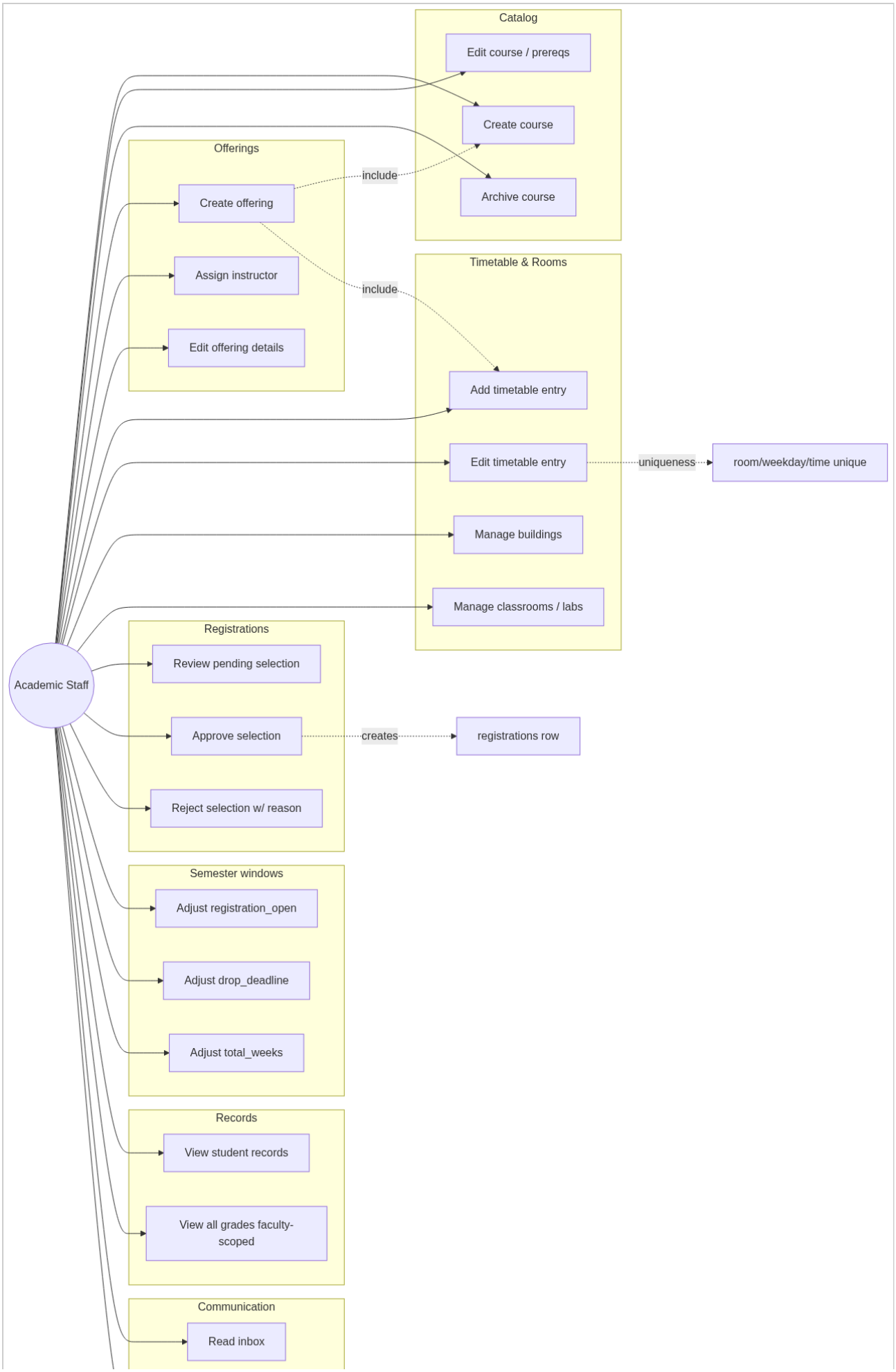
sequence-diagrams-03

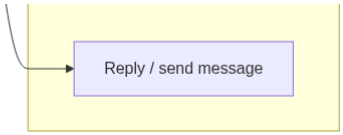


sequence-diagrams-04


Academic Staff
Frontend
Backend
MySQL
Edit timetable entry 1500 (move to Room B4)
PATCH /api/staff/timetable-entries/1500 {classroom_id:412}
SELECT timetable_entry
SELECT existing entries WHERE classroom_id=412 AND weekday=Tue AND time overlap
alt
[overlap]
400 conflict {other_offering_id}
Conflict banner
[free]
UPDATE timetable_entry SET classroom_id=412
UPDATE related class_sessions
200
Saved
Academic Staff
Frontend
Backend
MySQL

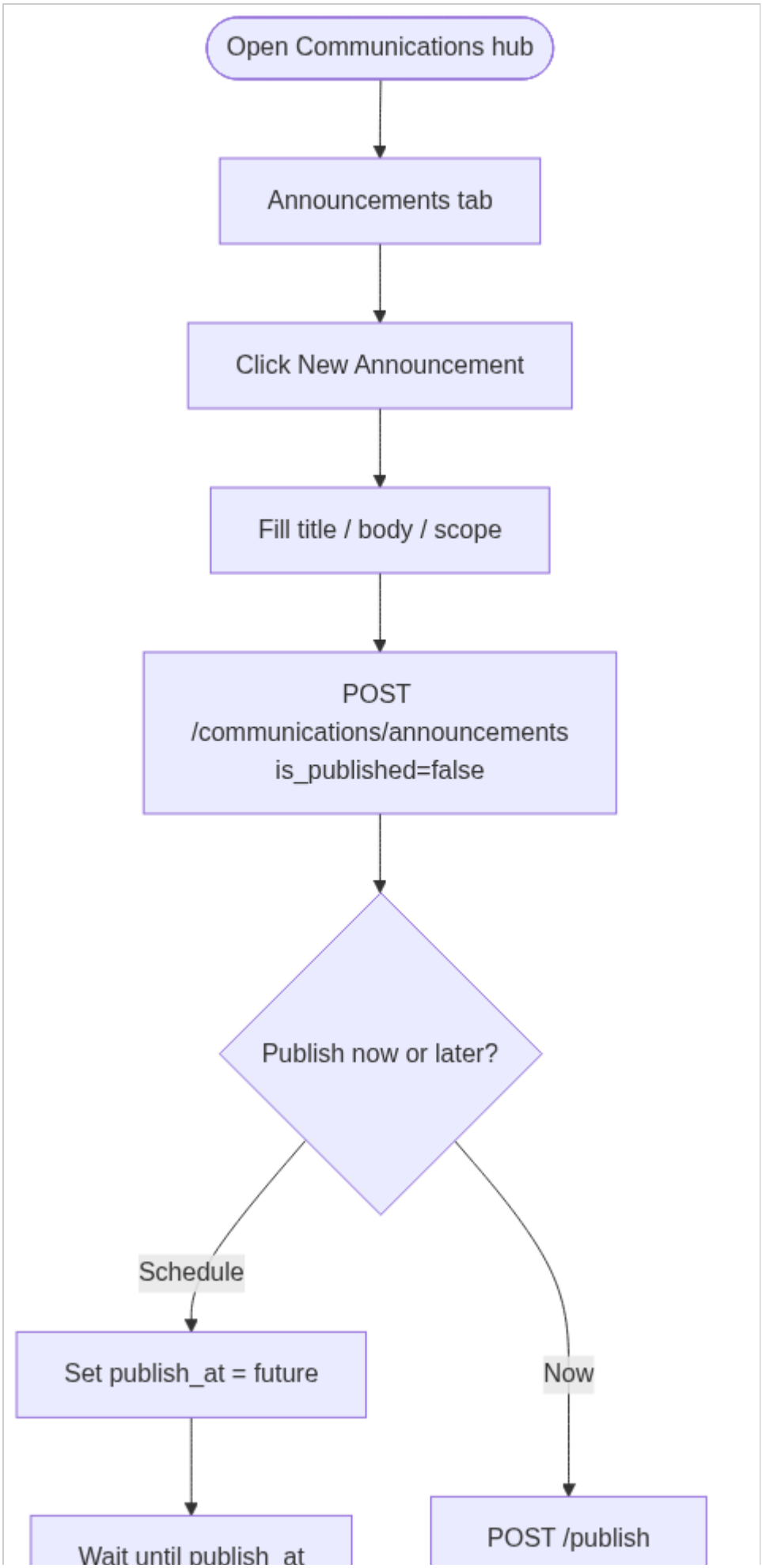
use-cases-01

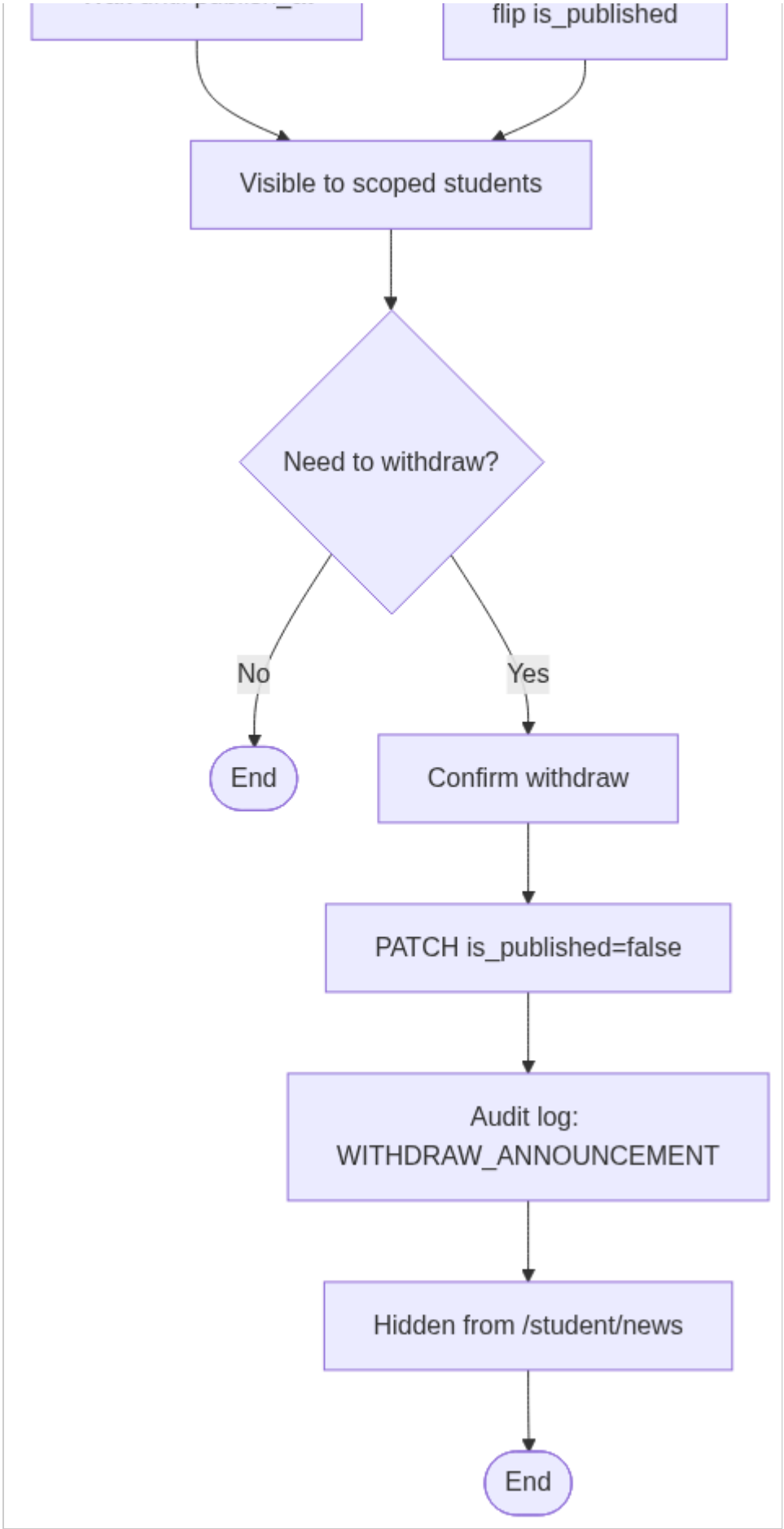

Catalog
Edit course / prereqs
Create course
Archive course
Offerings
include
Create offering
Assign instructor
Edit offering details
include
Timetable & Rooms
Add timetable entry
Edit timetable entry
uniqueness
room/weekday/time unique
Manage buildings
Manage classrooms / labs
Academic Staff
Registrations
Review pending selection
Approve selection
creates
registrations row
Reject selection w/ reason
Semester windows
Adjust registration_open
Adjust drop_deadline
Adjust total_weeks
Records
View student records
View all grades faculty-scoped
Communication
Read inbox



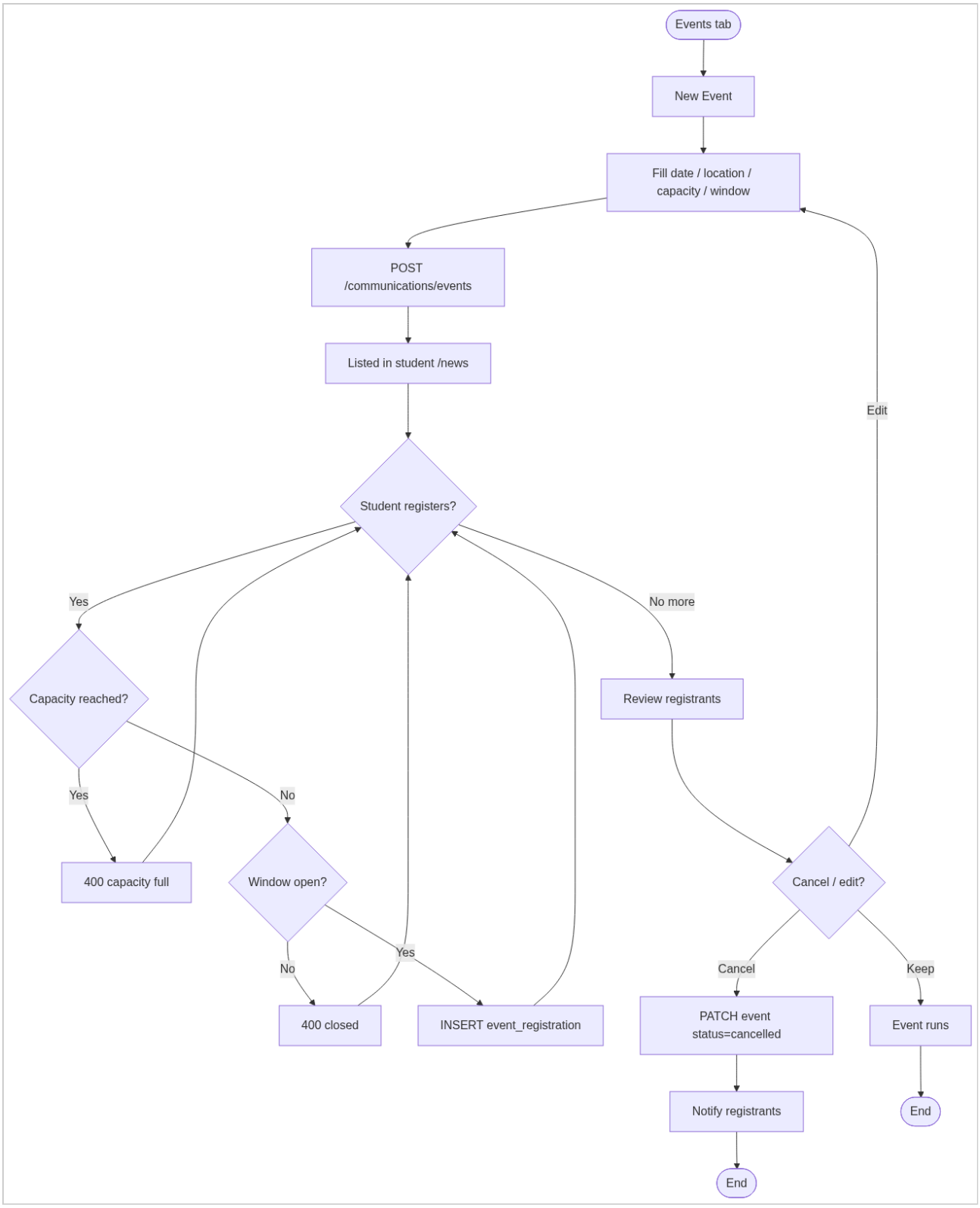
Communications Staff

activity-diagrams-01

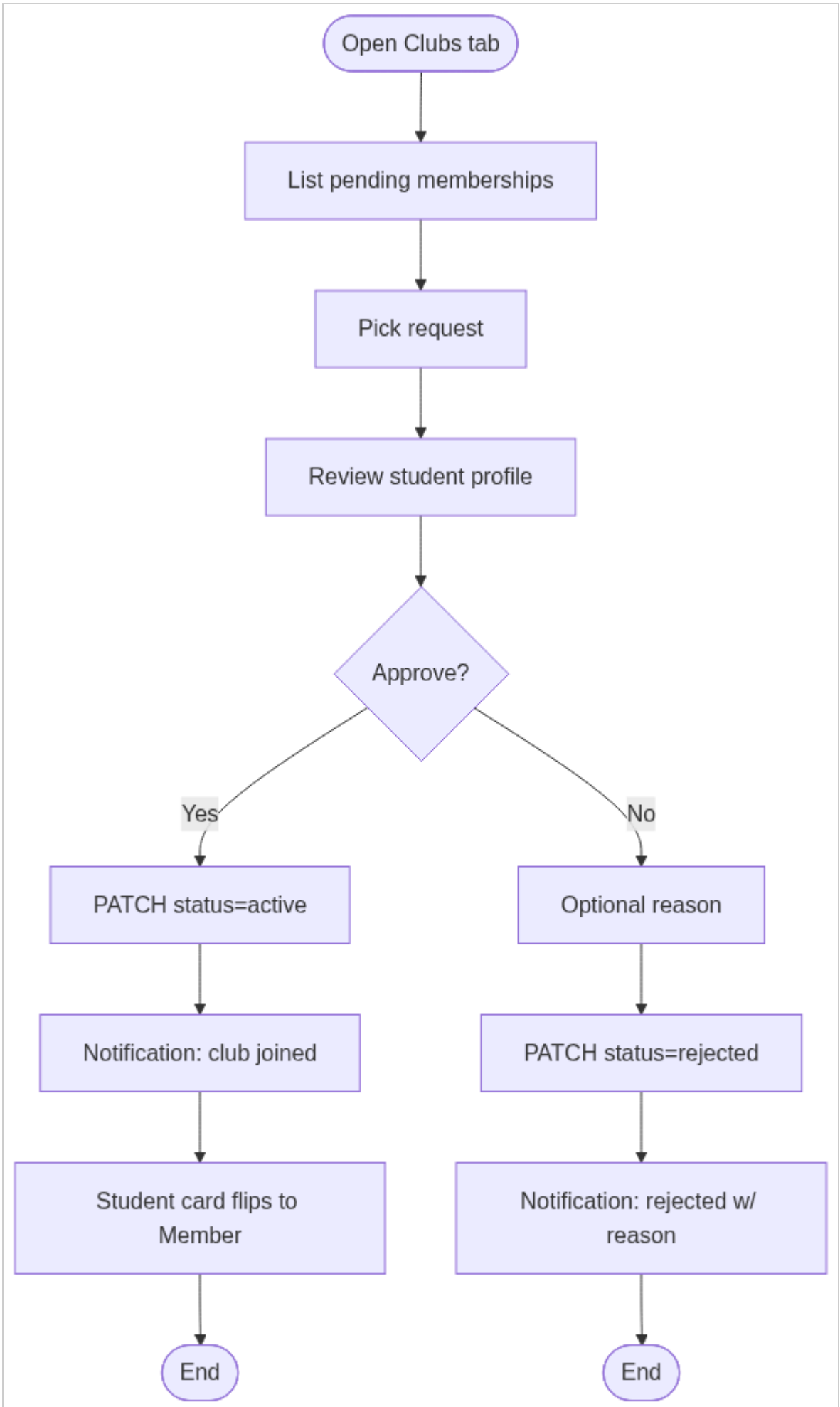

Open Communications hub
Announcements tab
Click New Announcement
Fill title / body / scope
POST
/communications/announcements
is_published=false
Publish now or later?
Schedule
Set publish_at = future
Wait until publish_at
Now
POST /publish



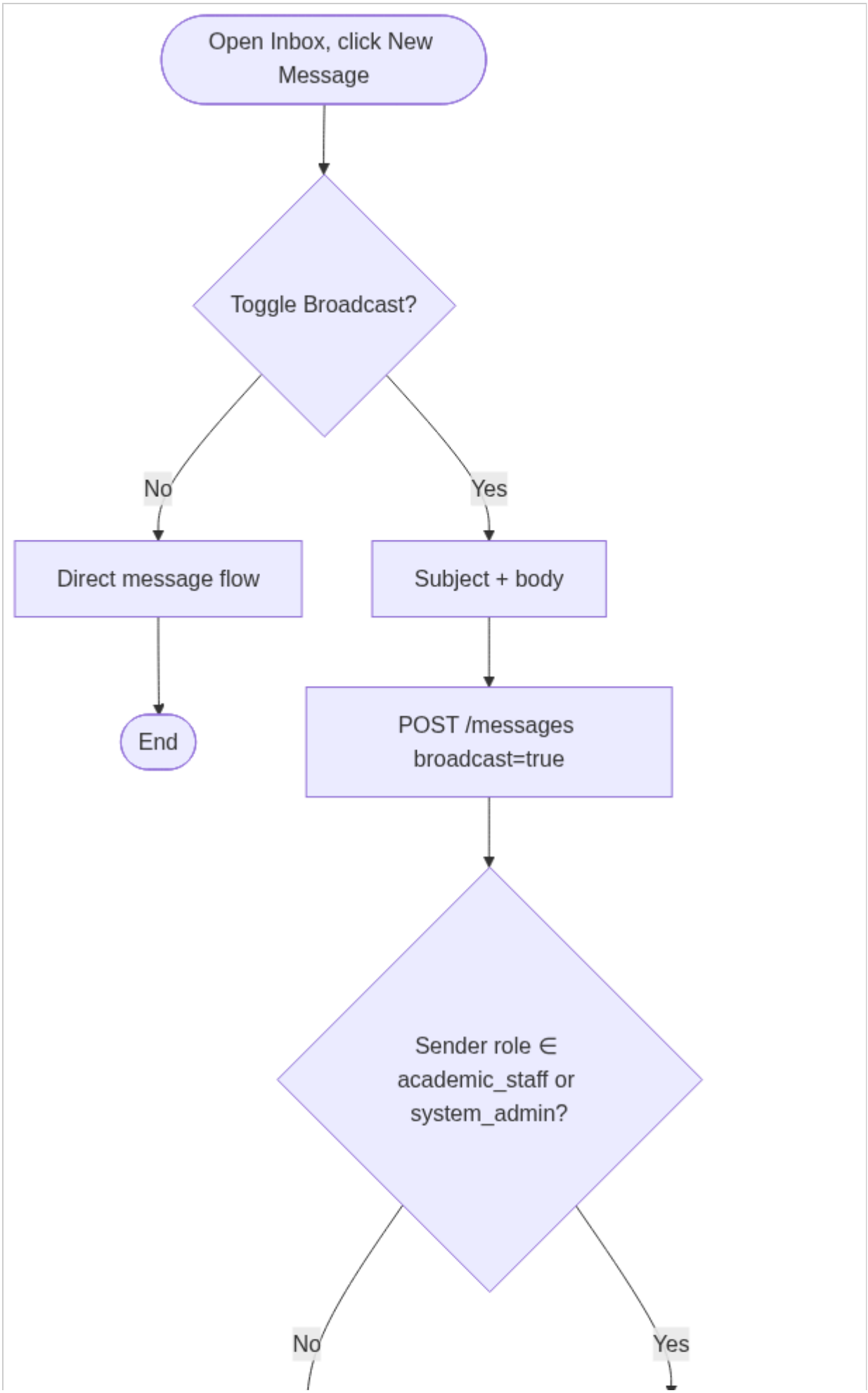
activity-diagrams-02

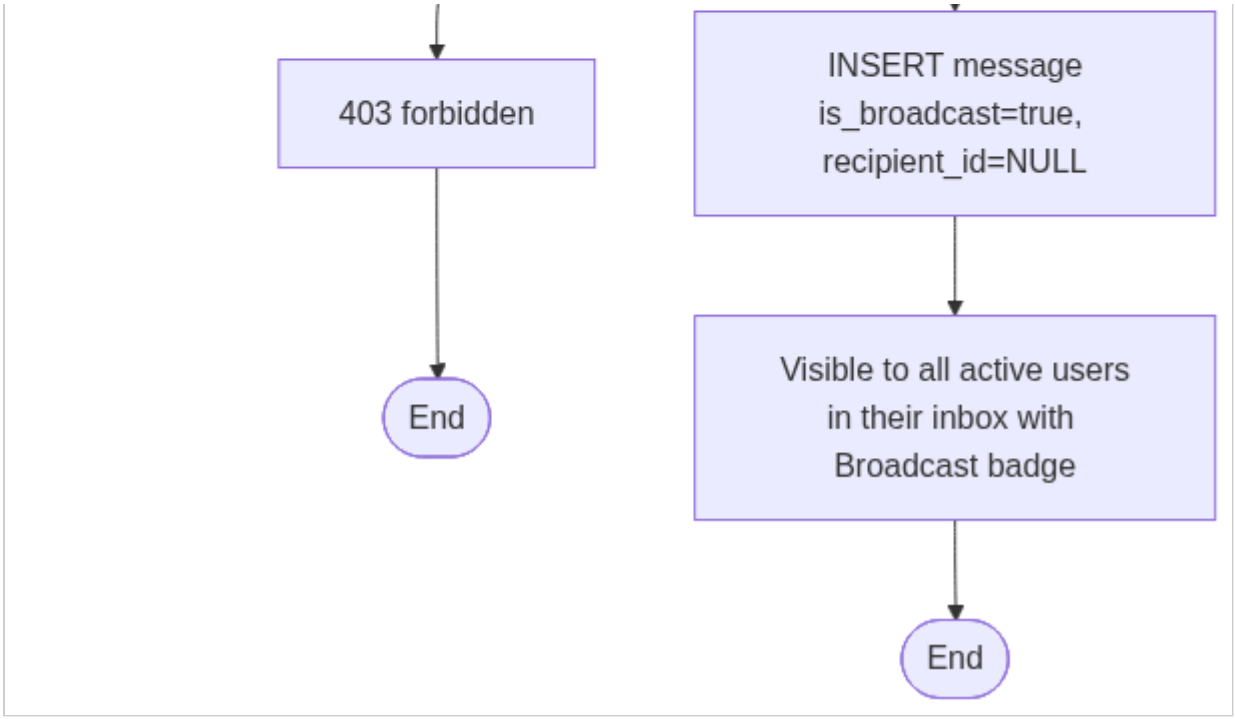


activity-diagrams-03

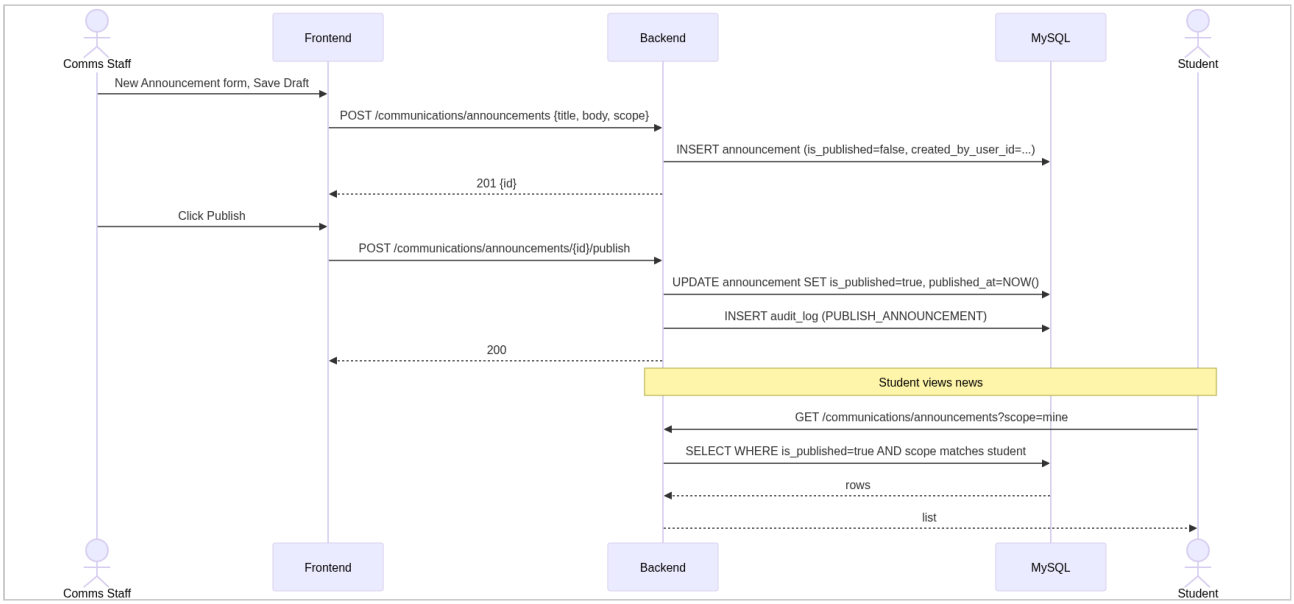

Open Clubs tab
List pending memberships
Pick request
Review student profile
Approve?
Yes
No
PATCH status=active
Optional reason
Notification: club joined
PATCH status=rejected
Student card flips to Member
Notification: rejected w/ reason
End
End

activity-diagrams-04

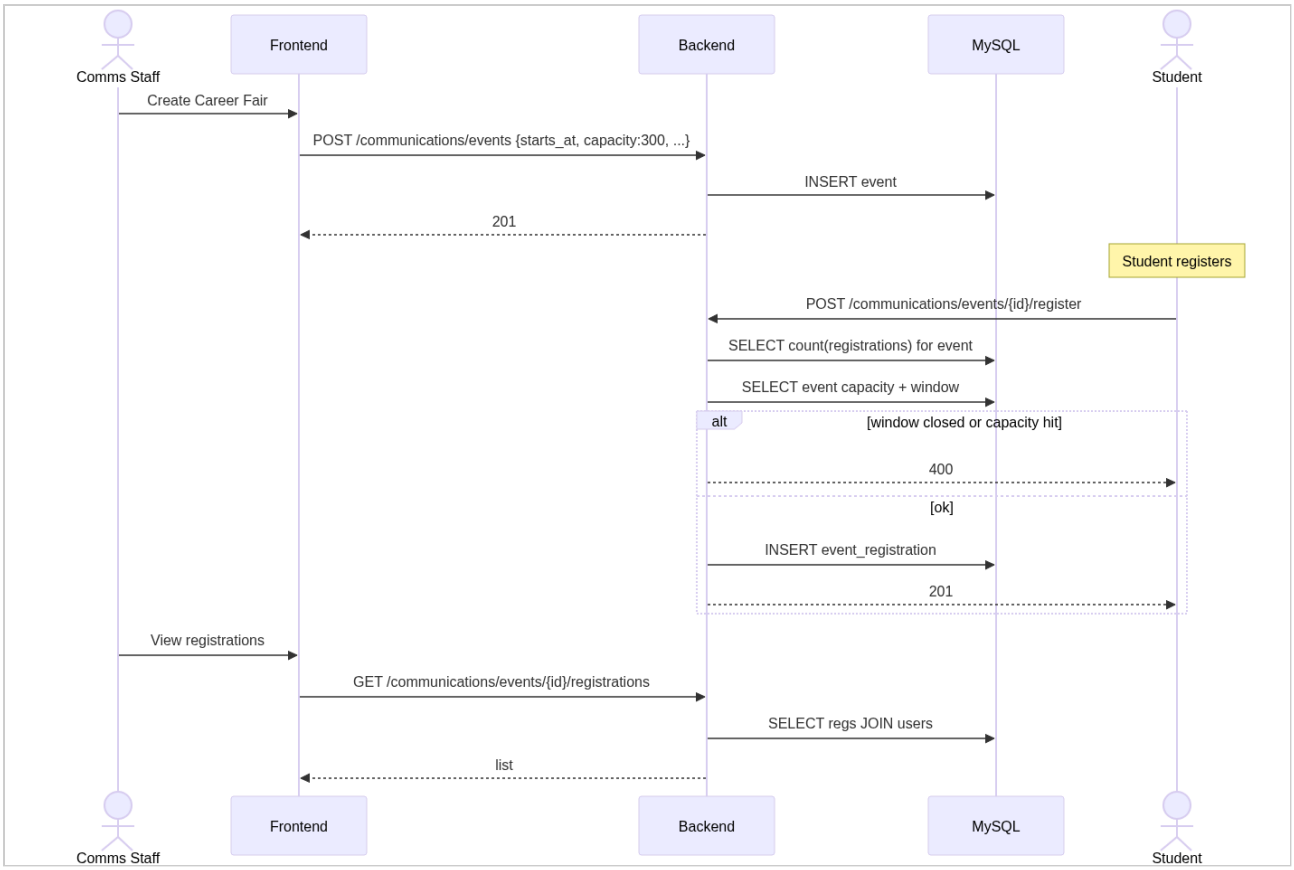

Open Inbox, click New Message
Toggle Broadcast?
No
Yes
Direct message flow
Subject + body
End
POST /messages broadcast=true
Sender role ∈ academic_staff or system_admin?
No
Yes



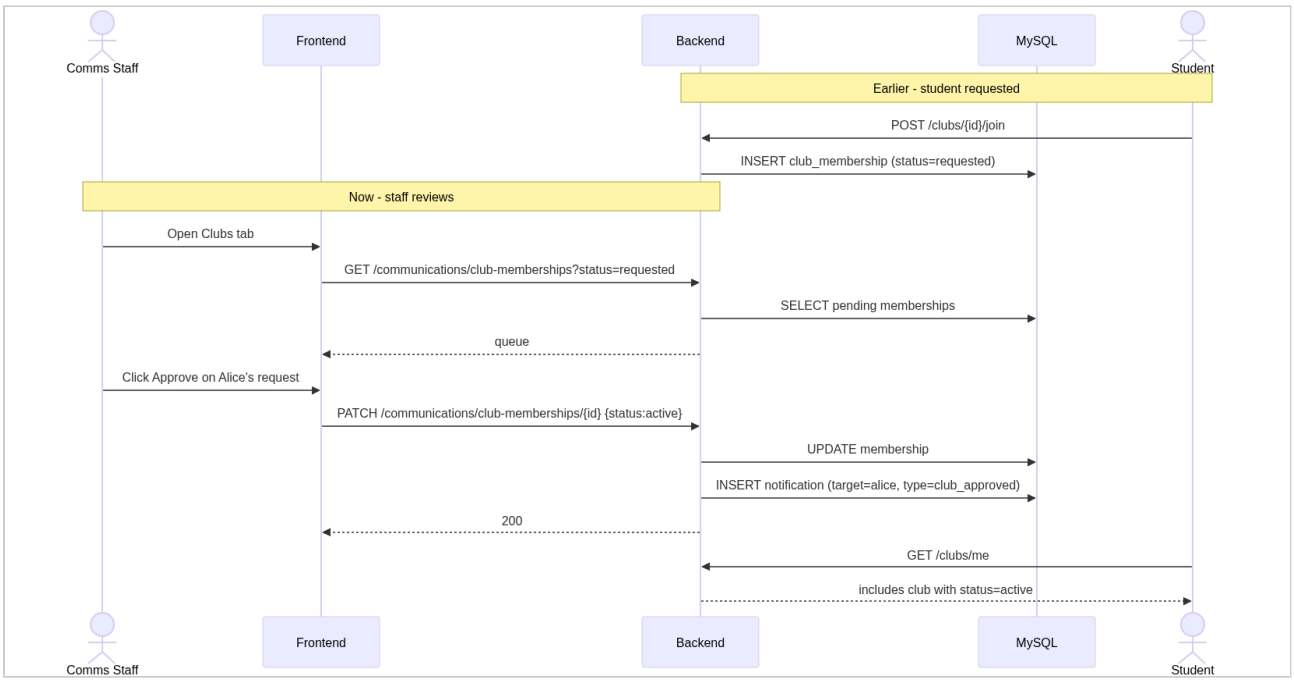
sequence-diagrams-01



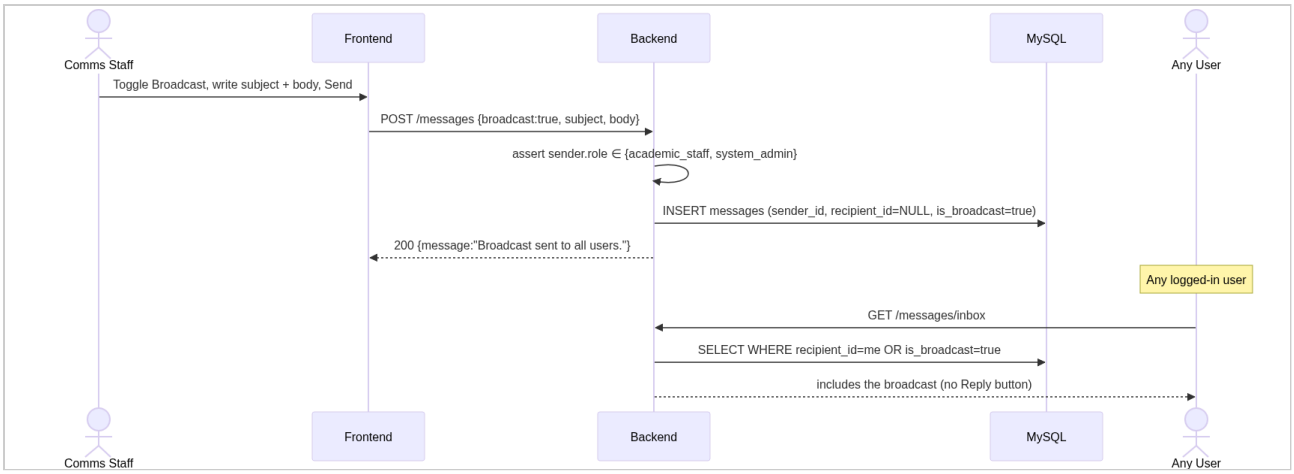
sequence-diagrams-02



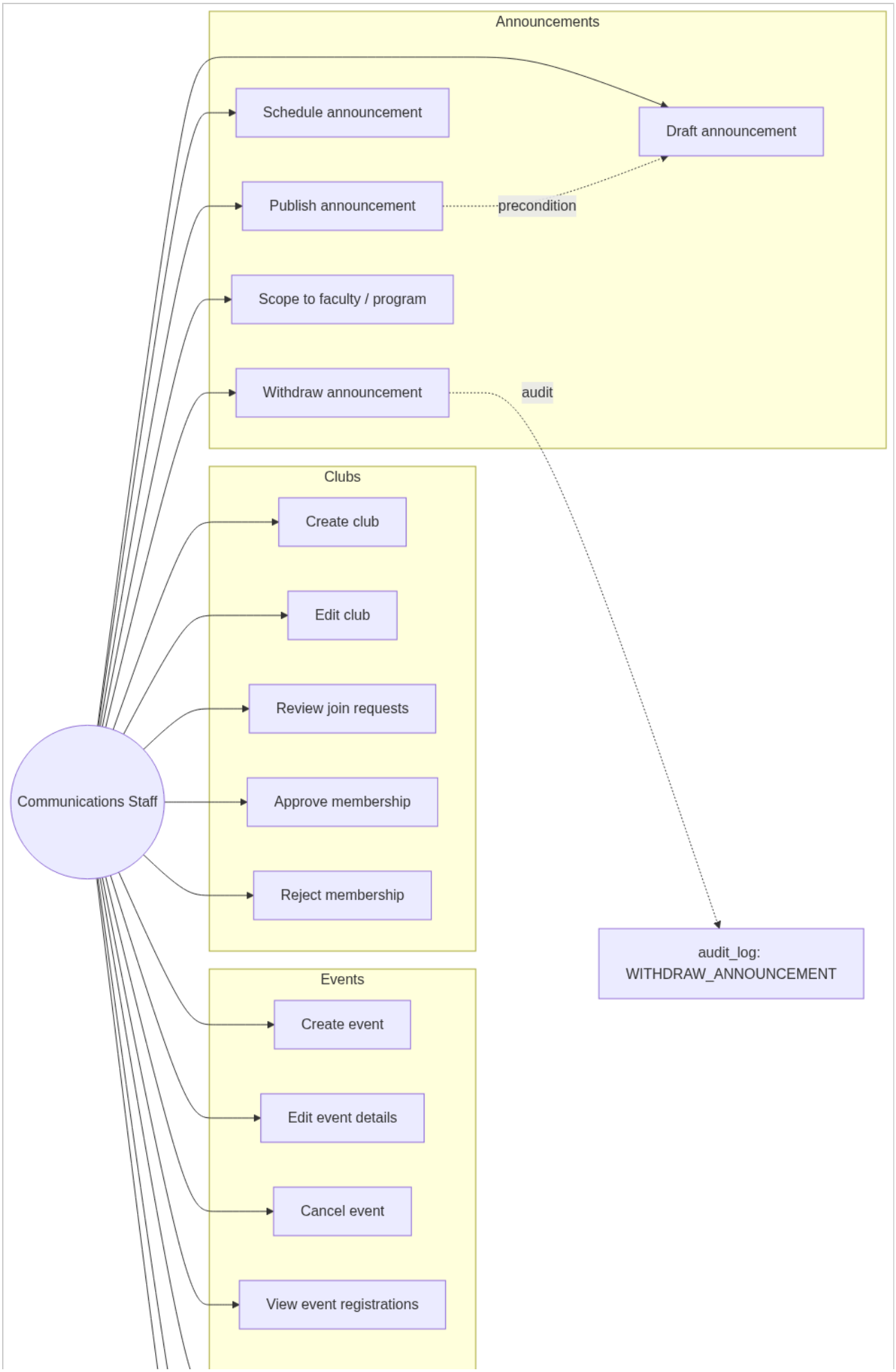
sequence-diagrams-03

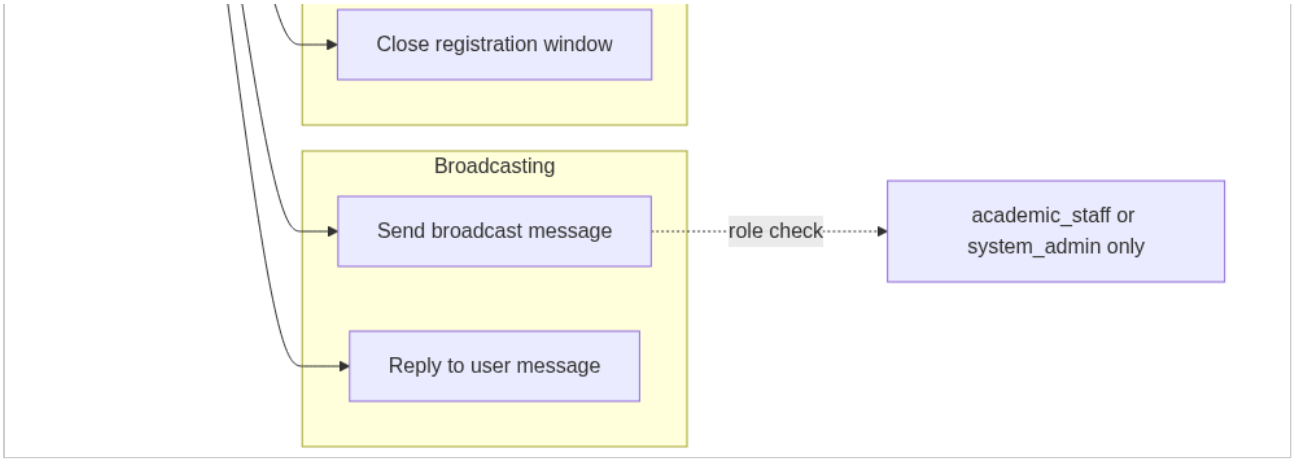


sequence-diagrams-04


Comms Staff
Frontend
Backend
MySQL
Any User
Toggle Broadcast, write subject + body, Send
POST /messages {broadcast:true, subject, body}
assert sender.role ∈ {academic_staff, system_admin}
INSERT messages (sender_id, recipient_id=NULL, is_broadcast=true)
200 {message:"Broadcast sent to all users."}
Any logged-in user
GET /messages/inbox
SELECT WHERE recipient_id=me OR is_broadcast=true
includes the broadcast (no Reply button)
Comms Staff
Frontend
Backend
MySQL
Any User

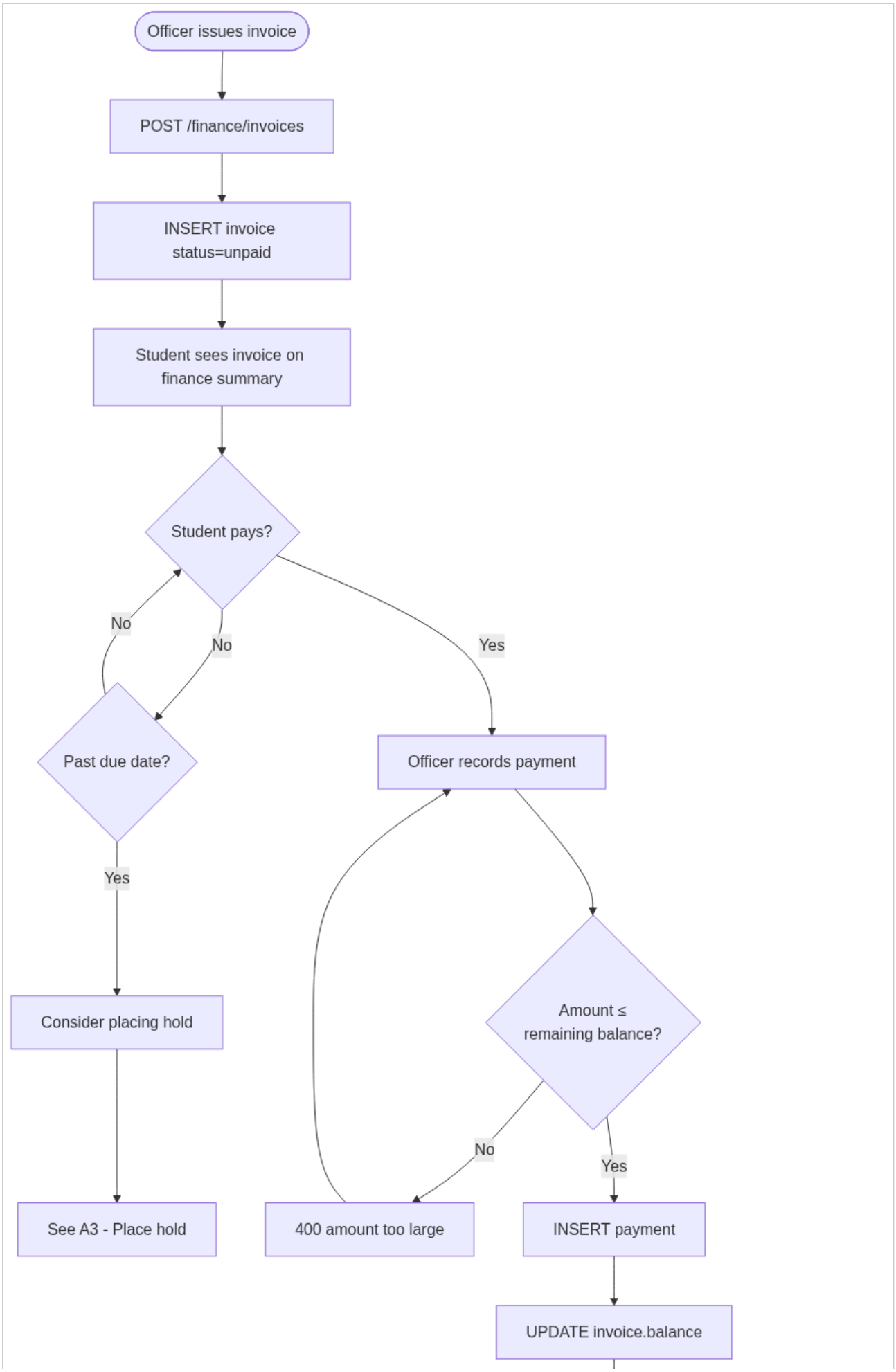
use-cases-01

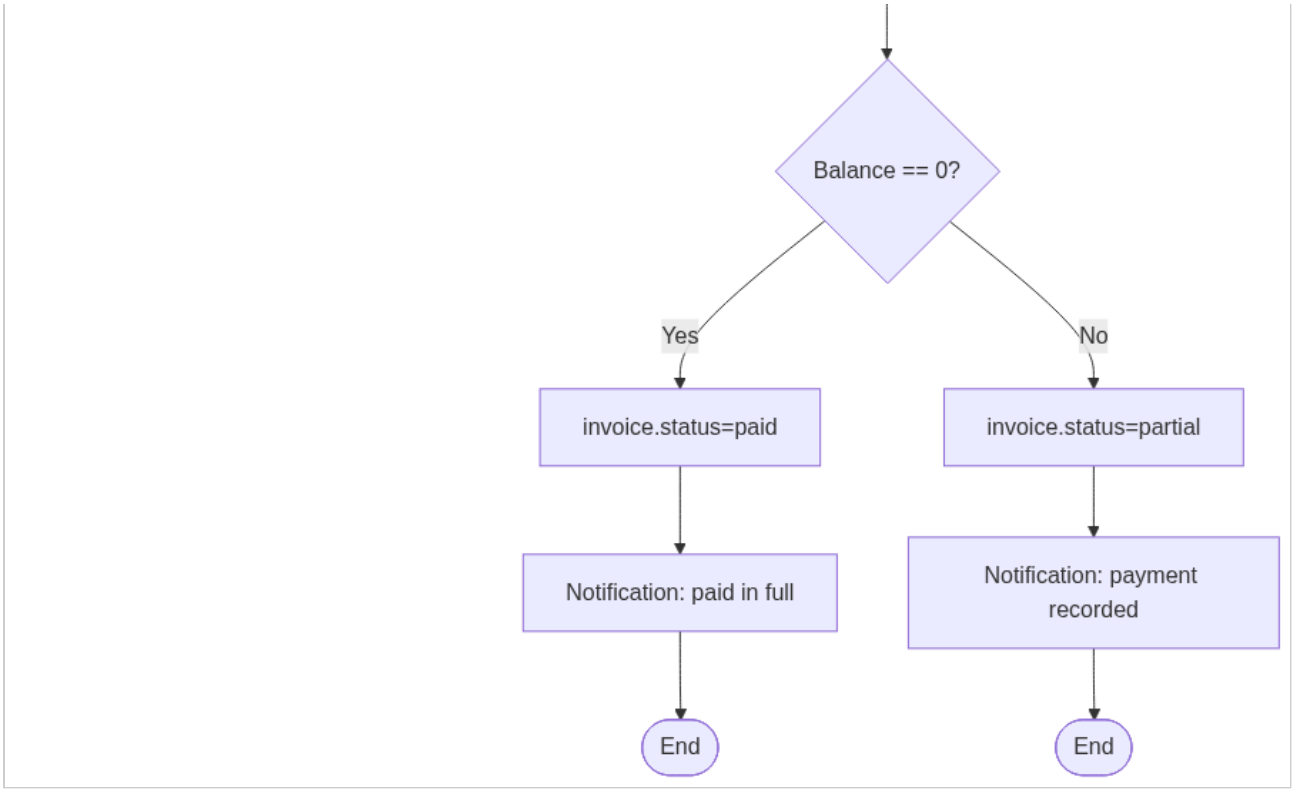

Announcements
Schedule announcement
Draft announcement
Publish announcement
precondition
Scope to faculty / program
Withdraw announcement
audit
Clubs
Create club
Edit club
Review join requests
Communications Staff
Approve membership
Reject membership
audit_log:
WITHDRAW_ANNOUNCEMENT
Events
Create event
Edit event details
Cancel event
View event registrations



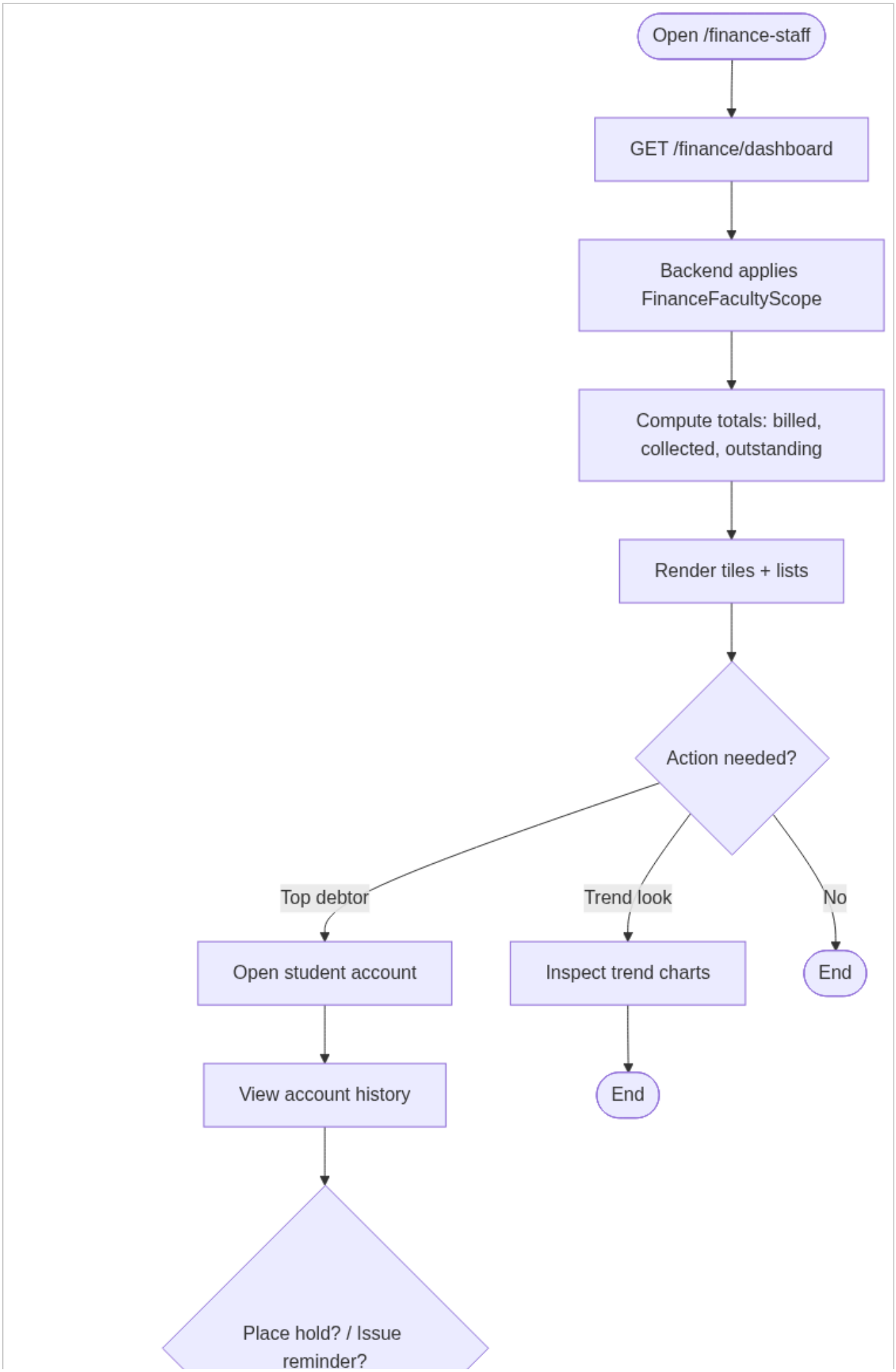
Finance Staff

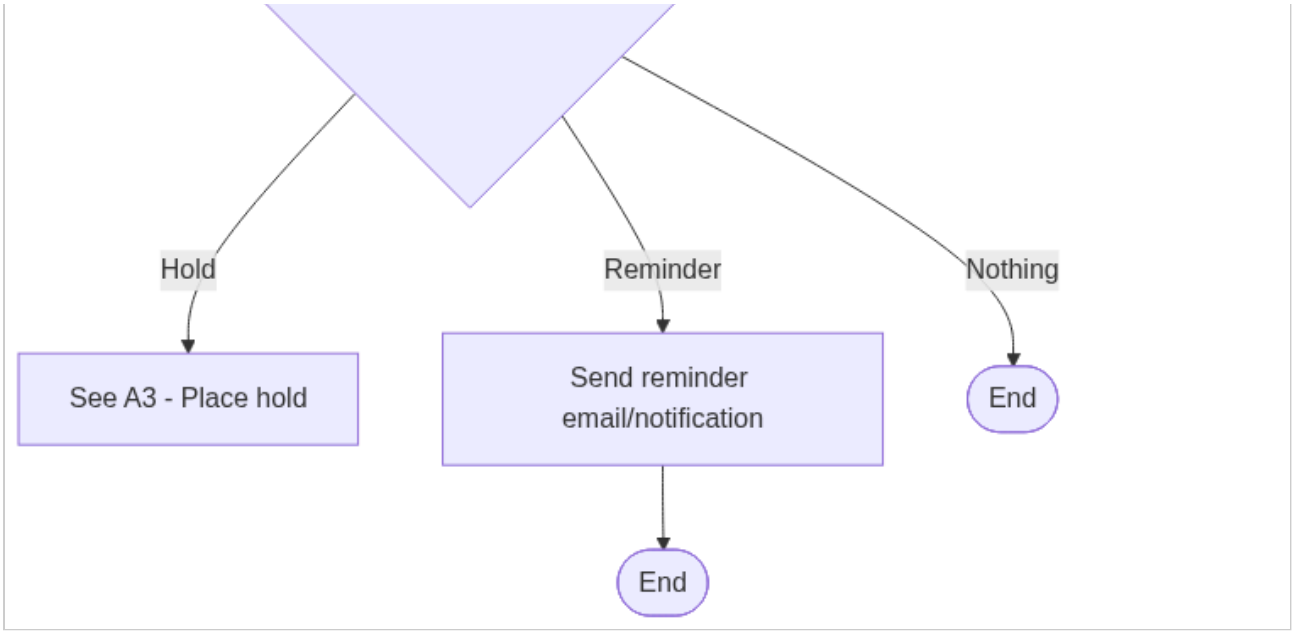
activity-diagrams-01


Officer issues invoice
POST /finance/invoices
INSERT invoice
status=unpaid
Student sees invoice on
finance summary
Student pays?
No
No
Yes
Past due date?
Officer records payment
Yes
Consider placing hold
Amount ≤
remaining balance?
No
Yes
See A3 - Place hold
400 amount too large
INSERT payment
UPDATE invoice.balance

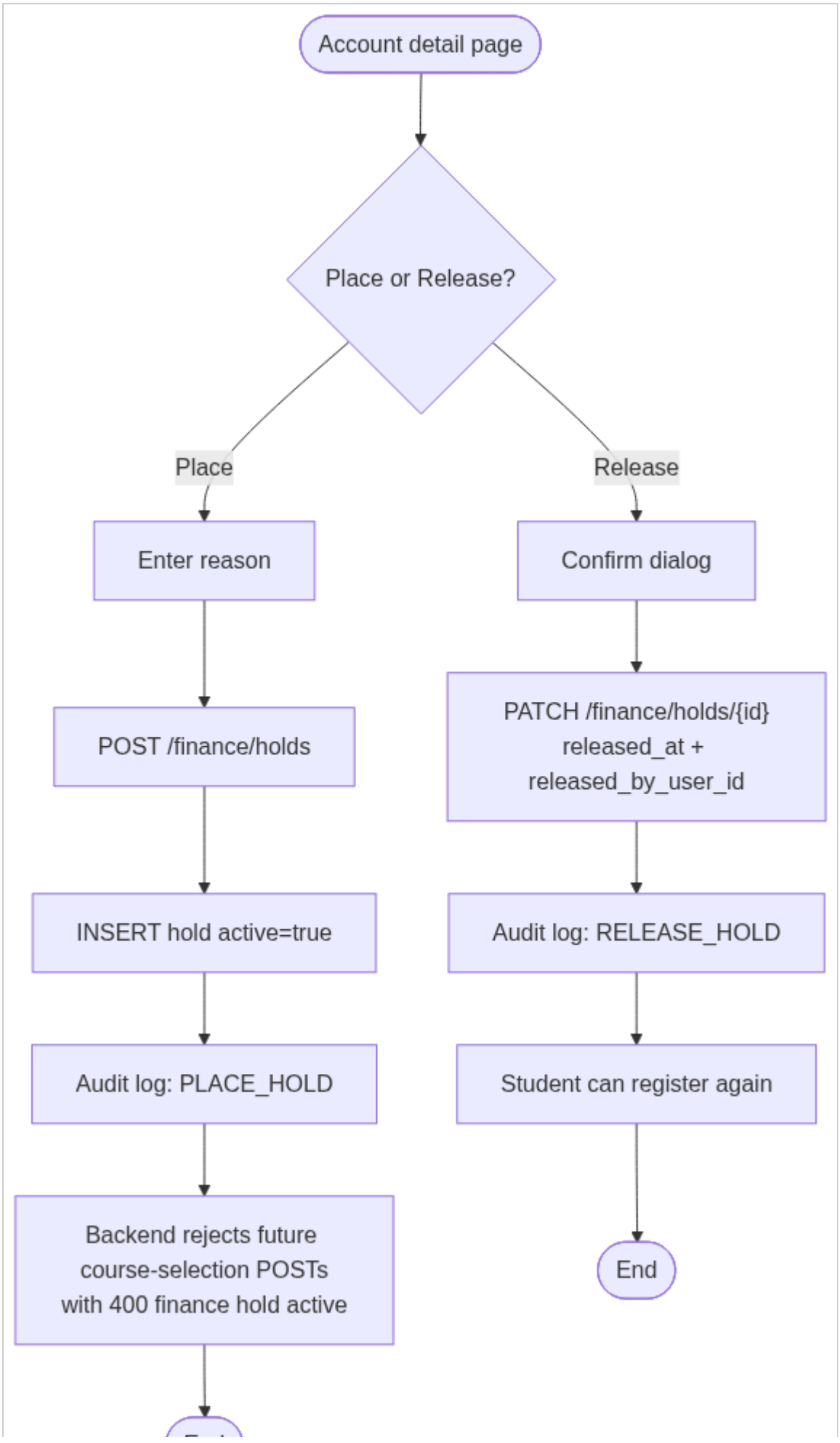


activity-diagrams-02

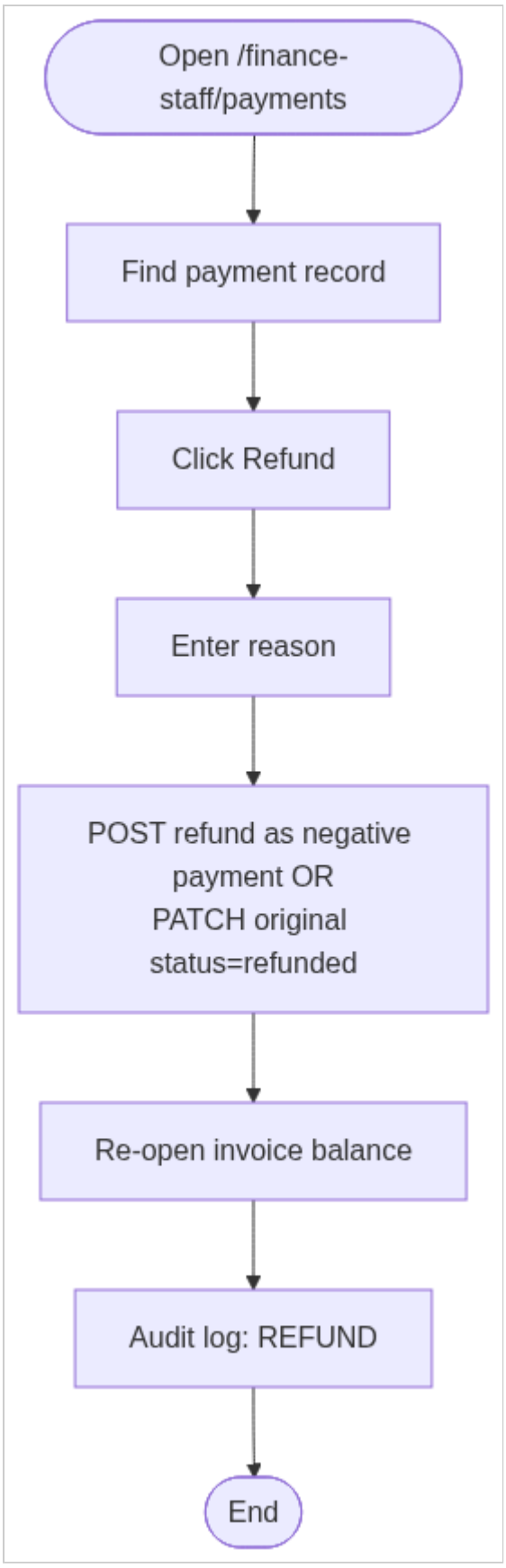

Open /finance-staff
GET /finance/dashboard
Backend applies FinanceFacultyScope
Compute totals: billed, collected, outstanding
Render tiles + lists
Action needed?
Top debtor
Trend look
No
Open student account
Inspect trend charts
End
View account history
End
Place hold? / Issue reminder?



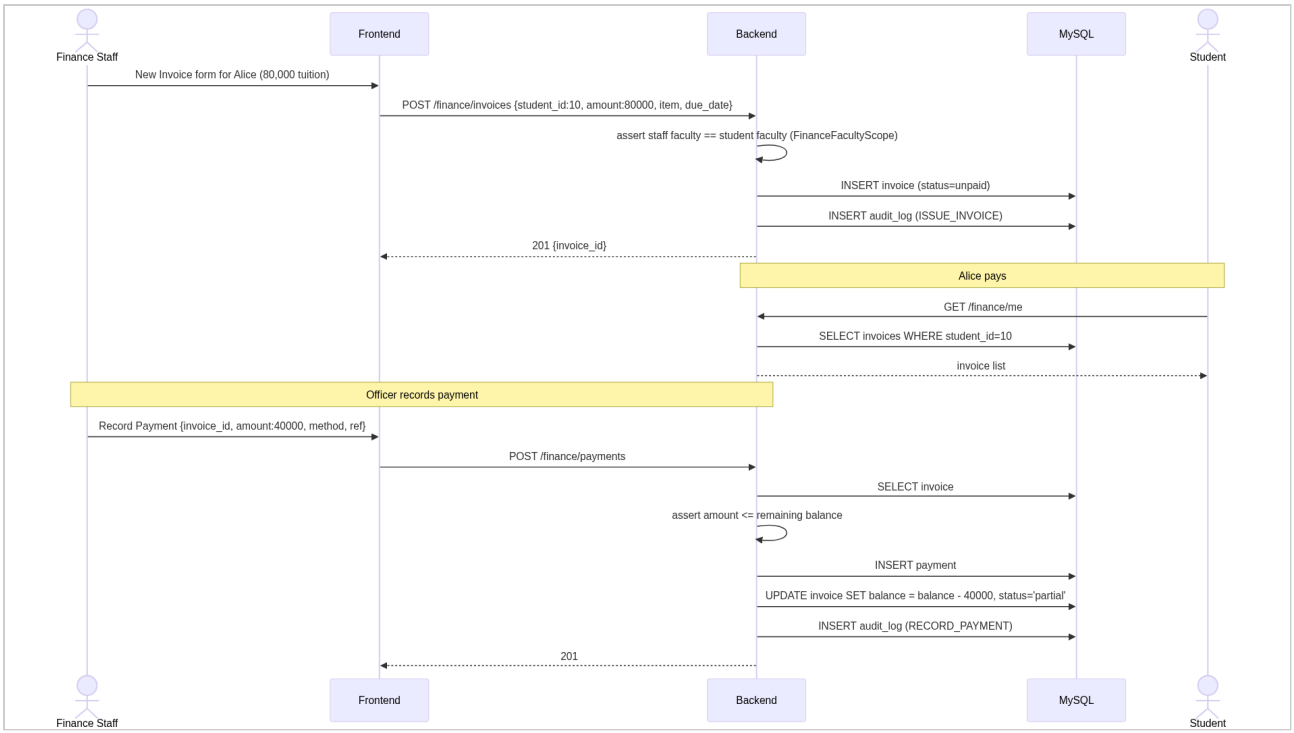
activity-diagrams-03


Account detail page
Place or Release?
Place
Release
Enter reason
Confirm dialog
POST /finance/holds
PATCH /finance/holds/{id}
released_at +
released_by_user_id
INSERT hold active=true
Audit log: RELEASE_HOLD
Audit log: PLACE_HOLD
Student can register again
Backend rejects future
course-selection POSTs
with 400 finance hold active
End

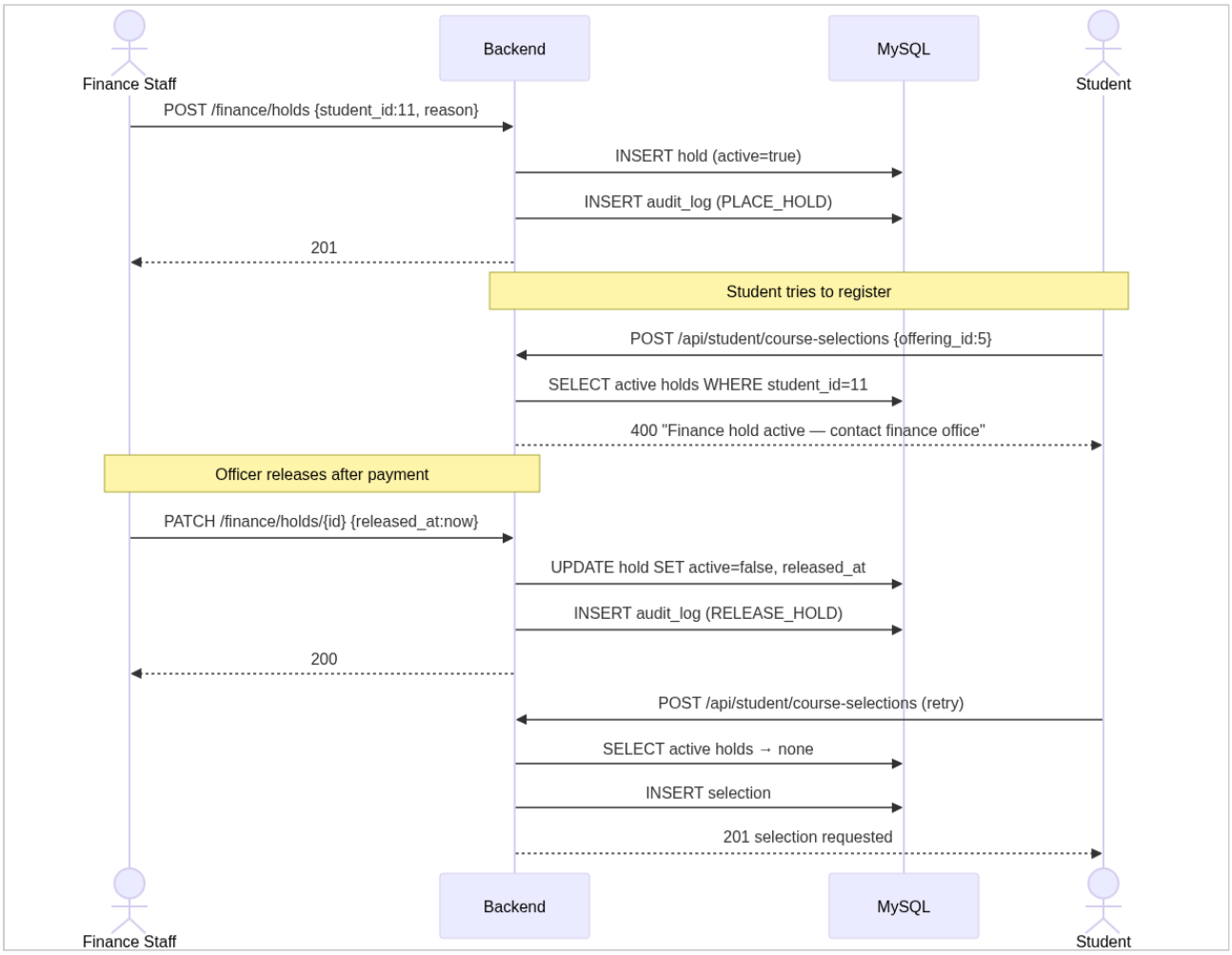
activity-diagrams-04



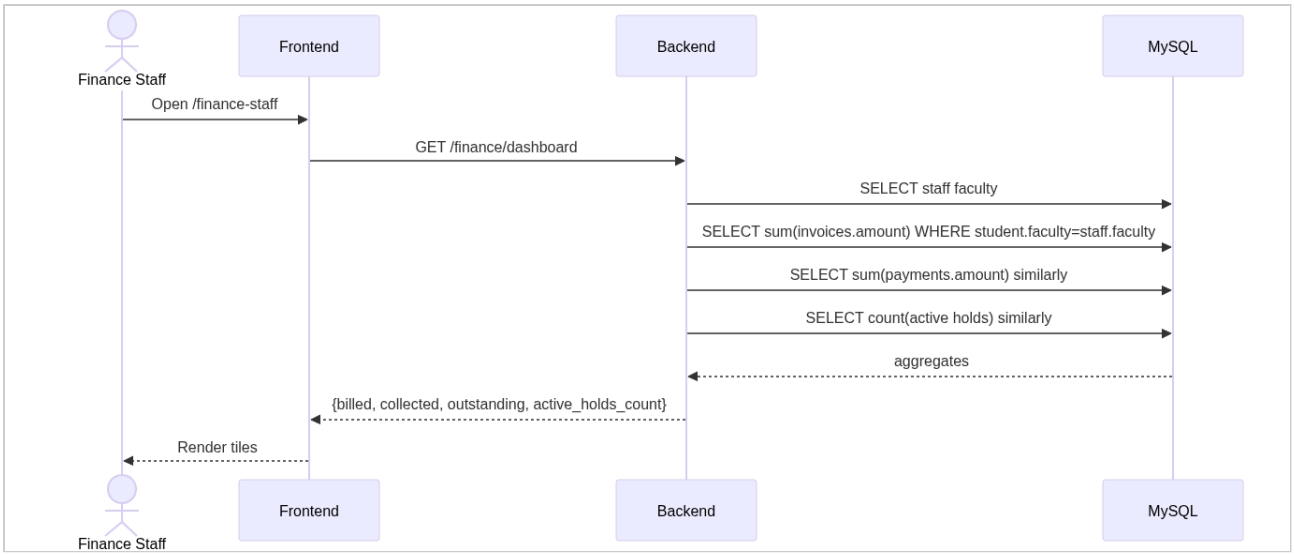
sequence-diagrams-01



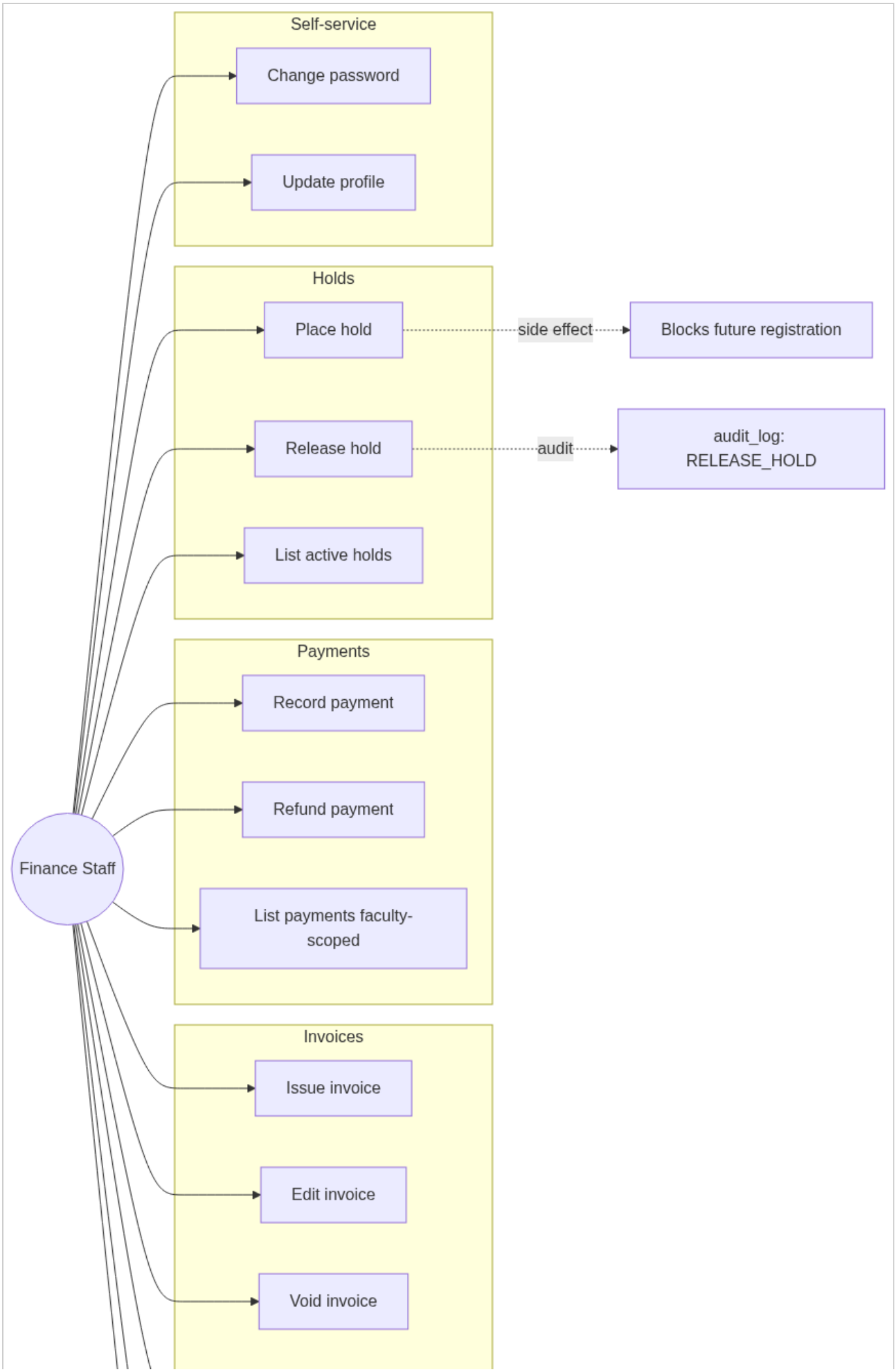
sequence-diagrams-02

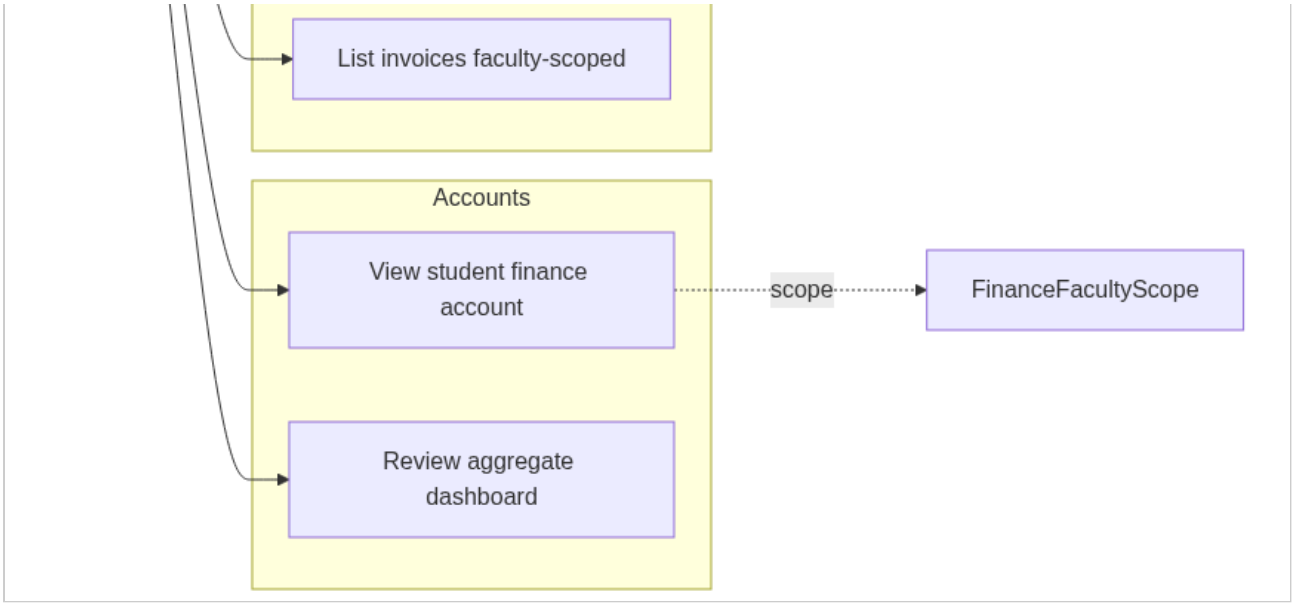


sequence-diagrams-03


Finance Staff
Frontend
Backend
MySQL
Open /finance-staff
GET /finance/dashboard
SELECT staff faculty
SELECT sum(invoices.amount) WHERE student.faculty=staff.faculty
SELECT sum(payments.amount) similarly
SELECT count(active holds) similarly
aggregates
{billed, collected, outstanding, active_holds_count}
Render tiles
Frontend
Backend
MySQL
Finance Staff

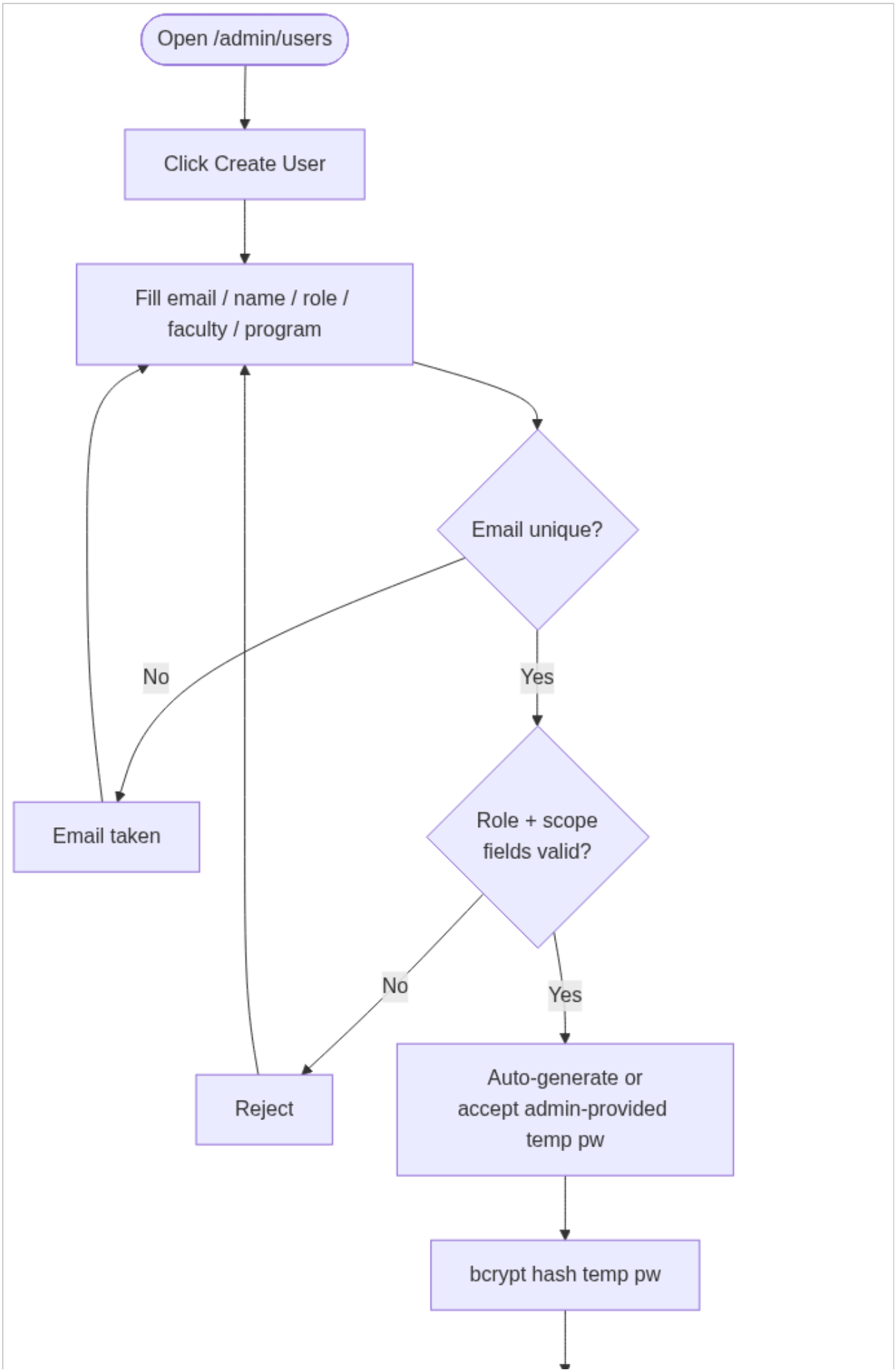
use-cases-01

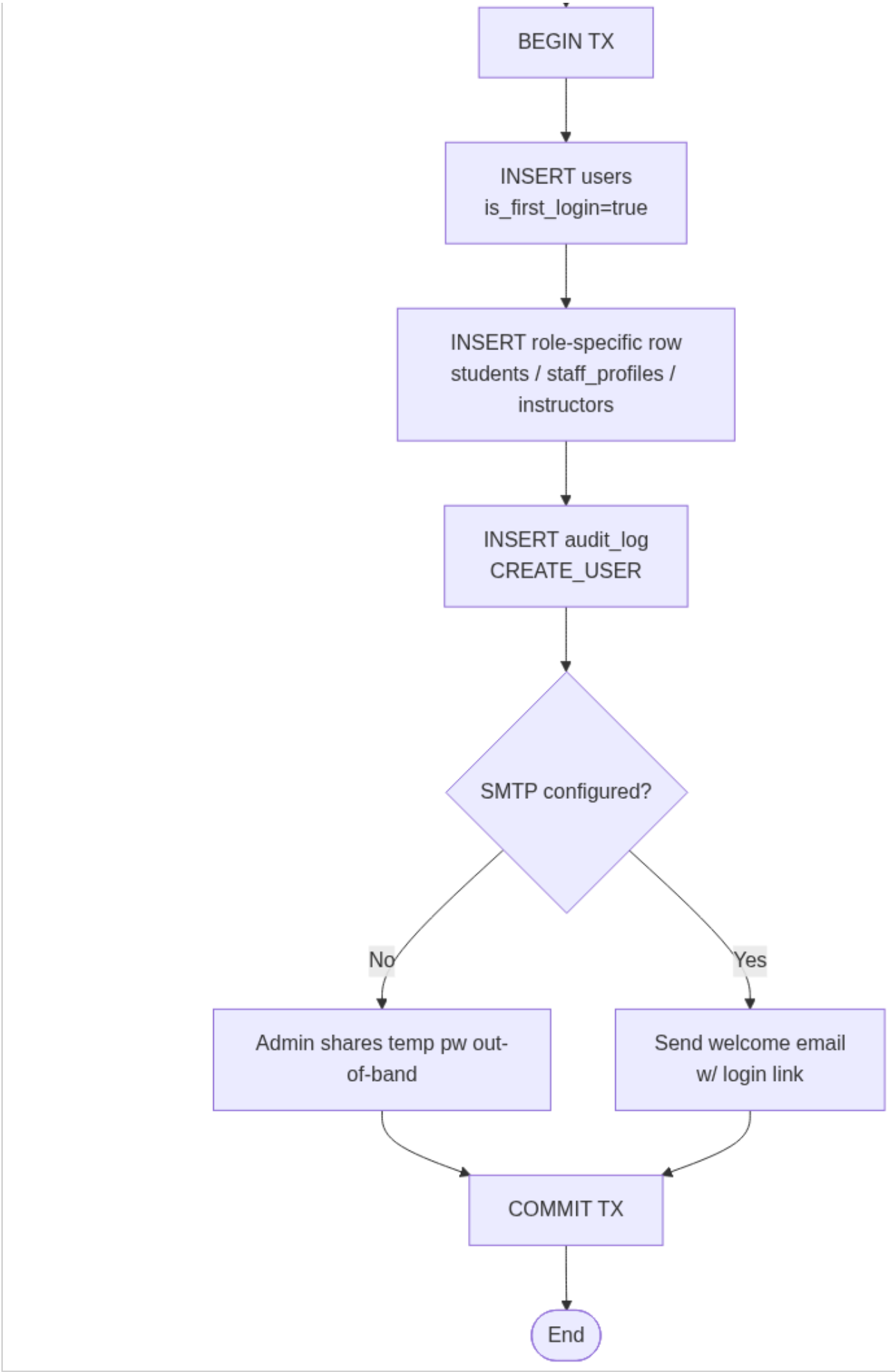

Self-service
Change password
Update profile
Holds
Place hold
side effect
Blocks future registration
Release hold
audit
audit_log:
RELEASE_HOLD
List active holds
Payments
Record payment
Refund payment
Finance Staff
List payments faculty-
scoped
Invoices
Issue invoice
Edit invoice
Void invoice



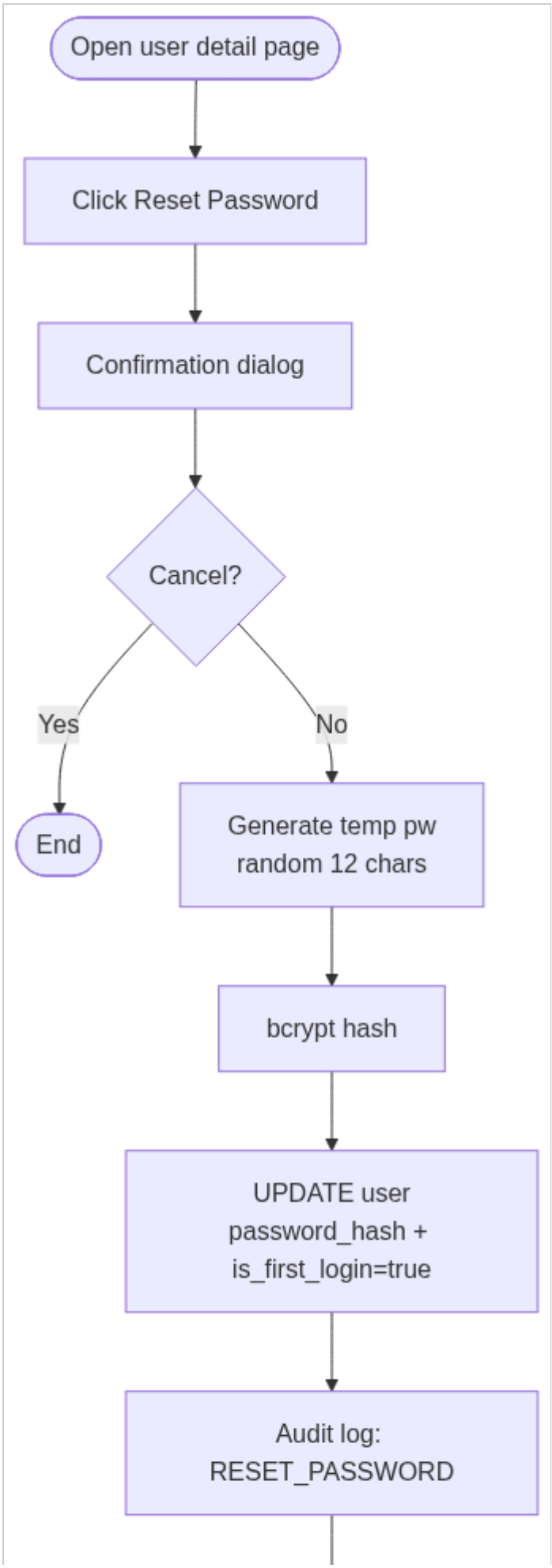
System Admin

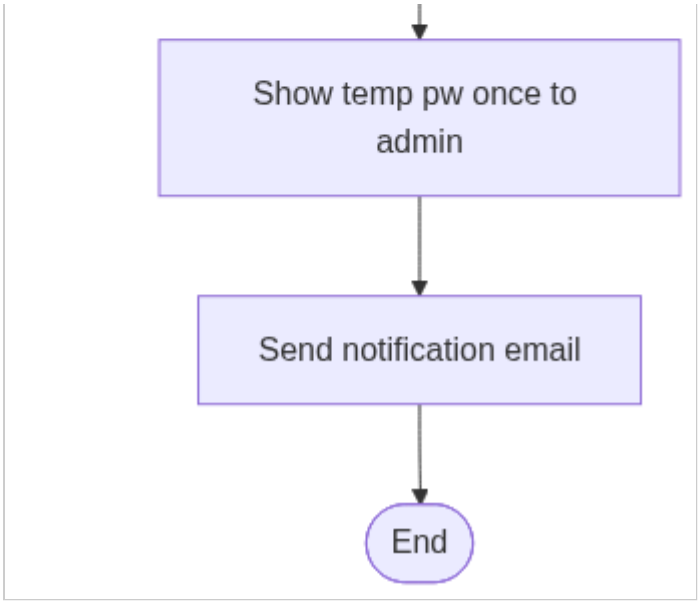
activity-diagrams-01


Open /admin/users
Click Create User
Fill email / name / role / faculty / program
Email unique?
No
Yes
Email taken
Role + scope fields valid?
No
Yes
Reject
Auto-generate or accept admin-provided temp pw
bcrypt hash temp pw

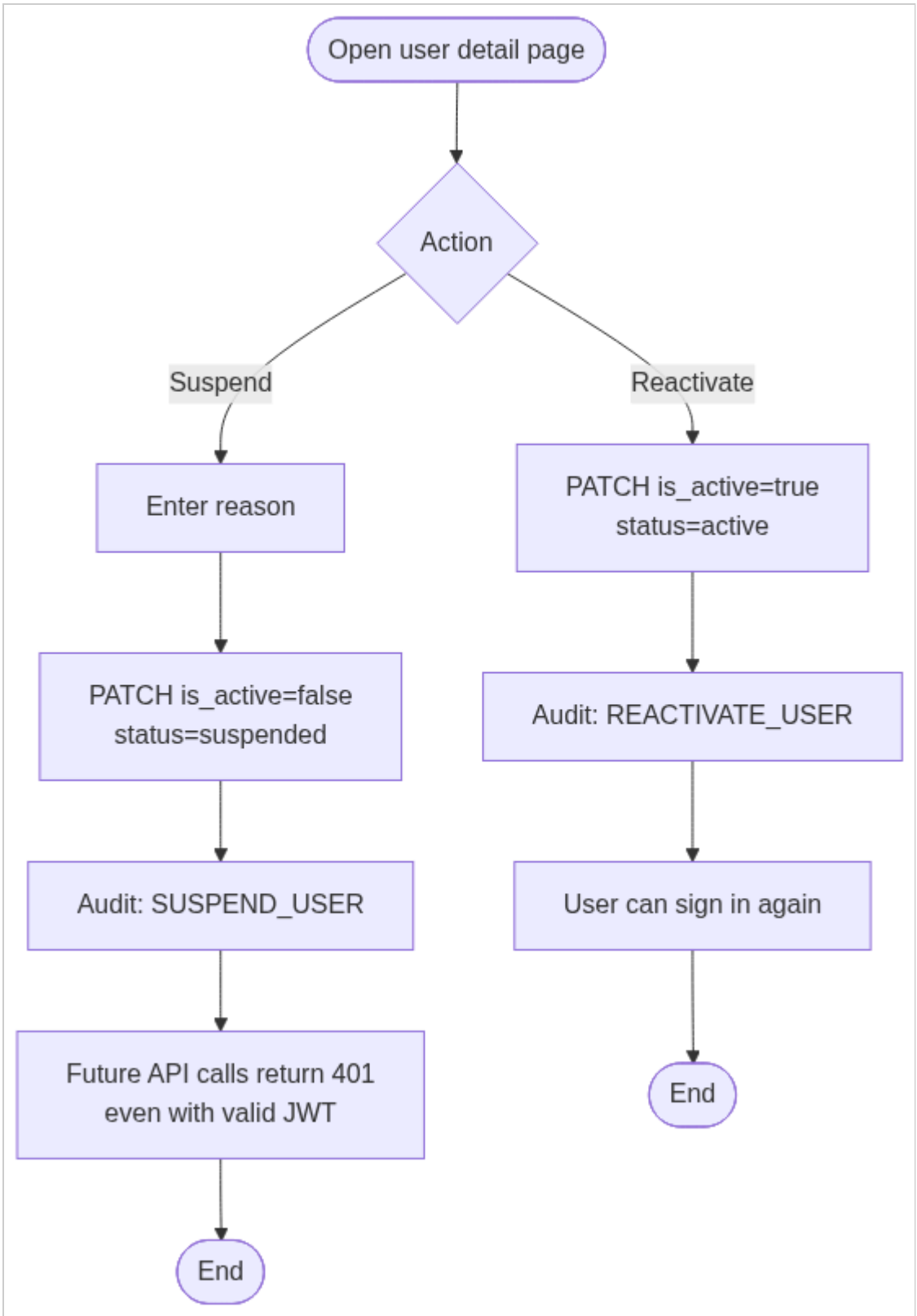

BEGIN TX
INSERT users
is_first_login=true
INSERT role-specific row
students / staff_profiles /
instructors
INSERT audit_log
CREATE_USER
SMTP configured?
No
Yes
Admin shares temp pw out-
of-band
Send welcome email
w/ login link
COMMIT TX
End

activity-diagrams-02

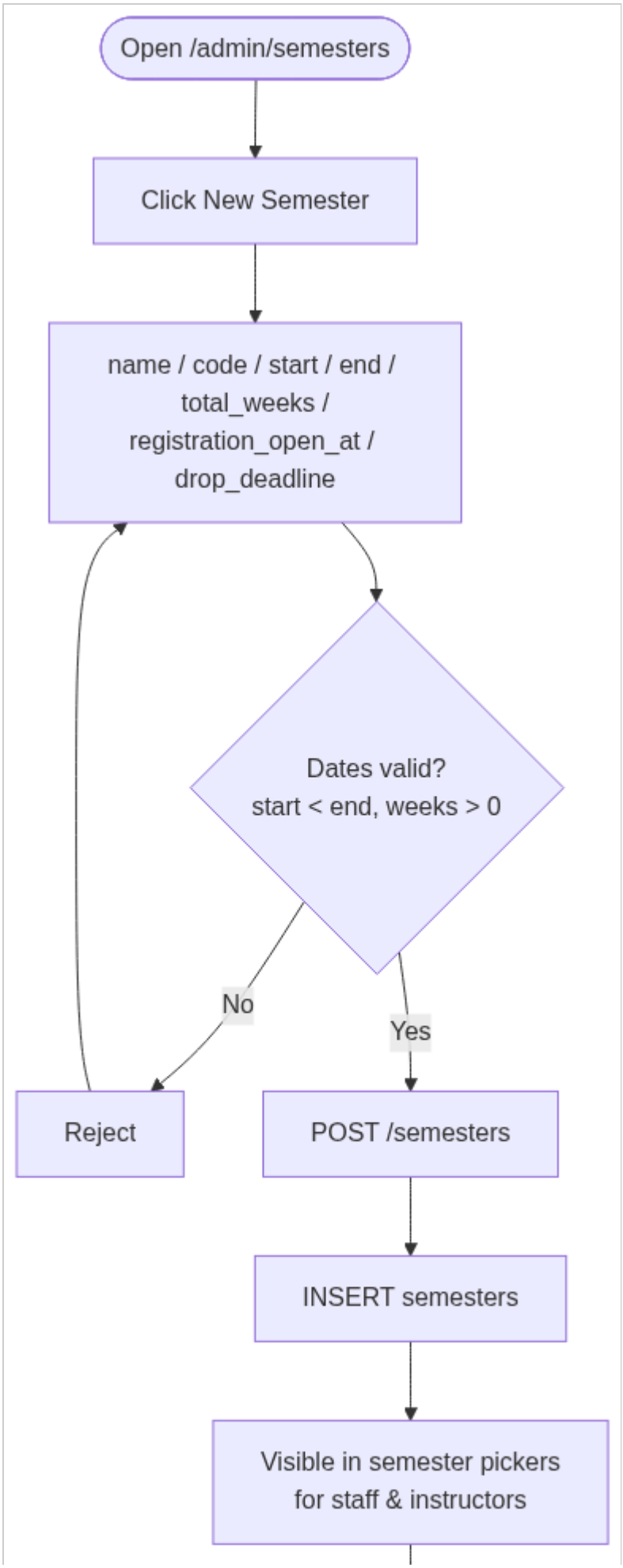

Open user detail page
Click Reset Password
Confirmation dialog
Cancel?
Yes
No
End
Generate temp pw
random 12 chars
bcrypt hash
UPDATE user
password_hash +
is_first_login=true
Audit log:
RESET_PASSWORD

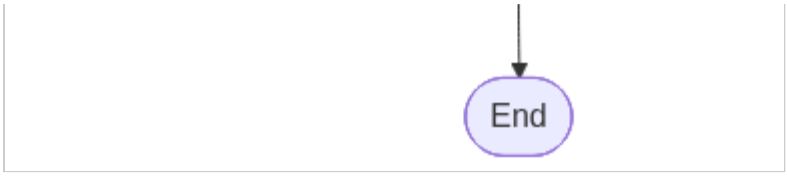


activity-diagrams-03

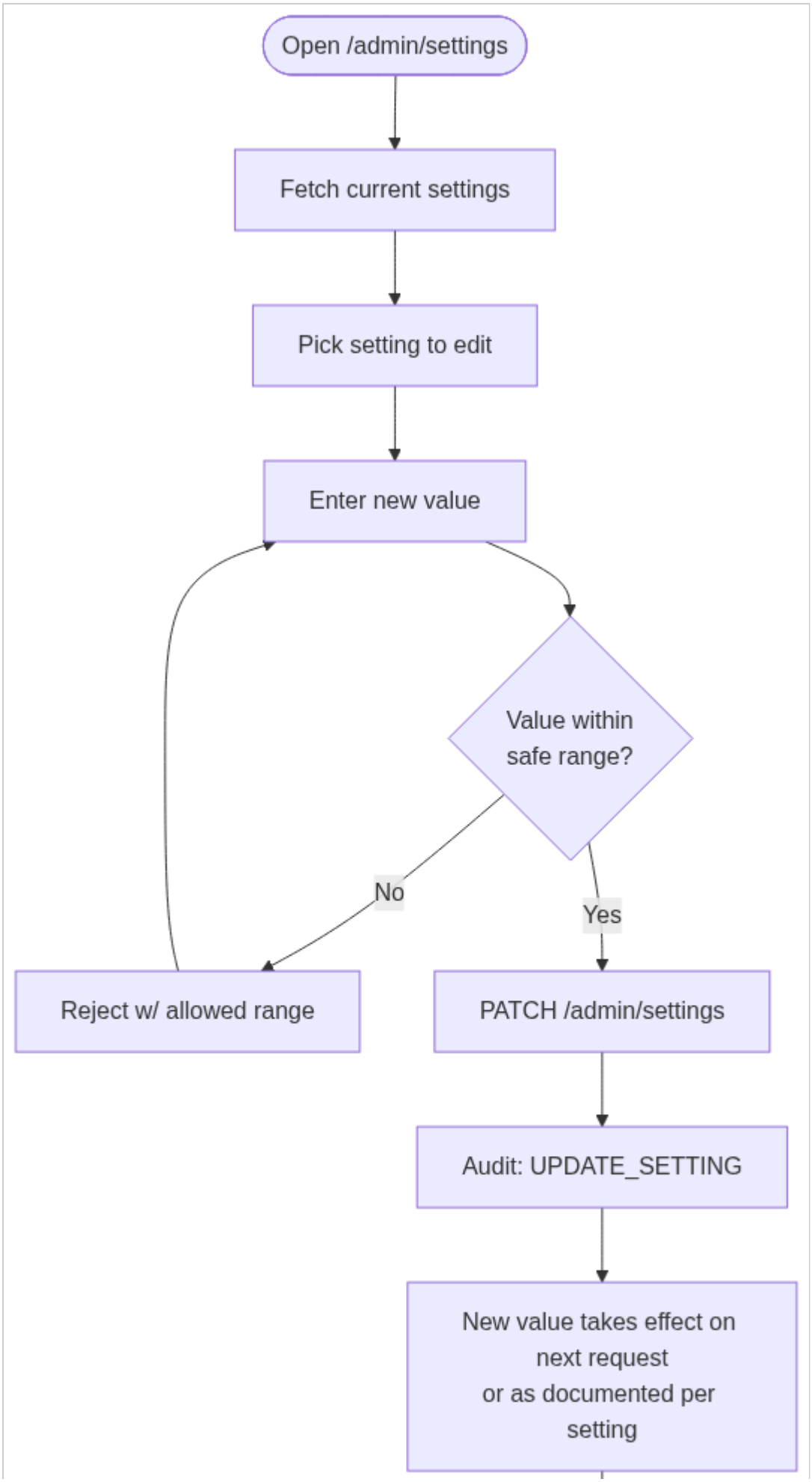


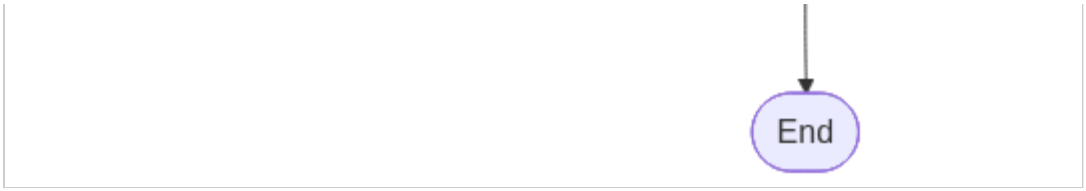
activity-diagrams-04


Open /admin/semesters
Click New Semester
name / code / start / end /
total_weeks /
registration_open_at /
drop_deadline
Dates valid?
start < end, weeks > 0
No
Yes
Reject
POST /semesters
INSERT semesters
Visible in semester pickers
for staff & instructors

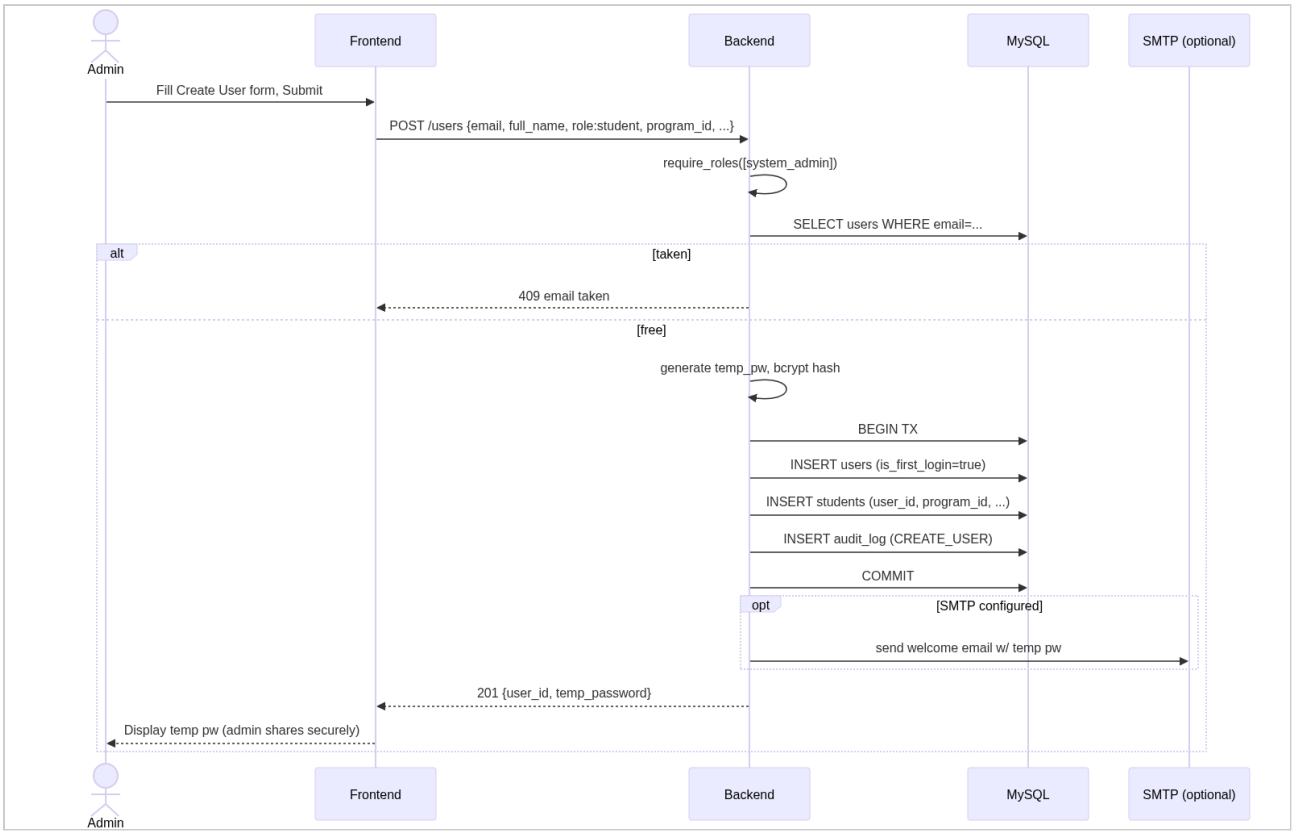


activity-diagrams-05

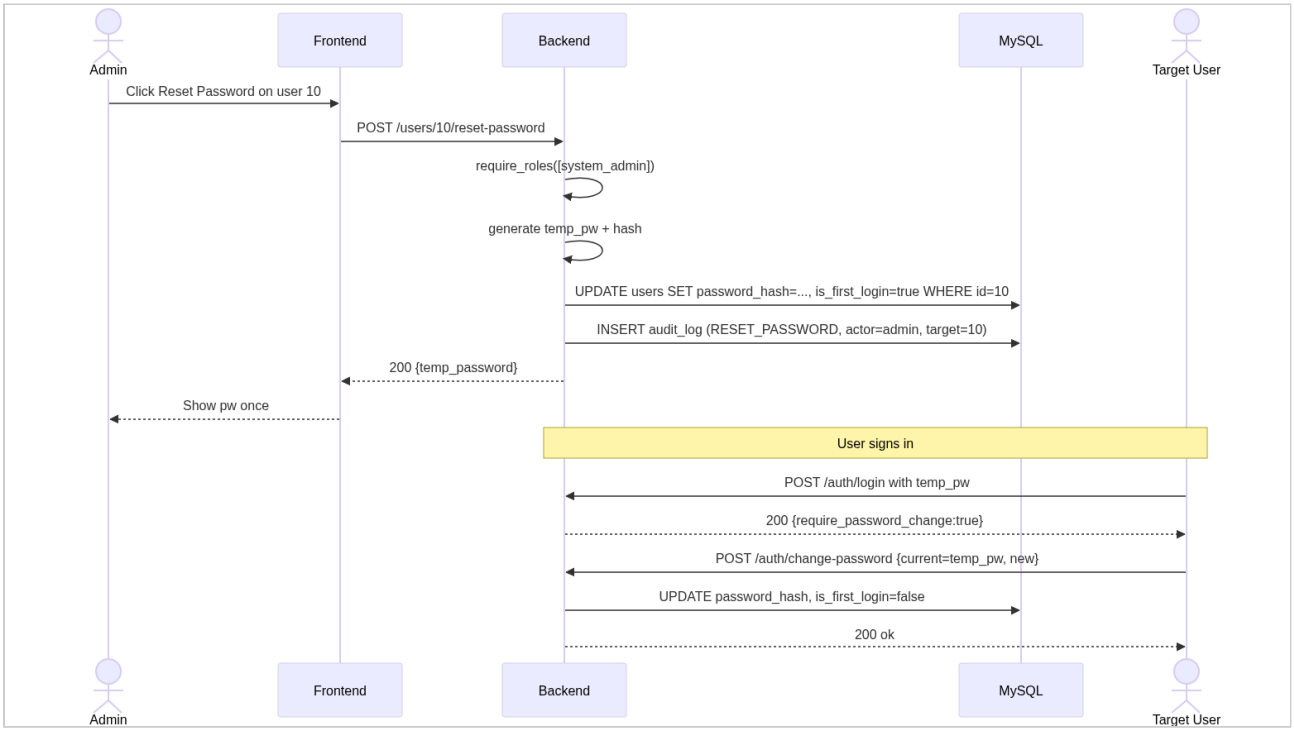

Open /admin/settings
Fetch current settings
Pick setting to edit
Enter new value
Value within safe range?
No
Yes
Reject w/ allowed range
PATCH /admin/settings
Audit: UPDATE_SETTING
New value takes effect on next request or as documented per setting



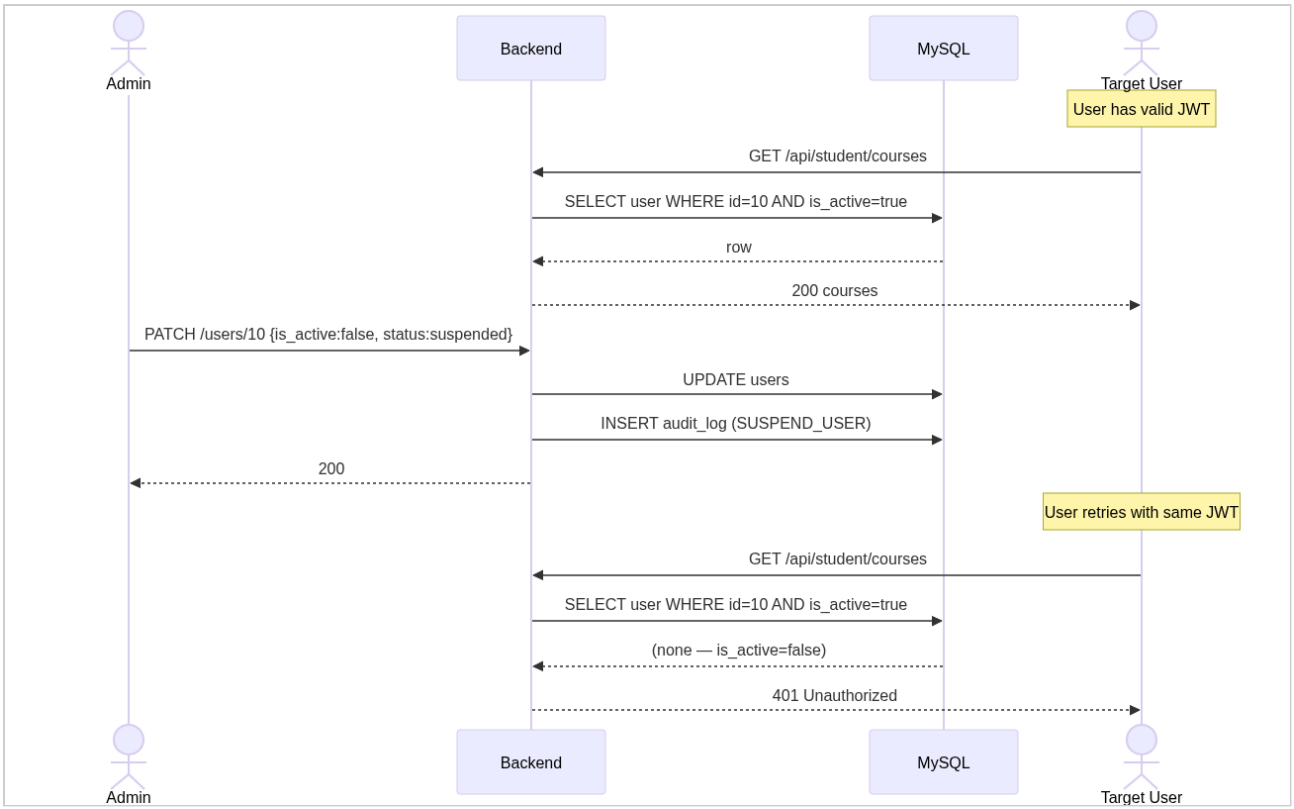
sequence-diagrams-01



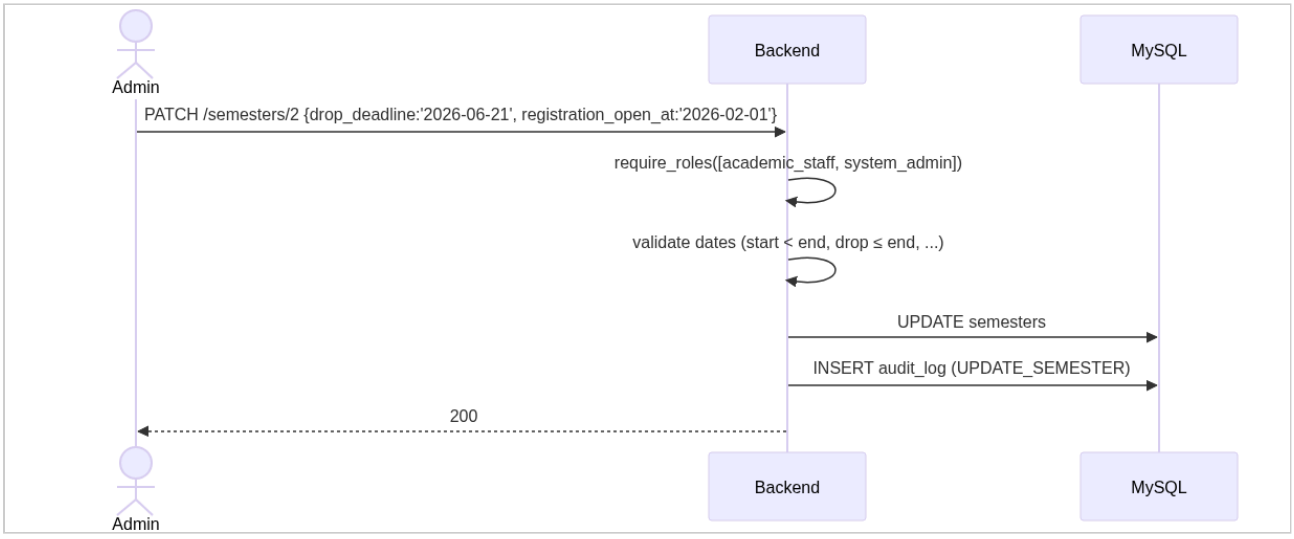
sequence-diagrams-02



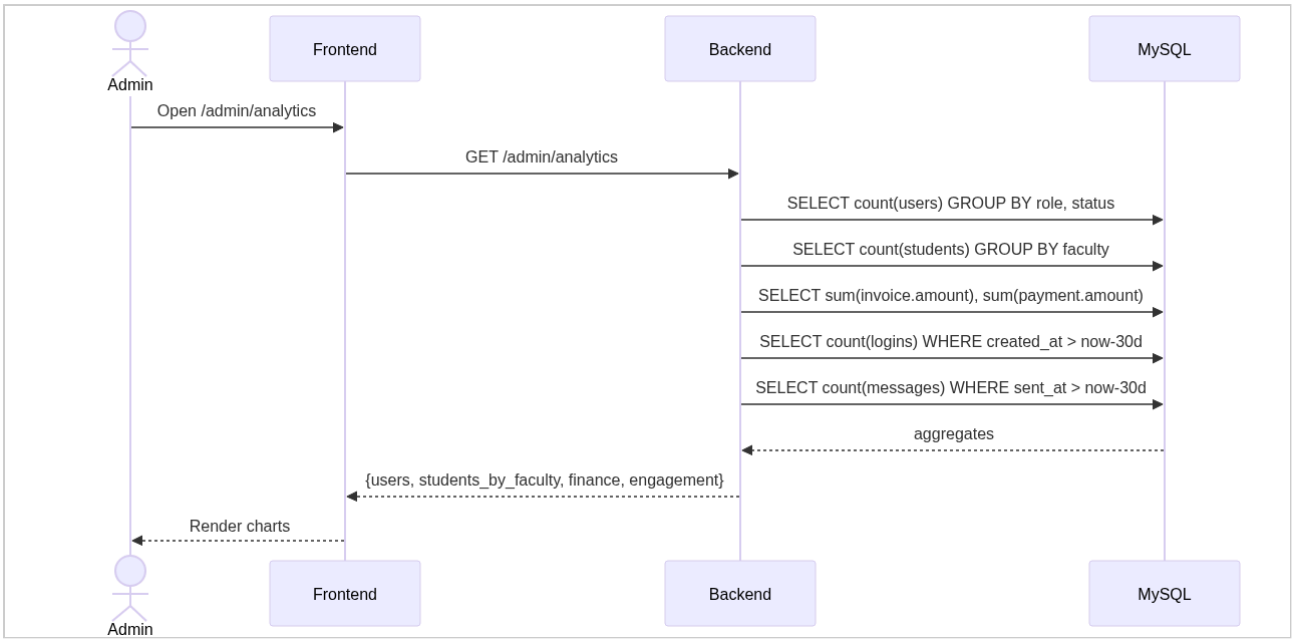
sequence-diagrams-03



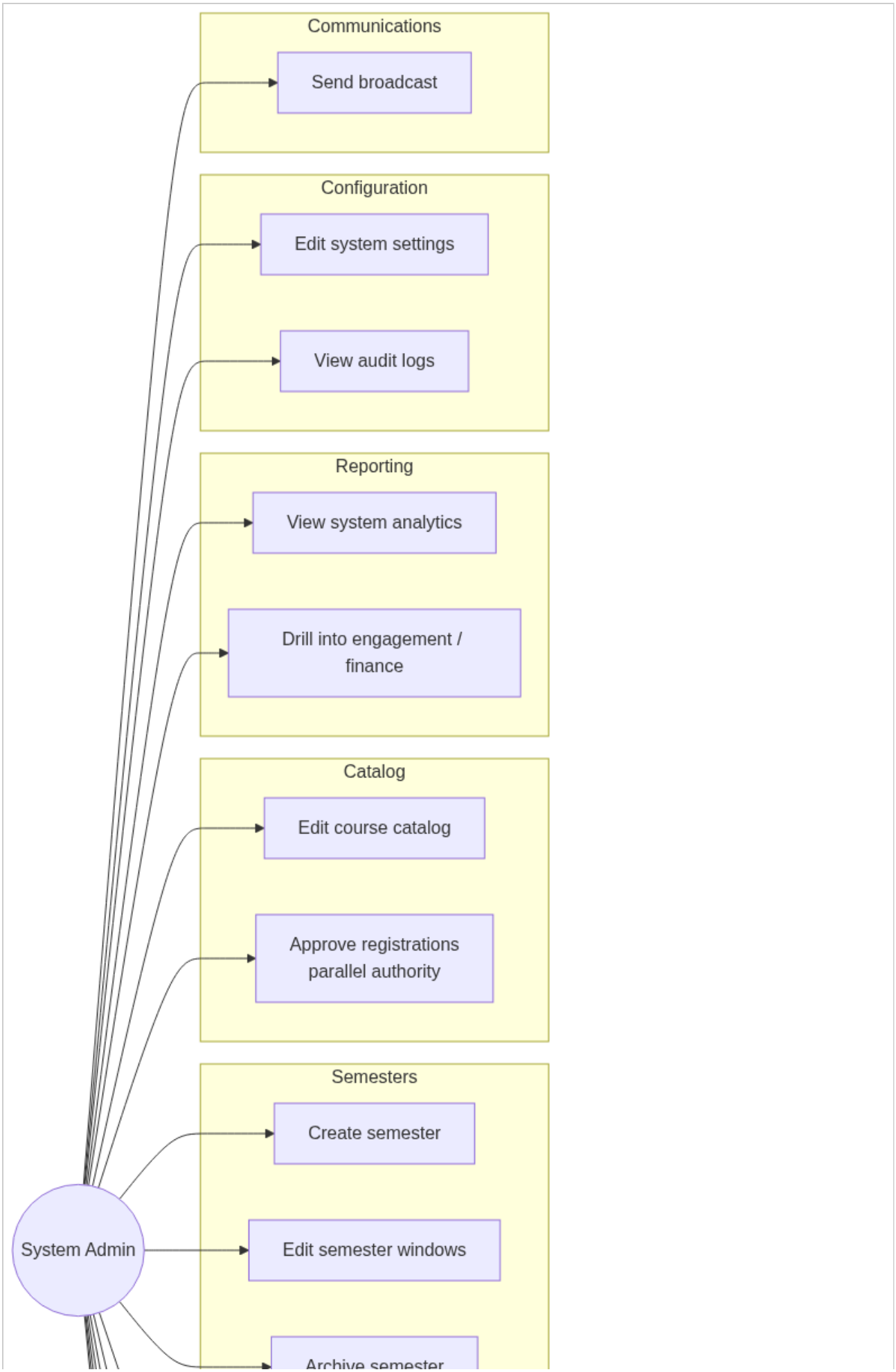
sequence-diagrams-04

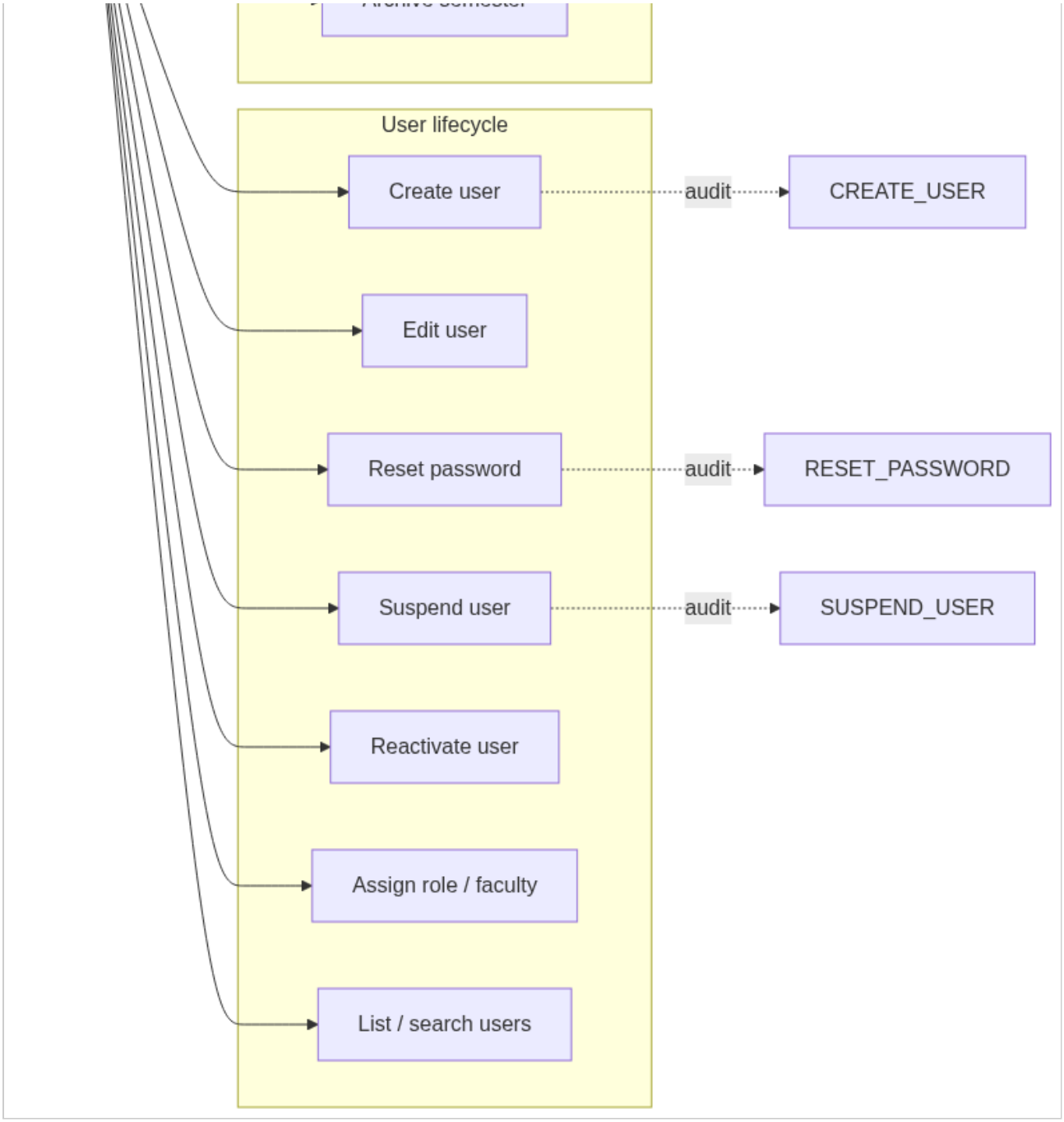


sequence-diagrams-05


Admin
Frontend
Backend
MySQL
Open /admin/analytics
GET /admin/analytics
SELECT count(users) GROUP BY role, status
SELECT count(students) GROUP BY faculty
SELECT sum(invoice.amount), sum(payment.amount)
SELECT count(logins) WHERE created_at > now-30d
SELECT count(messages) WHERE sent_at > now-30d
aggregates
{users, students_by_faculty, finance, engagement}
Render charts
Admin
Frontend
Backend
MySQL

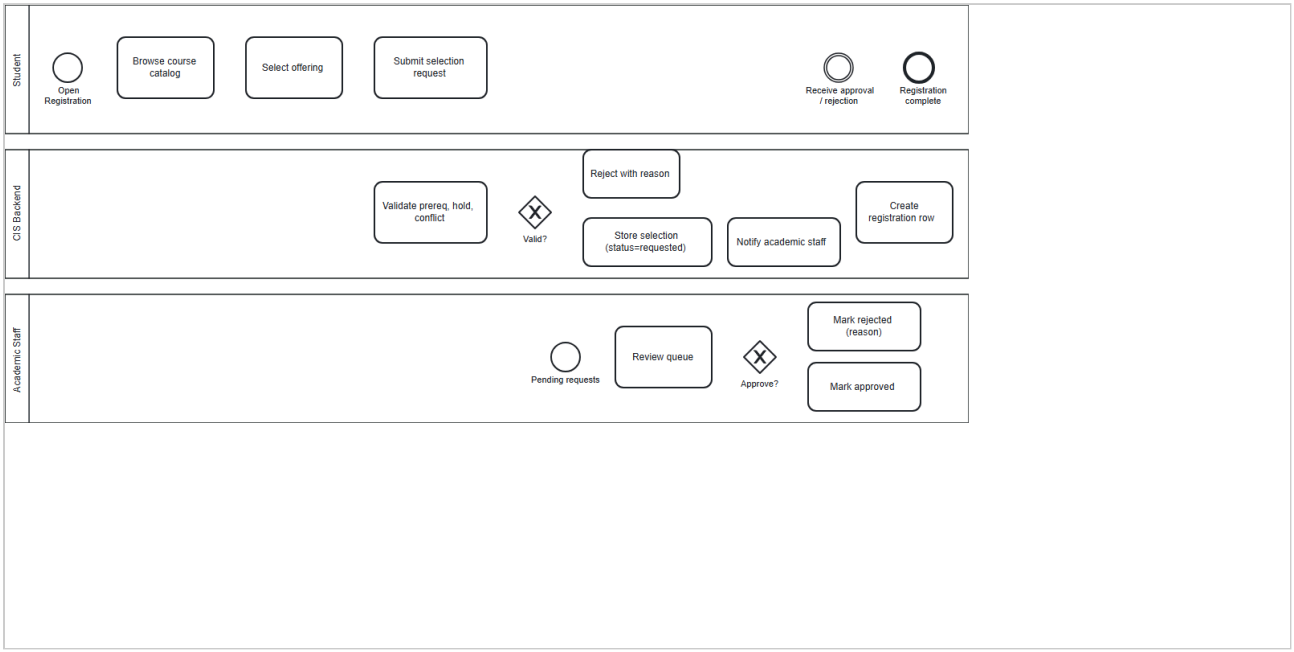
use-cases-01


Communications
Send broadcast
Configuration
Edit system settings
View audit logs
Reporting
View system analytics
Drill into engagement / finance
Catalog
Edit course catalog
Approve registrations parallel authority
Semesters
Create semester
Edit semester windows
Archive semester
System Admin

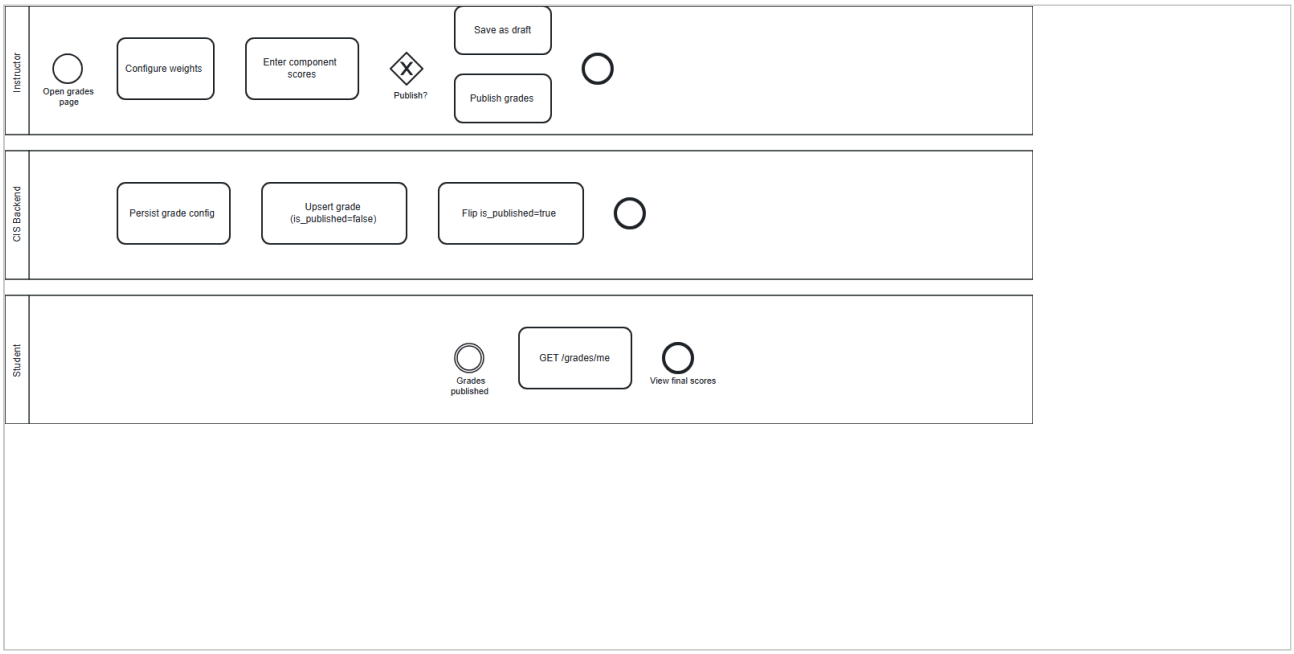

User lifecycle
Create user
audit
CREATE_USER
Edit user
Reset password
audit
RESET_PASSWORD
Suspend user
audit
SUSPEND_USER
Reactivate user
Assign role / faculty
List / search users

Appendix B — BPMN Diagrams

01-course-registration

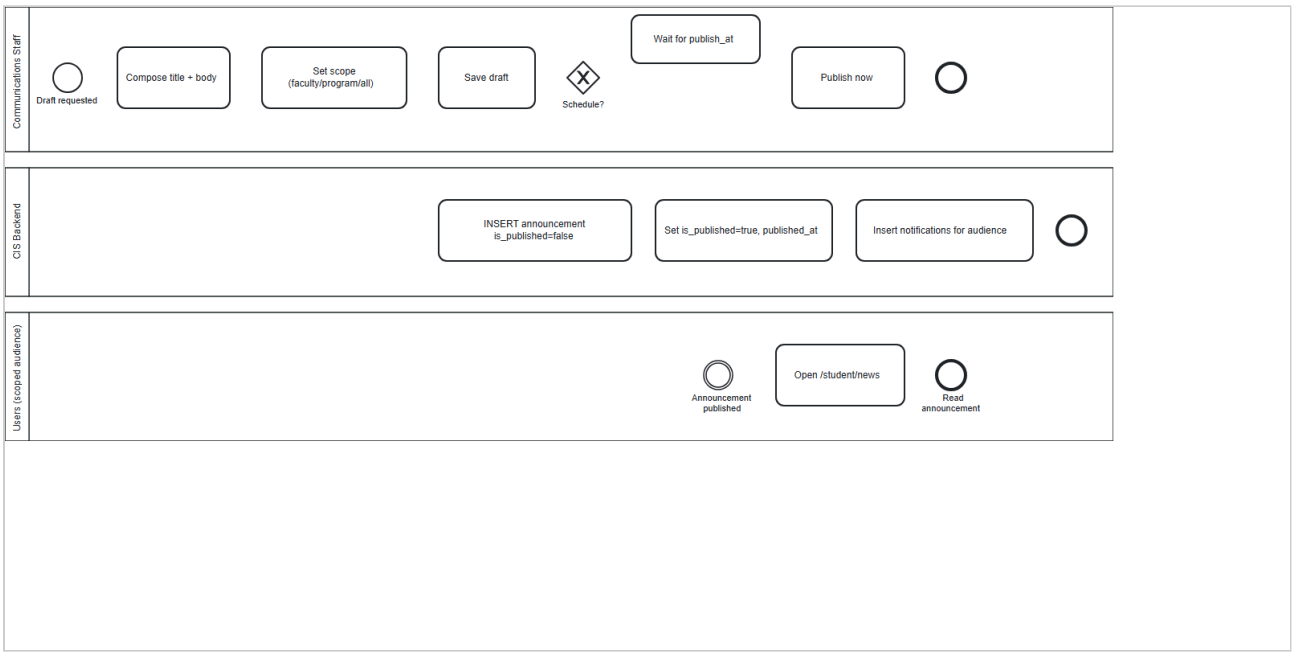


02-grade-publication



03-invoice-payment-hold

04-announcement-broadcast



05-user-onboarding


System Admin
New person admitted
Fill Create User form
Email unique?
Adjust form
Securely share temp password
CIS Backend
Validate email uniqueness
bcrypt(temp_password)
INSERT users + role row + audit
Verify password + JWT
is_first_login?
Return require_password_change=true
Update hash + is_first_login=false
New User
Receive temp pw
Sign in
Change to new password
Workspace access

Appendix C — Entity-Relationship Diagram (full)